使用 Astra Trident

Astra Trident

NetApp
May 20, 2022

This PDF was generated from https://docs.netapp.com/zh-cn/trident/trident-use/anf.html on May 20, 2022. Always check docs.netapp.com for the latest.

目录

使用 Astra Trident . . . 1
配置后端 . . . 1
使用 kubectl 创建后端 . . . 71
使用 kubectl 执行后端管理 . . . 77
使用 tridentctl 执行后端管理 . . . 78
在后端管理选项之间移动 . . . 79
管理存储类 . . . 86
执行卷操作 . . . 88
准备工作节点 . . . 113
自动工作节点准备 . . . 116
监控 Astra Trident . . . 116

使用 Astra Trident

配置后端

后端定义了 Astra Trident 与存储系统之间的关系。它告诉 Astra Trident 如何与该存储系统进行通信，以及 Astra Trident 如何从该存储系统配置卷。Astra Trident 将自动从后端提供符合存储类定义的要求的存储池。了解有关根据您拥有的存储系统类型配置后端的更多信息。

- "配置 Azure NetApp Files 后端"
- "配置适用于 Google 云平台的 Cloud Volumes Service 后端"
- "配置 NetApp HCI 或 SolidFire 后端"
- "使用 ONTAP 或 Cloud Volumes ONTAP NAS 驱动程序配置后端"
- "使用 ONTAP 或 Cloud Volumes ONTAP SAN 驱动程序配置后端"
- "将 Astra Trident 与适用于 NetApp ONTAP 的 Amazon FSX 结合使用"

配置 Azure NetApp Files 后端

了解如何使用提供的示例配置将 Azure NetApp Files （ANF ）配置为 Astra Trident 安装的后端。

> Azure NetApp Files 服务不支持小于 100 GB 的卷。如果请求的卷较小，则 Astra Trident 会自动创建 100 GB 的卷。

配置和使用 "Azure NetApp Files" 后端，您需要满足以下要求：

- `ssubscriptionID` 来自启用了 Azure NetApp Files 的 Azure 订阅。
- 租户 `ID`， `clientID` 和 `clientSecret` 来自 "应用程序注册" 在 Azure Active Directory 中，具有足够的 Azure NetApp Files 服务权限。应用程序注册应使用` Azure 预定义的 `owner` 或 `Contributor… 角色。

  > 要了解有关 Azure 内置角色的详细信息，请参见 "Azure 文档"。

- 至少包含一个的 Azure 位置 "委派子网"。从 Trident 22.01 开始， `location` 参数是后端配置文件顶层的必填字段。在虚拟池中指定的位置值将被忽略。
- 如果您是首次使用 Azure NetApp Files 或在新位置使用，则需要进行一些初始配置。请参见 "《快速入门指南》"。

根据后端配置（子网，虚拟网络，服务级别和位置）， Trident 会在请求位置提供的容量池上创建 ANF 卷，并与请求的服务级别和子网匹配。

> 注意： Astra Trident 不支持手动 QoS 容量池。

后端配置选项

有关后端配置选项，请参见下表：

参数	Description	Default
ve版本		始终为 1
storageDriverName	存储驱动程序的名称	"Azure-netapp-files"
backendName	自定义名称或存储后端	驱动程序名称 + "_" + 随机字符
ssubscriptionID 。	Azure 订阅中的订阅 ID	
租户 ID 。	应用程序注册中的租户 ID	
客户端 ID 。	应用程序注册中的客户端 ID	
clientSecret 。	应用程序注册中的客户端密钥	
s服务级别	S标准 ， 高级 或 超级 之一	"" （随机）
位置	要创建新卷的 Azure 位置的名称	
s服务级别	S标准 ， 高级 或 超级 之一	"" （随机）
resourceGroups	用于筛选已发现资源的资源组列表	"[]" （无筛选器）
netappAccounts	用于筛选已发现资源的 NetApp 帐户列表	"[]" （无筛选器）
容量池	用于筛选已发现资源的容量池列表	"[]" （无筛选器，随机）
virtualNetwork	具有委派子网的虚拟网络的名称	""
ssubnet	委派给 Microsoft.Netapp/volumes 的子网的名称	""
nfsMountOptions	精细控制 NFS 挂载选项。	"nfsvers=3"
limitVolumeSize	如果请求的卷大小超过此值，则配置失败	"" （默认情况下不强制实施）
debugTraceFlags	故障排除时要使用的调试标志。示例，`\ {"api"： false ， "method"： true ， "discovery"： true} `。除非您正在进行故障排除并需要详细的日志转储，否则请勿使用此功能。	空

如果在尝试创建 PVC 时遇到 " 未找到容量池 " 错误，则您的应用程序注册可能没有关联的所需权限和资源（子网，虚拟网络，容量池）。启用调试后， Astra Trident 将记录在创建后端时发现的 Azure 资源。请务必检查是否正在使用适当的角色。

如果要使用 NFS 4.1 挂载卷，可以在逗号分隔的挂载选项列表中包含 nfsver=4 以选择 NFS v4.1 。存储类中设置的任何挂载选项都会覆盖后端配置文件中设置的挂载选项。

可以使用短名称或完全限定名称指定 resourceGroups ， netappAccounts ， capacityPools ， virtualNetwork 和 ssubnet 的值。短名称可以与多个同名资源匹配，因此在大多数情况下，建议使用完全限定名称。resourceGroups ， netappAccounts 和 capacityPools 值是一种筛选器，可将发现的一组资源限制为此存储后端提供的资源，并可通过任意组合方式指定。完全限定名称的格式如下：

Type	格式。
Resource group	< 资源组 >
NetApp 帐户	< 资源组 >/< NetApp 帐户 >
容量池	< 资源组 >/< NetApp 帐户 >/< 容量池 >
虚拟网络	< 资源组 >/< 虚拟网络 >
Subnet	< 资源组 >/< 虚拟网络 >/< 子网 >

您可以通过在配置文件的特殊部分中指定以下选项来控制默认配置每个卷的方式。请参见以下配置示例。

参数	Description	Default
`exportRule`	新卷的导出规则	"0.0.0.0/0
`snapshotDir`	控制 .snapshot 目录的可见性	false
`s`大小	新卷的默认大小	"100 克 "
`unixPermissions`	新卷的 UNIX 权限（ 4 个八进制数字）	"" （预览功能，需要在订阅中列入白名单）

`exportRule` 值必须是以 CIDR 表示法表示的 IPv4 地址或 IPv4 子网任意组合的逗号分隔列表。

> 对于在 ANF 后端创建的所有卷， Astra Trident 会在配置存储池时将存储池上的所有标签复制到该存储卷。存储管理员可以为每个存储池定义标签，并对存储池中创建的所有卷进行分组。这样，您就可以根据后端配置中提供的一组可自定义标签来方便地区分卷了。

示例 **1**： 最低配置

这是绝对的最低后端配置。使用此配置， Astra Trident 会发现在已配置位置委派给 ANF 的所有 NetApp 帐户，容量池和子网，并随机将新卷放置在其中一个池和子网上。

当您刚开始使用 ANF 并尝试执行相关操作时，此配置是理想的选择，但实际上，您希望为所配置的卷提供更多范围界定。

```
{
    "version": 1,
    "storageDriverName": "azure-netapp-files",
    "subscriptionID": "9f87c765-4774-fake-ae98-a721add45451",
    "tenantID": "68e4f836-edc1-fake-bff9-b2d865ee56cf",
    "clientID": "dd043f63-bf8e-fake-8076-8de91e5713aa",
    "clientSecret": "SECRET",
    "location": "eastus"
}
```

示例 **2**： 使用容量池筛选器的特定服务级别配置

此后端配置会将卷放置在 Azure 的 `Easus` 位置的 **超高** 容量池中。Astra Trident 会自动发现该位置委派给 ANF

的所有子网，并随机在其中一个子网上放置一个新卷。

```
    {
        "version": 1,
        "storageDriverName": "azure-netapp-files",
        "subscriptionID": "9f87c765-4774-fake-ae98-a721add45451",
        "tenantID": "68e4f836-edc1-fake-bff9-b2d865ee56cf",
        "clientID": "dd043f63-bf8e-fake-8076-8de91e5713aa",
        "clientSecret": "SECRET",
        "location": "eastus",
        "serviceLevel": "Ultra",
        "capacityPools": [
            "application-group-1/account-1/ultra-1",
            "application-group-1/account-1/ultra-2"
],
    }
```

示例 **3**： 高级配置

此后端配置进一步将卷放置范围缩小为一个子网，并修改了某些卷配置默认值。

```
    {
        "version": 1,
        "storageDriverName": "azure-netapp-files",
        "subscriptionID": "9f87c765-4774-fake-ae98-a721add45451",
        "tenantID": "68e4f836-edc1-fake-bff9-b2d865ee56cf",
        "clientID": "dd043f63-bf8e-fake-8076-8de91e5713aa",
        "clientSecret": "SECRET",
        "location": "eastus",
        "serviceLevel": "Ultra",
        "capacityPools": [
            "application-group-1/account-1/ultra-1",
            "application-group-1/account-1/ultra-2"
],
        "virtualNetwork": "my-virtual-network",
        "subnet": "my-subnet",
        "nfsMountOptions": "vers=3,proto=tcp,timeo=600",
        "limitVolumeSize": "500Gi",
        "defaults": {
            "exportRule": "10.0.0.0/24,10.0.1.0/24,10.0.2.100",
            "snapshotDir": "true",
            "size": "200Gi",
            "unixPermissions": "0777"
=======
        }
    }
```

示例 **4**： 虚拟存储池配置

此后端配置可在一个文件中定义多个存储池。如果您有多个容量池支持不同的服务级别，并且您希望在 Kubernetes 中创建表示这些服务级别的存储类，则此功能非常有用。

```
{
    "version": 1,
    "storageDriverName": "azure-netapp-files",
    "subscriptionID": "9f87c765-4774-fake-ae98-a721add45451",
    "tenantID": "68e4f836-edc1-fake-bff9-b2d865ee56cf",
    "clientID": "dd043f63-bf8e-fake-8076-8de91e5713aa",
    "clientSecret": "SECRET",
    "location": "eastus",
    "resourceGroups": ["application-group-1"],
    "nfsMountOptions": "vers=3,proto=tcp,timeo=600",
    "labels": {
        "cloud": "azure"
    },
    "location": "eastus",

    "storage": [
        {
            "labels": {
                "performance": "gold"
            },
            "serviceLevel": "Ultra",
            "capacityPools": ["ultra-1", "ultra-2"]
        },
        {
            "labels": {
                "performance": "silver"
            },
            "serviceLevel": "Premium",
            "capacityPools": ["premium-1"]
        },
        {
            "labels": {
                "performance": "bronze"
            },
            "serviceLevel": "Standard",
            "capacityPools": ["standard-1", "standard-2"]
        }
    ]
}
```

以下 S存储类 定义是指上述存储池。通过使用 `parameters.selector` 字段，您可以为每个 `StorageClass` 指定用于托管卷的实际池。卷将在选定池中定义各个方面。

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: gold
provisioner: csi.trident.netapp.io
parameters:
  selector: "performance=gold"
allowVolumeExpansion: true
---
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: silver
provisioner: csi.trident.netapp.io
parameters:
  selector: "performance=silver"
allowVolumeExpansion: true
---
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: bronze
provisioner: csi.trident.netapp.io
parameters:
  selector: "performance=bronze"
allowVolumeExpansion: true
```

下一步是什么?

创建后端配置文件后，运行以下命令：

```
tridentctl create backend -f <backend-file>
```

如果后端创建失败，则后端配置出现问题。您可以运行以下命令来查看日志以确定发生原因：

```
tridentctl logs
```

确定并更正配置文件中的问题后，您可以再次运行 create 命令。

配置 Astra 数据存储后端

了解如何使用提供的示例配置为 Astra Trident 安装配置 Astra 数据存储（ADS）后端。

以配置和使用 "Astra 数据存储" 后端，您需要满足以下要求：

- 支持的 ADS 存储系统。请参见 "Astra Data Store 预览文档" 了解详细信息。
- 托管 ADS 的 Kubernetes 集群的凭据。托管 ADS 的 Kubernetes 集群必须具有一个专用于此 Astra Trident 后端创建和管理的卷，快照和导出策略资源的命名空间。必须为托管支持以下功能的 ADS 的 Kubernetes 集群提供 `kubeconfig`：
  - 读取 `astrads-system` 命名空间中的所有对象
  - 读取 / 写入为该 Astra Trident 后端使用而创建的命名空间中的对象
  - 列出所有集群命名空间

后端配置选项

有关后端配置选项，请参见下表：

参数	Description	Default
`ve`版本		始终为 1
`storageDriverName`	存储驱动程序的名称	"astrads-NAS"
`backendName`	自定义名称或存储后端	ADS 集群名称
集群	AstraDSCluster 资源的名称	
命名空间	Astra Trident 将在其中创建所有 ADS 自定义资源的命名空间	
`kubeconfig`	ADS Kubernetes 集群的凭据（Base64 Compact JSON）	`nfsMountOptions`
精细控制 NFS 挂载选项	"vers=4.1"	`autosExportPolicy`
启用自动创建和更新导出策略 [布尔值]	false	`autosExportCIDR`
启用 `AutoExportPolicy` 时用于筛选 Kubernetes 节点 IP 的 CIDR 列表	["0.0.0.0/0， "：： /0"	`limitVolumeSize`
如果请求的卷大小超过此值，则配置失败	"" （默认情况下不强制实施）	`debugTraceFlags`
故障排除时要使用的调试标志。示例： { "api"： false， "method： true }	空	标签

请勿使用 `debugTraceFlags`，除非您正在进行故障排除并需要详细的日志转储。

您应先将 `kubeconfig` 值从 YAML 转换为紧凑型 JSON 格式，然后再转换为 Base64 格式，然后再将其包含在后端配置中。

每个后端都会在托管 ADS 的 Kubernetes 集群上的一个命名空间中配置卷。要在其他命名空间中创建卷，您可以定义其他后端。可以将 ADS 卷附加到托管集群中的任何命名空间，任何其他 Kubernetes 集群或可以挂载 NFS 共享的任何其他位置。

您可以在配置文件的特殊部分中使用这些选项控制每个卷的默认配置。

请参见以下配置示例。

参数	Description	Default
`exportPolicy`	要使用的导出策略	default
`unixPermissions`	新卷的模式必须为八进制，并且以 "0" 开头	"0777"
`sSnapshot` 预留	为快照预留的卷百分比	"5"
`snapshotDir`	控制 ` .snapshot` 目录的可见性	false
`qosPolicy`	要为创建的卷分配的 QoS 策略	""

> 对于在 ADS 后端创建的所有卷， Astra Trident 会在配置存储池时将存储池上的所有标签复制到该存储卷。存储管理员可以为每个存储池定义标签，并对每个存储池创建的所有卷进行分组。这样，您就可以根据后端配置文件中提供的一组可自定义标签来方便地区分卷了。

示例 **1**： 最小后端配置

这是绝对的最低后端配置。

示例 **1.** 示例


```
{
    "version": 1,
    "storageDriverName": "astrads-nas",
    "cluster": "astrads-sti-c6220-09-10-11-12",
    "namespace": "test",
    "kubeconfig":
"eyJjdXJyZW50LWNvbnRleHQiOiJmZWRlcmFsLWNvbnRleHQiLCJhcGlWZXJzaW9uIjoidj
EiLCJjbHVzdGVycyI6W3siY2x1c3RlciI6eyJhcGktdmVyc2lvbiI6InYxIiwic2VydmVyI
joiaHR0cDovL2Nvdy5vcmc6ODA4MCJ9LCJuYW1lIjoiY293LWNsdXN0ZXIifSx7ImNsdXN0
ZXIiOnsiY2VydGlmaWNhdGUtYXV0aG9yaXR5IjoicGF0aC90by9teS9jYWZpbGUiLCJzZXJ
2ZXIiOiJodHRwczovL2hvcnNlLm9yZzo0NDQzIn0sIm5hbWUiOiJob3JzZS1jbHVzdGVyIn
0seyJjbHVzdGVyIjp7Imluc2VjdXJlLXNraXAtdGxzLXZlcmlmeSI6dHJ1ZSwic2VydmVyI
joiaHR0cHM6Ly9waWcub3JnOjQ0MyJ9LCJuYW1lIjoicGlnLWNsdXN0ZXIifV0sImNvbnRl
eHRzIjpbeyJjb250ZXh0Ijp7ImNsdXN0ZXIiOiJob3JzZS1jbHVzdGVyIiwibmFtZXNwYWN
lIjoiY2hpc2VsLW5zIiwidXNlciI6ImdyZWVuLXVzZXIifSwibmFtZSI6ImZlZGVyYWwtY2
9udGV4dCJ9LHsiY29udGV4dCI6eyJjbHVzdGVyIjoicGlnLWNsdXN0ZXIiLCJuYW1lc3BhY
2UiOiJzYXctbnMiLCJ1c2VyIjoiYmxhY2stdXNlciJ9LCJuYW1lIjoicXVlZW4tYW5uZS1j
b250ZXh0In1dLCJraW5kIjoiQ29uZmlnIiwicHJlZmVyZW5jZXMiOnsiY29sb3JzIjp0cnV
lfSwidXNlcnMiOlt7Im5hbWUiOiJibHVlLXVzZXIiLCJ1c2VyIjp7InRva2VuIjoiYmx1ZS
10b2tlbiJ9fSx7Im5hbWUiOiJncmVlbi11c2VyIiwidXNlciI6eyJjbGllbnQtY2VydGlma
WNhdGUiOiJwYXRoL3RvL215L2NsaWVudC9jZXJ0IiwiY2xpZW50LWtleSI6InBhdGgvdG8v
bXkvY2xpZW50L2tleSJ9fV19"
}
```


示例 **2**： 单服务级别配置

此示例显示了一个后端文件，该文件对所有由 Astra Trident 创建的存储应用相同的方面。

示例 **2.** 示例


```
{
    "version": 1,
        "storageDriverName": "astrads-nas",
        "cluster": "astrads-sti-c6220-09-10-11-12",
        "namespace": "test",
        "kubeconfig":
"eyJjdXJyZW50LWNvbnRleHQiOiJmZWRlcmFsLWNvbnRleHQiLCJhcGlWZXJzaW9uIjoidj
EiLCJjbHVzdGVycyI6W3siY2x1c3RlciI6eyJhcGktdmVyc2lvbiI6InYxIiwic2VydmVyI
joiaHR0cDovL2Nvdy5vcmc6ODA4MCJ9LCJuYW1lIjoiY293LWNsdXN0ZXIifSx7ImNsdXN0
ZXIiOnsiY2VydGlmaWNhdGUtYXV0aG9yaXR5IjoicGF0aC90by9teS9jYWZpbGUiLCJzZXJ
2ZXIiOiJodHRwczovL2hvcnNlLm9yZzo0NDQzIn0sIm5hbWUiOiJob3JzZS1jbHVzdGVyIn
0seyJjbHVzdGVyIjp7Imluc2VjdXJlLXNraXAtdGxzLXZlcmlmeSI6dHJ1ZSwic2VydmVyI
joiaHR0cHM6Ly9waWcub3JnOjQ0MyJ9LCJuYW1lIjoicGlnLWNsdXN0ZXIifV0sImNvbnRl
eHRzIjpbeyJjb250ZXh0Ijp7ImNsdXN0ZXIiOiJob3JzZS1jbHVzdGVyIiwibmFtZXNwYWN
lIjoiY2hpc2VsLW5zIiwidXNlciI6ImdyZWVuLXVzZXIifSwibmFtZSI6ImZlZGVyYWwtY2
9udGV4dCJ9LHsiY29udGV4dCI6eyJjbHVzdGVyIjoicGlnLWNsdXN0ZXIiLCJuYW1lc3BhY
2UiOiJzYXctbnMiLCJ1c2VyIjoiYmxhY2stdXNlciJ9LCJuYW1lIjoicXVlZW4tYW5uZS1j
b250ZXh0In1dLCJraW5kIjoiQ29uZmlnIiwicHJlZmVyZW5jZXMiOnsiY29sb3JzIjp0cnV
lfSwidXNlcnMiOlt7Im5hbWUiOiJibHVlLXVzZXIiLCJ1c2VyIjp7InRva2VuIjoiYmx1ZS
10b2tlbiJ9fSx7Im5hbWUiOiJncmVlbi11c2VyIiwidXNlciI6eyJjbGllbnQtY2VydGlma
WNhdGUiOiJwYXRoL3RvL215L2NsaWVudC9jZXJ0IiwiY2xpZW50LWtleSI6InBhdGgvdG8v
bXkvY2xpZW50L2tleSJ9fV19",
        "defaults": {
            "exportPolicy": "myexportpolicy1",
            "qosPolicy": "bronze",
            "snapshotReserve": "10"
        },
        "labels": {"cloud": "on-prem", "creator": "ads-cluster-1",
"performance": "bronze"}
}
```


示例 **3**： 虚拟存储池配置

此示例显示了使用虚拟存储池配置的后端定义文件以及引用这些池的 StorageClasses 。

示例 **3.** 示例


```
{
    "version": 1,
    "storageDriverName": "astrads-nas",
    "cluster": "astrads-sti-c6220-09-10-11-12",
    "namespace": "test",
    "kubeconfig":
"eyJjdXJyZW50LWNvbnRleHQiOiJmZWRlcmFsLWNvbnRleHQiLCJhcGlWZXJzaW9uIjoidj
EiLCJjbHVzdGVycyI6W3siY2x1c3RlciI6eyJhcGktdmVyc2lvbiI6InYxIiwic2VydmVyI
joiaHR0cDovL2Nvdy5vcmc6ODA4MCJ9LCJuYW1lIjoiY293LWNsdXN0ZXIifSx7ImNsdXN0
ZXIiOnsiY2VydGlmaWNhdGUtYXV0aG9yaXR5IjoicGF0aC90by9teS9jYWZpbGUiLCJzZXJ
2ZXIiOiJodHRwczovL2hvcnNlLm9yZzo0NDQzIn0sIm5hbWUiOiJob3JzZS1jbHVzdGVyIn
0seyJjbHVzdGVyIjp7Imluc2VjdXJlLXNraXAtdGxzLXZlcmlmeSI6dHJ1ZSwic2VydmVyI
joiaHR0cHM6Ly9waWcub3JnOjQ0MyJ9LCJuYW1lIjoicGlnLWNsdXN0ZXIifV0sImNvbnRl
eHRzIjpbeyJjb250ZXh0Ijp7ImNsdXN0ZXIiOiJob3JzZS1jbHVzdGVyIiwibmFtZXNwYWN
lIjoiY2hpc2VsLW5zIiwidXNlciI6ImdyZWVuLXVzZXIifSwibmFtZSI6ImZlZGVyYWwtY2
9udGV4dCJ9LHsiY29udGV4dCI6eyJjbHVzdGVyIjoicGlnLWNsdXN0ZXIiLCJuYW1lc3BhY
2UiOiJzYXctbnMiLCJ1c2VyIjoiYmxhY2stdXNlciJ9LCJuYW1lIjoicXVlZW4tYW5uZS1j
b250ZXh0In1dLCJraW5kIjoiQ29uZmlnIiwicHJlZmVyZW5jZXMiOnsiY29sb3JzIjp0cnV
lfSwidXNlcnMiOlt7Im5hbWUiOiJibHVlLXVzZXIiLCJ1c2VyIjp7InRva2VuIjoiYmx1ZS
10b2tlbiJ9fSx7Im5hbWUiOiJncmVlbi11c2VyIiwidXNlciI6eyJjbGllbnQtY2VydGlma
WNhdGUiOiJwYXRoL3RvL215L2NsaWVudC9jZXJ0IiwiY2xpZW50LWtleSI6InBhdGgvdG8v
bXkvY2xpZW50L2tleSJ9fV19",

    "autoExportPolicy": true,
    "autoExportCIDRs": ["10.211.55.0/24"],

    "labels": {"cloud": "on-prem", "creator": "ads-cluster-1"},
    "defaults": {"snapshotReserve": "5"},

    "storage": [
        {
            "labels": {"performance": "gold", "cost": "3"},
            "defaults": {
                "qosPolicy": "gold",
                "snapshotReserve": "10"
            }
        },
        {
            "labels": {"performance": "silver", "cost": "2"},
            "defaults": {"qosPolicy": "silver"}
        },
        {
            "labels": {"performance": "bronze", "cost": "1"},
            "defaults": {"qosPolicy": "bronze"}
```

```
            }
        ]
}
```

以下 StorageClass 定义引用了上述存储池。通过使用 `parameters.selector` 字段，您可以为每个 StorageClass 指定用于托管卷的虚拟池。卷将在选定虚拟池中定义各个方面。

示例 **4.** 示例

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: ads
provisioner: csi.trident.netapp.io
parameters:
  backendType: astrads-nas
allowVolumeExpansion: true

---
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: ads-gold
provisioner: csi.trident.netapp.io
parameters:
  backendType: astrads-nas
  selector: performance=gold
allowVolumeExpansion: true

---
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: ads-silver
provisioner: csi.trident.netapp.io
parameters:
  backendType: astrads-nas
  selector: performance=silver
allowVolumeExpansion: true

---
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: ads-bronze
provisioner: csi.trident.netapp.io
parameters:
  backendType: astrads-nas
  selector: performance=bronze
allowVolumeExpansion: true
```

下一步是什么？

创建后端配置文件后，运行以下命令：

```
tridentctl create backend -f <backend-file>
```

如果后端创建失败，则后端配置出现问题。您可以运行以下命令来查看日志以确定发生原因：

```
tridentctl logs
```

确定并更正配置文件中的问题后，您可以再次运行 create 命令。

为 GCP 后端配置 CVS

了解如何使用提供的示例配置将适用于 Google 云平台（GCP）的 NetApp Cloud Volumes Service （CVS）配置为 Astra Trident 安装的后端。

> NetApp Cloud Volumes Service for Google Cloud 不支持大小小于 100 GiB 的 CVS-Performance 卷或小于 300 GiB 的 CVS 卷。如果请求的卷小于最小大小，则 Astra Trident 会自动创建最小大小的卷。

以配置和使用 "适用于 Google Cloud 的 Cloud Volumes Service" 后端，您需要满足以下要求：

- 配置了 NetApp CVS 的 Google Cloud 帐户
- Google Cloud 帐户的项目编号
- 具有 `netappcloudvolumes.admin` 角色的 Google Cloud 服务帐户
- CVS 服务帐户的 API 密钥文件

Astra Trident 现在支持使用默认值的较小卷 "GCP 上的 CVS 服务类型"。对于使用 `storageClass=software` 创建的后端，卷的最小配置大小将为 300 GiB 。CVS 目前在 " 受控可用性 " 下提供此功能，不提供技术支持。用户必须注册才能访问低于 1TiB 的卷 "此处"。NetApp 建议客户对非生产工作负载使用 1 TiB 以下的卷。

> 使用默认 CVS 服务类型（`storageClass=software`）部署后端时，用户必须获得 GCP 上有关项目编号和项目 ID 的 1 TiB 卷功能的访问权限。这对于 Astra Trident 配置低于 1TiB 的卷是必需的。否则，对于小于 600 GiB 的 PVC ，卷创建将失败。使用访问低于 1TiB 的卷 "此表单"。

由 Astra Trident 为默认 CVS 服务级别创建的卷将按以下方式进行配置：

- 小于 300 GiB 的 PVC 将导致 Astra Trident 创建 300 GiB CVS 卷。
- 如果 PVC 介于 300 GiB 到 600 GiB 之间，则 Astra Trident 将创建请求大小的 CVS 卷。
- 介于 600 GiB 和 1 TiB 之间的 PVC 将导致 Astra Trident 创建 1 TiB CVS 卷。
- 如果 PVC 大于 1 TiB ，则 Astra Trident 将创建请求大小的 CVS 卷。

后端配置选项

有关后端配置选项，请参见下表：

参数	Description	Default
ve版本		始终为 1
storageDriverName	存储驱动程序的名称	"GCP-CVS"
backendName	自定义名称或存储后端	驱动程序名称 + "_" + API 密钥的一部分
s存储类	存储类型可选择 硬件 （性能优化）或 s软件 （ CVS 服务类型）	
projectNumber	Google Cloud 帐户项目编号。该值可在 Google Cloud 门户的主页页面上找到。	
区域	CVS 帐户区域。后端将在该区域配置卷。	
apiKey	具有 netappcloudvolumes.admin 角色的 Google Cloud 服务帐户的 API 密钥。它包括 Google Cloud 服务帐户专用密钥文件的 JSON 格式的内容（逐字复制到后端配置文件）。	
代理 URL	代理服务器需要连接到 CVS 帐户时的代理 URL 。代理服务器可以是 HTTP 代理，也可以是 HTTPS 代理。对于 HTTPS 代理，将跳过证书验证，以允许在代理服务器中使用自签名证书。不支持启用了身份验证的代理服务器。	
nfsMountOptions	精细控制 NFS 挂载选项。	"nfsvers=3"
limitVolumeSize	如果请求的卷大小超过此值，则配置失败	"" （默认情况下不强制实施）
s服务级别	新卷的 CVS 服务级别。这些值包括 "standard" ， "premer" 和 "Extreme " 。	标准
debugTraceFlags	故障排除时要使用的调试标志。示例，`\ ｛"api"： false ， "method： true"｝`。除非您正在进行故障排除并需要详细的日志转储，否则请勿使用此功能。	空

如果使用共享 VPC 网络，则必须同时指定 projectNumber 和 hostProjectNumber 。在这种情况下，projectNumber 是服务项目，而 hostProjectNumber 是主机项目。

"apiRegion"` 表示 Astra Trident 在其中创建 CVS 卷的 GCP 区域。 在 `跨区域 Kubernetes 集群时，在 "apiRegion" 中创建的 CVS 卷可用于在多个 GCP 区域的节点上计划的工作负载。请注意，跨区域流量会产

生额外成本。

- 要启用跨区域访问， `allowedTopologies` 的 StorageClass 定义必须包括所有地区。例如：

```
- key: topology.kubernetes.io/region
  values:
  - us-east1
  - europe-west1
```

- `storageClass` 是一个可选参数，您可以使用该参数选择所需的 "CVS 服务类型"。您可以从基本 CVS 服务类型（ `storageClass=software` ）或 CVS-Performance 服务类型（ `storageClass=hardware` ）中进行选择， Trident 默认使用此服务类型。请 指定一个 `"apiRegion"` ，用于在后端定义中提供相应的 CVS `storageClass` 。

Astra Trident 与 Google Cloud 上的基本 CVS 服务类型集成是一项 * 测试版功能 * ，不适用于生产工作负载。在 CVS-Performance 服务类型中， Trident 是 "* 完全支持 "* ，默认情况下会使用它。

每个后端都会在一个 Google Cloud 区域中配置卷。要在其他区域创建卷，您可以定义其他后端。

您可以通过在配置文件的特殊部分中指定以下选项来控制默认配置每个卷的方式。请参见以下配置示例。

参数	Description	Default
`exportRule`	新卷的导出规则	"0.0.0.0/0
`snapshotDir`	访问 `.snapshot` 目录	false
`sSnapshot` 预留	为快照预留的卷百分比	"" （接受 CVS 默认值为 0 ）
s大小	新卷的大小	"100Gi"

`exportRule` 值必须是以 CIDR 表示法表示的 IPv4 地址或 IPv4 子网任意组合的逗号分隔列表。

对于在 CVS Google Cloud 后端创建的所有卷， Trident 会在配置存储池时将其上的所有标签复制到该存储卷。存储管理员可以为每个存储池定义标签，并对存储池中创建的所有卷进行分组。这样，您就可以根据后端配置中提供的一组可自定义标签来方便地区分卷了。

示例 **1** ： 最低配置

这是绝对的最低后端配置。

```
{
    "version": 1,
    "storageDriverName": "gcp-cvs",
    "projectNumber": "012345678901",
    "apiRegion": "us-west2",
    "apiKey": {
```

```
        "type": "service_account",
        "project_id": "my-gcp-project",
        "private_key_id": "1234567890123456789012345678901234567890",
        "private_key": "-----BEGIN PRIVATE KEY-----
\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZ
srrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisI
sAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSa
PIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZN
chRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlz
ZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl
/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kw
s8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY
9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHc
zZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHi
sIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOgu
SaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyA
ZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGz
lzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3
bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4
Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5o
jY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nzn
HczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrt
HisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbO
guSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKe
yAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRA
GzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4j
K3bl/qp8B4Kws8zX5ojY9m\nXsYg6gyxy4zq7OlwWgLwGa==\n-----END PRIVATE
KEY-----\n",
        "client_email": "cloudvolumes-admin-sa@my-gcp-
project.iam.gserviceaccount.com",
        "client_id": "123456789012345678901",
        "auth_uri": "https://accounts.google.com/o/oauth2/auth",
        "token_uri": "https://oauth2.googleapis.com/token",
        "auth_provider_x509_cert_url":
"https://www.googleapis.com/oauth2/v1/certs",
        "client_x509_cert_url":
"https://www.googleapis.com/robot/v1/metadata/x509/cloudvolumes-admin-
sa%40my-gcp-project.iam.gserviceaccount.com"
    }
}
```


示例 **2**： 基本 **CVS** 服务类型配置

此示例显示了使用基本 CVS 服务类型的后端定义，该服务类型适用于通用工作负载，可提供轻 / 中性能以及高区域可用性。

```
{
    "version": 1,
    "storageDriverName": "gcp-cvs",
    "projectNumber": "012345678901",
    "storageClass": "software",
    "apiRegion": "us-east4",
    "apiKey": {
        "type": "service_account",
        "project_id": "my-gcp-project",
        "private_key_id": "1234567890123456789012345678901234567890",
        "private_key": "-----BEGIN PRIVATE KEY-----
\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZ
srrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisI
sAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSa
PIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZN
chRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlz
ZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl
/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kw
s8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY
9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHc
zZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHi
sIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOgu
SaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyA
ZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGz
lzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3
bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4
Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5o
jY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nzn
HczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrt
HisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbO
guSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKe
yAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRA
GzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4j
K3bl/qp8B4Kws8zX5ojY9m\nXsYg6gyxy4zq7OlwWgLwGa==\n-----END PRIVATE
KEY-----\n",
        "client_email": "cloudvolumes-admin-sa@my-gcp-
project.iam.gserviceaccount.com",
        "client_id": "123456789012345678901",
        "auth_uri": "https://accounts.google.com/o/oauth2/auth",
        "token_uri": "https://oauth2.googleapis.com/token",
        "auth_provider_x509_cert_url":
"https://www.googleapis.com/oauth2/v1/certs",
        "client_x509_cert_url":
"https://www.googleapis.com/robot/v1/metadata/x509/cloudvolumes-admin-
sa%40my-gcp-project.iam.gserviceaccount.com"
    }
```

```
}
```

示例 **3**： 单服务级别配置

此示例显示了一个后端文件，该文件对 Google Cloud us-west2 区域中由 Astra Trident 创建的所有存储应用相同的方面。此示例还显示了后端配置文件中使用的 `proxyURL` 。

```
{
    "version": 1,
    "storageDriverName": "gcp-cvs",
    "projectNumber": "012345678901",
    "apiRegion": "us-west2",
    "apiKey": {
        "type": "service_account",
        "project_id": "my-gcp-project",
        "private_key_id": "1234567890123456789012345678901234567890",
        "private_key": "-----BEGIN PRIVATE KEY-----
\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZ
srrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisI
sAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSa
PIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZN
chRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlz
ZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl
/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kw
s8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY
9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHc
zZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHi
sIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOgu
SaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyA
ZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGz
lzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3
bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4
Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5o
jY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nzn
HczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrt
HisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbO
guSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKe
yAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRA
GzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4j
K3bl/qp8B4Kws8zX5ojY9m\nXsYg6gyxy4zq7OlwWgLwGa==\n-----END PRIVATE
KEY-----\n",
        "client_email": "cloudvolumes-admin-sa@my-gcp-
project.iam.gserviceaccount.com",
        "client_id": "123456789012345678901",
        "auth_uri": "https://accounts.google.com/o/oauth2/auth",
```

```
        "token_uri": "https://oauth2.googleapis.com/token",
        "auth_provider_x509_cert_url":
"https://www.googleapis.com/oauth2/v1/certs",
        "client_x509_cert_url":
"https://www.googleapis.com/robot/v1/metadata/x509/cloudvolumes-admin-
sa%40my-gcp-project.iam.gserviceaccount.com"
    },
    "proxyURL": "http://proxy-server-hostname/",
    "nfsMountOptions": "vers=3,proto=tcp,timeo=600",
    "limitVolumeSize": "10Ti",
    "serviceLevel": "premium",
    "defaults": {
        "snapshotDir": "true",
        "snapshotReserve": "5",
        "exportRule": "10.0.0.0/24,10.0.1.0/24,10.0.2.100",
        "size": "5Ti"
    }
}
```

示例 **4**： 虚拟存储池配置

此示例显示了使用虚拟存储池配置的后端定义文件以及引用这些池的 `StorageClasses` 。

在下面显示的示例后端定义文件中，为所有存储池设置了特定的默认值，这些默认值会将 `snapshotReserve` 设置为 5% ，并将 `exportRule` 设置为 0.0.0.0/0 。虚拟存储池在 `s`存储 部分中进行定义。在此示例中，每个存储池都会设置自己的 `serviceLevel` ，而某些池会覆盖默认值。

```
{
    "version": 1,
    "storageDriverName": "gcp-cvs",
    "projectNumber": "012345678901",
    "apiRegion": "us-west2",
    "apiKey": {
        "type": "service_account",
        "project_id": "my-gcp-project",
        "private_key_id": "1234567890123456789012345678901234567890",
        "private_key": "-----BEGIN PRIVATE KEY-----
\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZ
srrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisI
sAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSa
PIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZN
chRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlz
ZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl
/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kw
s8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY
9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHc
```

```
zZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHi
sIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOgu
SaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyA
ZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGz
lzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3
bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4
Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5o
jY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nzn
HczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrt
HisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbO
guSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKe
yAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRA
GzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4j
K3bl/qp8B4Kws8zX5ojY9m\nXsYg6gyxy4zq7OlwWgLwGa==\n-----END PRIVATE
KEY-----\n",
        "client_email": "cloudvolumes-admin-sa@my-gcp-
project.iam.gserviceaccount.com",
        "client_id": "123456789012345678901",
        "auth_uri": "https://accounts.google.com/o/oauth2/auth",
        "token_uri": "https://oauth2.googleapis.com/token",
        "auth_provider_x509_cert_url":
"https://www.googleapis.com/oauth2/v1/certs",
        "client_x509_cert_url":
"https://www.googleapis.com/robot/v1/metadata/x509/cloudvolumes-admin-
sa%40my-gcp-project.iam.gserviceaccount.com"
    },
    "nfsMountOptions": "vers=3,proto=tcp,timeo=600",

    "defaults": {
        "snapshotReserve": "5",
        "exportRule": "0.0.0.0/0"
    },

    "labels": {
        "cloud": "gcp"
    },
    "region": "us-west2",

    "storage": [
        {
            "labels": {
                "performance": "extreme",
                "protection": "extra"
            },
            "serviceLevel": "extreme",
            "defaults": {
```

```
                "snapshotDir": "true",
                "snapshotReserve": "10",
                "exportRule": "10.0.0.0/24"
            }
        },
        {
            "labels": {
                "performance": "extreme",
                "protection": "standard"
            },
            "serviceLevel": "extreme"
        },
        {
            "labels": {
                "performance": "premium",
                "protection": "extra"
            },
            "serviceLevel": "premium",
            "defaults": {
                "snapshotDir": "true",
                "snapshotReserve": "10"
            }
        },

        {
            "labels": {
                "performance": "premium",
                "protection": "standard"
            },
            "serviceLevel": "premium"
        },

        {
            "labels": {
                "performance": "standard"
            },
            "serviceLevel": "standard"
        }
    ]
}
```

以下 StorageClass 定义引用了上述存储池。通过使用 `parameters.selector` 字段，您可以为每个 StorageClass 指定用于托管卷的虚拟池。卷将在选定池中定义各个方面。

第一个 StorageClass （`cvs-ext-protection`）映射到第一个虚拟存储池。这是唯一一个可提供极高性能且 Snapshot 预留为 10% 的池。最后一个 StorageClass （`cvs-extra protection`）调用提供 10% 快照预留

的任何存储池。Astra Trident 决定选择哪个虚拟存储池，并确保满足快照预留要求。

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: cvs-extreme-extra-protection
provisioner: netapp.io/trident
parameters:
  selector: "performance=extreme; protection=extra"
allowVolumeExpansion: true
---
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: cvs-extreme-standard-protection
provisioner: netapp.io/trident
parameters:
  selector: "performance=premium; protection=standard"
allowVolumeExpansion: true
---
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: cvs-premium-extra-protection
provisioner: netapp.io/trident
parameters:
  selector: "performance=premium; protection=extra"
allowVolumeExpansion: true
---
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: cvs-premium
provisioner: netapp.io/trident
parameters:
  selector: "performance=premium; protection=standard"
allowVolumeExpansion: true
---
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: cvs-standard
provisioner: netapp.io/trident
parameters:
  selector: "performance=standard"
allowVolumeExpansion: true
```

```
---
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: cvs-extra-protection
provisioner: netapp.io/trident
parameters:
  selector: "protection=extra"
allowVolumeExpansion: true
```

下一步是什么?

创建后端配置文件后，运行以下命令：

```
tridentctl create backend -f <backend-file>
```

如果后端创建失败，则后端配置出现问题。您可以运行以下命令来查看日志以确定发生原因：

```
tridentctl logs
```

确定并更正配置文件中的问题后，您可以再次运行 create 命令。

配置 NetApp HCI 或 SolidFire 后端

了解如何在 Astra Trident 安装中创建和使用 Element 后端。

您需要什么？ **#8217** ；将需要什么

- 运行 Element 软件的受支持存储系统。
- NetApp HCI/SolidFire 集群管理员或租户用户的凭据，可用于管理卷。
- 所有 Kubernetes 工作节点都应安装适当的 iSCSI 工具。请参见 "工作节点准备信息"。

`solidfire-san` 存储驱动程序支持两种卷模式： file 和 block 。对于 `filesystem` volumemode ， Astra Trident 将创建一个卷并创建一个文件系统。文件系统类型由 StorageClass 指定。

驱动程序	协议	卷模式	支持的访问模式	支持的文件系统
`solidfire-san`	iSCSI	块	rwo ， rox ， rwx	无文件系统。原始块设备。
`solidfire-san`	iSCSI	块	rwo ， rox ， rwx	无文件系统。原始块设备。
`solidfire-san`	iSCSI	文件系统	工单， ROX	`xfs` ， `ext3` ， `ext4`

驱动程序	协议	卷模式	支持的访问模式	支持的文件系统
solidfire-san	iSCSI	文件系统	工单， ROX	xfs ， ext3 ， ext4

> Astra Trident 在用作增强型 CSI 配置程序时使用 CHAP 。如果您使用的是 CHAP （这是 CSI 的默认设置），则无需进行进一步准备。建议明确设置 UseCHAP 选项，以便对非 CSI Trident 使用 CHAP 。否则，请参见 "此处"。

> 只有适用于 Astra Trident 的传统非 CSI 框架才支持卷访问组。如果配置为在 CSI 模式下运行，则 Astra Trident 将使用 CHAP 。

如果未设置 AccessGroups 或 UseCHAP ，则适用以下规则之一：

- 如果检测到默认的 trident 访问组，则会使用访问组。
- 如果未检测到访问组，并且 Kubernetes 版本为 1.7 或更高版本，则会使用 CHAP 。

后端配置选项

有关后端配置选项，请参见下表：

参数	**Description**	**Default**
ve版本		始终为 1
storageDriverName	存储驱动程序的名称	始终为 "solidfire-san"
backendName	自定义名称或存储后端	SolidFire + 存储（ iSCSI ） IP 地址
端点	使用租户凭据的 SolidFire 集群的 MVIP	
sVIP	存储（ iSCSI ） IP 地址和端口	
标签	要应用于卷的一组任意 JSON 格式的标签。	"
租户名称	要使用的租户名称（如果未找到，则创建）	
InitiatorIFace	将 iSCSI 流量限制为特定主机接口	default
UseCHAP	使用 CHAP 对 iSCSI 进行身份验证	true
访问组	要使用的访问组 ID 列表	查找名为 "trident " 的访问组的 ID
类型	QoS 规范	
limitVolumeSize	如果请求的卷大小超过此值，则配置失败	" （默认情况下不强制实施）
debugTraceFlags	故障排除时要使用的调试标志。示例 ｛"api"： false ， "method "： true｝	空

请勿使用 `debugTraceFlags` ，除非您正在进行故障排除并需要详细的日志转储。

对于创建的所有卷， Astra Trident 会在配置存储池时将存储池上的所有标签复制到备用存储 LUN 。存储管理员可以为每个存储池定义标签，并对存储池中创建的所有卷进行分组。这样，您就可以根据后端配置中提供的一组可自定义标签来方便地区分卷了。

示例 **1**： 具有三种卷类型的 **solidfire-san** 驱动程序的后端配置

此示例显示了一个后端文件，该文件使用 CHAP 身份验证并使用特定 QoS 保证对三种卷类型进行建模。然后，您很可能会使用 `IOPS` storage class 参数定义存储类以使用其中的每种类型。

```
{
    "version": 1,
    "storageDriverName": "solidfire-san",
    "Endpoint": "https://<user>:<password>@<mvip>/json-rpc/8.0",
    "SVIP": "<svip>:3260",
    "TenantName": "<tenant>",
    "labels": {"k8scluster": "dev1", "backend": "dev1-element-cluster"},
    "UseCHAP": true,
    "Types": [{"Type": "Bronze", "Qos": {"minIOPS": 1000, "maxIOPS": 2000,
"burstIOPS": 4000}},
              {"Type": "Silver", "Qos": {"minIOPS": 4000, "maxIOPS": 6000,
"burstIOPS": 8000}},
              {"Type": "Gold", "Qos": {"minIOPS": 6000, "maxIOPS": 8000,
"burstIOPS": 10000}}]
}
```

示例 **2**： 具有虚拟存储池的 **solidfire-san** 驱动程序的后端和存储类配置

此示例显示了使用虚拟存储池配置的后端定义文件以及引用这些池的 StorageClasses 。

在下面所示的示例后端定义文件中，为所有存储池设置了特定的默认值，即将 `type` 设置为 Silver 。虚拟存储池在 `s`**存储** 部分中进行定义。在此示例中，某些存储池设置了自己的类型，而某些池将覆盖上述默认值。

```
{
    "version": 1,
    "storageDriverName": "solidfire-san",
    "Endpoint": "https://<user>:<password>@<mvip>/json-rpc/8.0",
    "SVIP": "<svip>:3260",
    "TenantName": "<tenant>",
    "UseCHAP": true,
    "Types": [{"Type": "Bronze", "Qos": {"minIOPS": 1000, "maxIOPS": 2000,
"burstIOPS": 4000}},
              {"Type": "Silver", "Qos": {"minIOPS": 4000, "maxIOPS": 6000,
"burstIOPS": 8000}},
              {"Type": "Gold", "Qos": {"minIOPS": 6000, "maxIOPS": 8000,
"burstIOPS": 10000}}],

    "type": "Silver",
    "labels":{"store":"solidfire", "k8scluster": "dev-1-cluster"},
    "region": "us-east-1",

    "storage": [
        {
            "labels":{"performance":"gold", "cost":"4"},
            "zone":"us-east-1a",
            "type":"Gold"
        },
        {
            "labels":{"performance":"silver", "cost":"3"},
            "zone":"us-east-1b",
            "type":"Silver"
        },
        {
            "labels":{"performance":"bronze", "cost":"2"},
            "zone":"us-east-1c",
            "type":"Bronze"
        },
        {
            "labels":{"performance":"silver", "cost":"1"},
            "zone":"us-east-1d"
        }
    ]
}
```

以下 StorageClass 定义引用了上述虚拟存储池。使用 `parameters.selector` 字段，每个 StorageClass 都会调用可用于托管卷的虚拟池。卷将在选定虚拟池中定义各个方面。

第一个 StorageClass （ `solidfire-gold-四` ）将映射到第一个虚拟存储池。这是唯一一个性能卓越的池，

其 卷类型 `QoS` 值为金牌。最后一个 StorageClass （ `solidfire-silver` ）调用可提供银牌性能的任何存储池。Astra Trident 将决定选择哪个虚拟存储池，并确保满足存储要求。

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: solidfire-gold-four
provisioner: csi.trident.netapp.io
parameters:
  selector: "performance=gold; cost=4"
  fsType: "ext4"
---
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: solidfire-silver-three
provisioner: csi.trident.netapp.io
parameters:
  selector: "performance=silver; cost=3"
  fsType: "ext4"
---
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: solidfire-bronze-two
provisioner: csi.trident.netapp.io
parameters:
  selector: "performance=bronze; cost=2"
  fsType: "ext4"
---
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: solidfire-silver-one
provisioner: csi.trident.netapp.io
parameters:
  selector: "performance=silver; cost=1"
  fsType: "ext4"
---
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: solidfire-silver
provisioner: csi.trident.netapp.io
parameters:
  selector: "performance=silver"
  fsType: "ext4"
```

了解更多信息

- "卷访问组"

使用 ONTAP 或 Cloud Volumes ONTAP SAN 驱动程序配置后端

了解如何使用 ONTAP 和 Cloud Volumes ONTAP SAN 驱动程序配置 ONTAP 后端。

- "准备"
- "配置和示例"

用户权限

Astra Trident 应以 ONTAP 或 SVM 管理员身份运行，通常使用 `admin` cluster 用户或 `vsadmin` SVM 用户，或者使用具有相同角色的其他名称的用户。对于适用于 NetApp ONTAP 的 Amazon FSX 部署， Astra Trident 应使用集群 `fsxadmin` user 或 `vsadmin` SVM 用户或具有相同角色的其他名称的用户作为 ONTAP 或 SVM 管理员运行。`fsxadmin` 用户是集群管理员用户的有限替代用户。

> 如果使用 `limitAggregateUsage` 参数，则需要集群管理员权限。将适用于 NetApp ONTAP 的 Amazon FSx 与 Astra Trident 结合使用时， `limitAggregateUsage` 参数不适用于 `vsadmin` 和 `fsxadmin` 用户帐户。如果指定此参数，配置操作将失败。

虽然可以在 ONTAP 中创建 Trident 驱动程序可以使用的限制性更强的角色，但我们不建议这样做。大多数新版本的 Trident 都会调用需要考虑的其他 API ，从而使升级变得困难且容易出错。

准备

了解如何准备使用 ONTAP SAN 驱动程序配置 ONTAP 后端。对于所有 ONTAP 后端， Astra Trident 需要至少为 SVM 分配一个聚合。

请记住，您还可以运行多个驱动程序，并创建指向其中一个驱动程序的存储类。例如，您可以配置使用 `ontap-san` 驱动程序的 `san-dev` 类和使用 `ontap-san-economy-one` 的 `san-default` 类。

所有 Kubernetes 工作节点都必须安装适当的 iSCSI 工具。请参见 "此处" 有关详细信息：

身份验证

Astra Trident 提供了两种对 ONTAP 后端进行身份验证的模式。

- Credential Based ：具有所需权限的 ONTAP 用户的用户名和密码。建议使用 `admin` 或 `vsadmin` 等预定义的安全登录角色，以确保与 ONTAP 版本的最大兼容性。
- 基于证书： Astra Trident 还可以使用后端安装的证书与 ONTAP 集群进行通信。此处，后端定义必须包含客户端证书，密钥和可信 CA 证书的 Base64 编码值（如果使用）（建议）。

用户还可以选择更新现有后端，选择从基于凭据迁移到基于证书，反之亦然。如果 * 同时提供了凭据和证书 * ，则 Astra Trident 将在发出警告以从后端定义中删除凭据时默认使用证书。

启用基于凭据的身份验证

Astra Trident 需要 SVM 范围 / 集群范围的管理员的凭据才能与 ONTAP 后端进行通信。建议使用标准的预定义

角色，例如 `admin` 或 `vsadmin` 。这样可以确保与未来的 ONTAP 版本向前兼容，这些版本可能会使功能 API 公开供未来的 Astra Trident 版本使用。可以创建自定义安全登录角色并将其用于 Astra Trident ，但不建议使用。

后端定义示例如下所示：

```
{
  "version": 1,
  "backendName": "ExampleBackend",
  "storageDriverName": "ontap-san",
  "managementLIF": "10.0.0.1",
  "dataLIF": "10.0.0.2",
  "svm": "svm_nfs",
  "username": "vsadmin",
  "password": "secret",
}
```

请注意，后端定义是凭据以纯文本格式存储的唯一位置。创建后端后，用户名 / 密码将使用 Base64 进行编码并存储为 Kubernetes 密钥。创建 / 更新后端是唯一需要了解凭据的步骤。因此，这是一项仅由管理员执行的操作，由 Kubernetes 或存储管理员执行。

启用基于证书的身份验证

新的和现有的后端可以使用证书并与 ONTAP 后端进行通信。后端定义需要三个参数。

- clientCertificate ： 客户端证书的 Base64 编码值。
- clientPrivateKey ： 关联私钥的 Base64 编码值。
- trustedCACertifate ： 受信任 CA 证书的 Base64 编码值。如果使用可信 CA ，则必须提供此参数。如果不使用可信 CA ，则可以忽略此设置。

典型的工作流包括以下步骤。

步骤

1. 生成客户端证书和密钥。生成时，将公用名（ Common Name ， CN ）设置为要作为身份验证的 ONTAP 用户。

   ```
   openssl req -x509 -nodes -days 1095 -newkey rsa:2048 -keyout k8senv.key
   -out k8senv.pem -subj "/C=US/ST=NC/L=RTP/O=NetApp/CN=admin"
   ```

2. 将可信 CA 证书添加到 ONTAP 集群。此问题可能已由存储管理员处理。如果未使用可信 CA ，则忽略。

```
security certificate install -type server -cert-name <trusted-ca-cert-
name> -vserver <vserver-name>
ssl modify -vserver <vserver-name> -server-enabled true -client-enabled
true -common-name <common-name> -serial <SN-from-trusted-CA-cert> -ca
<cert-authority>
```

3. 在 ONTAP 集群上安装客户端证书和密钥（从步骤 1 开始）。

```
security certificate install -type client-ca -cert-name <certificate-
name> -vserver <vserver-name>
security ssl modify -vserver <vserver-name> -client-enabled true
```

4. 确认 ONTAP 安全登录角色支持 `cert` 身份验证方法。

```
security login create -user-or-group-name admin -application ontapi
-authentication-method cert
security login create -user-or-group-name admin -application http
-authentication-method cert
```

5. 使用生成的证书测试身份验证。将 <SVM 管理 LIF> 和 <SVM 名称 > 替换为管理 LIF IP 和 ONTAP 名称。

```
curl -X POST -Lk https://<ONTAP-Management-
LIF>/servlets/netapp.servlets.admin.XMLrequest_filer --key k8senv.key
--cert ~/k8senv.pem -d '<?xml version="1.0" encoding="UTF-8"?><netapp
xmlns="http://www.netapp.com/filer/admin" version="1.21"
vfiler="<vserver-name>"><vserver-get></vserver-get></netapp>'
```

6. 使用 Base64 对证书，密钥和可信 CA 证书进行编码。

```
base64 -w 0 k8senv.pem >> cert_base64
base64 -w 0 k8senv.key >> key_base64
base64 -w 0 trustedca.pem >> trustedca_base64
```

7. 使用从上一步获得的值创建后端。

```
$ cat cert-backend.json
{
"version": 1,
"storageDriverName": "ontap-san",
"backendName": "SanBackend",
"managementLIF": "1.2.3.4",
"dataLIF": "1.2.3.8",
"svm": "vserver_test",
"clientCertificate": "Faaaakkkkeeee...Vaaallllluuuueeee",
"clientPrivateKey": "LS0tFaKE...0VaLuES0tLS0K",
"trustedCACertificate": "QNFinfO...SiqOyN",
"storagePrefix": "myPrefix_"
}

$ tridentctl create backend -f cert-backend.json -n trident
+------------+----------------+--------------------------------------
+--------+---------+
|    NAME    | STORAGE DRIVER |                  UUID                  |
STATE   | VOLUMES |
+------------+----------------+--------------------------------------
+--------+---------+
| SanBackend | ontap-san      | 586b1cd5-8cf8-428d-a76c-2872713612c1 |
online |       0 |
+------------+----------------+--------------------------------------
+--------+---------+
```

更新身份验证方法或轮换凭据

您可以更新现有后端以使用其他身份验证方法或轮换其凭据。这两种方式都适用：使用用户名 / 密码的后端可以更新为使用证书；使用证书的后端可以更新为基于用户名 / 密码的后端。为此，请使用更新后的 `backend.json` 文件，该文件包含执行 `tridentctl` **后端更新** 所需的参数。

```
$ cat cert-backend-updated.json
{
"version": 1,
"storageDriverName": "ontap-san",
"backendName": "SanBackend",
"managementLIF": "1.2.3.4",
"dataLIF": "1.2.3.8",
"svm": "vserver_test",
"username": "vsadmin",
"password": "secret",
"storagePrefix": "myPrefix_"
}

#Update backend with tridentctl
$ tridentctl update backend SanBackend -f cert-backend-updated.json -n
trident
+------------+----------------+--------------------------------------
+--------+---------+
|    NAME    | STORAGE DRIVER |                  UUID                |
STATE  | VOLUMES |
+------------+----------------+--------------------------------------
+--------+---------+
| SanBackend | ontap-san      | 586b1cd5-8cf8-428d-a76c-2872713612c1 |
online |       9 |
+------------+----------------+--------------------------------------
+--------+---------+
```

> 轮换密码时，存储管理员必须先在 ONTAP 上更新用户的密码。然后进行后端更新。轮换证书时，可以向用户添加多个证书。之后，后端将更新以使用新证书，然后可以从 ONTAP 集群中删除旧证书。

更新后端不会中断对已创建卷的访问，也不会影响在之后建立的卷连接。成功的后端更新表明， Astra Trident 可以与 ONTAP 后端进行通信并处理未来的卷操作。

指定 igroup

Astra Trident 使用 igroup 来控制对其配置的卷（ LUN ）的访问。在为后端指定 igroup 时，管理员有两种选择：

- Astra Trident 可以自动为每个后端创建和管理 igroup 。如果后端定义中未包含 `igroupName` ，则 Astra Trident 会在 SVM 上创建一个名为 `trident -<backender-UUUUUID>` 的 igroup 。这将确保每个后端都有一个专用的 igroup ，并处理 Kubernetes 节点 IQN 的自动添加 / 删除。
- 或者，也可以在后端定义中提供预先创建的 igroup 。可以使用 `igroupName` config 参数来执行此操作。Astra Trident 会将 Kubernetes 节点 IQN 添加 / 删除到已有的 igroup 中。

对于已定义 `igroupName` 的后端，可以使用 `tridentctl` **后端更新** 删除 `igroupName` ，以使 Astra Trident 自动处理 igroup 。这样不会中断对已连接到工作负载的卷的访问。未来的连接将使用创建的 igroup Astra

Trident 进行处理。

> 为 Astra Trident 的每个唯一实例指定一个 igroup 是一个最佳实践，对 Kubernetes 管理员和存储管理员都很有用。CSI Trident 可自动向 igroup 添加和删除集群节点 IQN ，从而极大地简化了其管理。在 Kubernetes 环境（以及 Astra Trident 安装）中使用相同的 SVM 时，使用专用的 igroup 可确保对一个 Kubernetes 集群所做的更改不会影响与另一个 Kubernetes 集群关联的 igroup 。此外，还必须确保 Kubernetes 集群中的每个节点都具有唯一的 IQN 。如上所述，Astra Trident 会自动处理 IQN 的添加和删除。在多个主机之间重复使用 IQN 可能会导致出现主机相互错误并拒绝访问 LUN 的不希望出现的情况。

如果将 Astra Trident 配置为充当 CSI 配置程序，则 Kubernetes 节点 IQN 会自动添加到 igroup 中或从 igroup 中删除。将节点添加到 Kubernetes 集群后， `trident - CSI` DemonSet 会在新添加的节点上部署一个 Pod （`trident - CSI - xxxxx` ），并注册可将卷连接到的新节点。节点 IQN 也会添加到后端的 igroup 中。在对节点进行隔离，清空并从 Kubernetes 中删除时，可以执行一组类似的步骤来删除 IQN 。

如果 Astra Trident 未作为 CSI 配置程序运行，则必须手动更新 igroup ，以包含 Kubernetes 集群中每个工作节点的 iSCSI IQN 。需要将加入 Kubernetes 集群的节点的 IQN 添加到 igroup 中。同样，必须从 igroup 中删除从 Kubernetes 集群中删除的节点的 IQN 。

使用双向 **CHAP** 对连接进行身份验证

Astra Trident 可以使用双向 CHAP 对 `ontap-san` 和 `ontap-san-economy-sn` 驱动程序的 iSCSI 会话进行身份验证。这需要在后端定义中启用 `useCHAP` 选项。如果设置为 `true` ，则 Astra Trident 会将 SVM 的默认启动程序安全性配置为双向 CHAP ，并从后端文件设置用户名和密码。NetApp 建议使用双向 CHAP 对连接进行身份验证。请参见以下配置示例：

```
{
    "version": 1,
    "storageDriverName": "ontap-san",
    "backendName": "ontap_san_chap",
    "managementLIF": "192.168.0.135",
    "svm": "ontap_iscsi_svm",
    "useCHAP": true,
    "username": "vsadmin",
    "password": "FaKePaSsWoRd",
    "igroupName": "trident",
    "chapInitiatorSecret": "cl9qxIm36DKyawxy",
    "chapTargetInitiatorSecret": "rqxigXgkesIpwxyz",
    "chapTargetUsername": "iJF4heBRT0TCwxyz",
    "chapUsername": "uh2aNCLSd6cNwxyz",
}
```

> `useCHAP` 参数是一个布尔选项，只能配置一次。默认情况下，此参数设置为 false 。将其设置为 true 后，无法将其设置为 false 。

除了 `useCHAP=true` 之外，后端定义还必须包括 `chapInitiatorSecret` ， `chapTargetInitiatorSecret` ， `chapTargetUsername` 和 `chapUsername` 字段。通过运行 `tridentctl update` 创建后端，可以更改这些密钥。

工作原理

通过将 `useCHAP` 设置为 true ，存储管理员指示 Astra Trident 在存储后端配置 CHAP 。其中包括：

- 在 SVM 上设置 CHAP ：
  - 如果 SVM 的默认启动程序安全类型为 none （默认设置） * 和 * 卷中没有已存在的 LUN ，则 Astra Trident 会将默认安全类型设置为 `CHAP` ，然后继续配置 CHAP 启动程序以及目标用户名和密码。
  - 如果 SVM 包含 LUN ，则 Astra Trident 不会在 SVM 上启用 CHAP 。这样可以确保对 SVM 上已存在的 LUN 的访问不受限制。
- 配置 CHAP 启动程序以及目标用户名和密码；必须在后端配置中指定这些选项（如上所示）。
- 管理向后端提供的 `igroupName` 添加启动程序的操作。如果未指定，则默认为 `trident` 。

创建后端后， Astra Trident 会创建相应的 `tridentbackend` CRD ，并将 CHAP 密码和用户名存储为 Kubernetes 密码。此后端由 Astra Trident 创建的所有 PV 都将通过 CHAP 进行挂载和连接。

轮换凭据并更新后端

您可以通过更新 `backend.json` 文件中的 CHAP 参数来更新 CHAP 凭据。这需要更新 CHAP 密码并使用 `tridentctl update` 命令反映这些更改。

> 更新后端的 CHAP 密码时，必须使用 `tridentctl` 来更新后端。请勿通过 CLI/ONTAP UI 更新存储集群上的凭据，因为 Astra Trident 将无法选取这些更改。

```
$ cat backend-san.json
{
    "version": 1,
    "storageDriverName": "ontap-san",
    "backendName": "ontap_san_chap",
    "managementLIF": "192.168.0.135",
    "svm": "ontap_iscsi_svm",
    "useCHAP": true,
    "username": "vsadmin",
    "password": "FaKePaSsWoRd",
    "igroupName": "trident",
    "chapInitiatorSecret": "cl9qxUpDaTeD",
    "chapTargetInitiatorSecret": "rqxigXgkeUpDaTeD",
    "chapTargetUsername": "iJF4heBRT0TCwxyz",
    "chapUsername": "uh2aNCLSd6cNwxyz",
}

$ ./tridentctl update backend ontap_san_chap -f backend-san.json -n
trident
+----------------+----------------+--------------------------------------
+--------+---------+
|   NAME         | STORAGE DRIVER |                 UUID                 |
STATE   | VOLUMES |
+----------------+----------------+--------------------------------------
+--------+---------+
| ontap_san_chap | ontap-san      | aa458f3b-ad2d-4378-8a33-1a472ffbeb5c |
online |       7 |
+----------------+----------------+--------------------------------------
+--------+---------+
```

现有连接将不受影响；如果凭据由 SVM 上的 Astra Trident 更新，则这些连接将继续保持活动状态。新连接将使用更新后的凭据，现有连接将继续保持活动状态。断开并重新连接旧的 PV 将导致它们使用更新后的凭据。

配置选项和示例

了解如何在您的 Astra Trident 安装中创建和使用 ONTAP SAN 驱动程序。本节提供了后端配置示例以及有关如何将后端映射到 StorageClasses 的详细信息。

后端配置选项

有关后端配置选项，请参见下表：

参数	Description	Default
`ve`版本		始终为 1

参数	Description	Default
storageDriverName	存储驱动程序的名称	"ontap-nas"， "ontap-nas-economy-"， "ontap-nas-flexgroup"， "ontap-san "， "ontap-san-economy-"
backendName	自定义名称或存储后端	驱动程序名称 + "_" + dataLIF
m年 月 日	集群或 SVM 管理 LIF 的 IP 地址	"10.0.0.1 "， "2001： 1234： abcd：：： fefe] "
dataLIF	协议 LIF 的 IP 地址。对于 IPv6，请使用方括号。设置后无法更新	由 SVM 派生，除非另有说明
使用 CHAP	使用 CHAP 对 iSCSI 的 ONTAP SAN 驱动程序进行身份验证 [布尔值]	false
chapInitiatorSecret	CHAP 启动程序密钥。如果为 useCHAP=true，则为必需项	"
标签	要应用于卷的一组任意 JSON 格式的标签	"
chapTargetInitiatorSecret	CHAP 目标启动程序密钥。如果为 useCHAP=true，则为必需项	"
chapUsername	入站用户名。如果为 useCHAP=true，则为必需项	"
chapTargetUsername	目标用户名。如果为 useCHAP=true，则为必需项	"
客户端证书	客户端证书的 Base64 编码值。用于基于证书的身份验证	"
clientPrivateKey	客户端专用密钥的 Base64 编码值。用于基于证书的身份验证	"
trustedCACertifate	受信任 CA 证书的 Base64 编码值。可选。用于基于证书的身份验证	"
用户名	用于连接到集群 /SVM 的用户名。用于基于凭据的身份验证	"
密码	连接到集群 /SVM 的密码。用于基于凭据的身份验证	"
sVM	要使用的 Storage Virtual Machine	如果指定了 SVM managementLIF，则派生
igroupName	要使用的 SAN 卷的 igroup 的名称	"trident — < 后端 UUID >"
s存储前缀	在 SVM 中配置新卷时使用的前缀。设置后无法更新	Trident
limitAggregateUsage	如果使用量超过此百分比，则配置失败。* 不适用于适用于 ONTAP 的 Amazon FSx *	" （默认情况下不强制实施）

参数	Description	Default
limitVolumeSize	如果经济型驱动程序请求的卷大小超过此值，则配置失败。	" （默认情况下不强制实施）
lunsPerFlexvol	每个 FlexVol 的最大 LUN 数，必须在 50 ， 200 范围内	100
debugTraceFlags	故障排除时要使用的调试标志。示例 ｛"api"： false ， "method "： true｝	空
useREST	用于使用 ONTAP REST API 的布尔参数。* 技术预览 *	false

> useREST 提供了一个 * 技术预览 * ，建议用于测试环境，而不是生产工作负载。如果设置为 true ，则 Astra Trident 将使用 ONTAP REST API 与后端进行通信。此功能需要使用 ONTAP 9.9 及更高版本。此外，使用的 ONTAP 登录角色必须能够访问 ONTAP 应用程序。这一点可通过预定义的 vsadmin 和 cluster-admin 角色来满足。

要与 ONTAP 集群通信，您应提供身份验证参数。这可以是安全登录的用户名 / 密码，也可以是已安装的证书。

> 如果您使用适用于 NetApp ONTAP 后端的 Amazon FSX ，请勿指定 limitAggregateUsage 参数。Amazon FSX for NetApp ONTAP 提供的 fsxadmin 和 vsadmin 角色不包含检索聚合使用情况并通过 Astra Trident 对其进行限制所需的访问权限。

> 请勿使用 debugTraceFlags ，除非您正在进行故障排除并需要详细的日志转储。

对于 ontap-san 驱动程序，默认使用 SVM 中的所有数据 LIF IP 并使用 iSCSI 多路径。为 ontap-san 驱动程序的 dataLIF 指定 IP 地址会强制其禁用多路径并仅使用指定的地址。

> 创建后端时，请记住，创建后无法修改 dataLIF 和 storagePrefix 。要更新这些参数，您需要创建一个新的后端。

可以将 igroupName 设置为已在 ONTAP 集群上创建的 igroup 。如果未指定，则 Astra Trident 会自动创建一个名为 trident -<backender-UUUUUID> 的 igroup 。如果要在环境之间共享 SVM ，则如果提供预定义的 igroupName ， NetApp 建议为每个 Kubernetes 集群使用一个 igroup 。这对于 Astra Trident 自动保持 IQN 添加 / 删除是必需的。

后端也可以在创建后更新 igroup ：

- 可以更新 igroupName 以指向在 Astra Trident 之外的 SVM 上创建和管理的新 igroup 。
- 可以省略 igroupName 。在这种情况下， Astra Trident 将自动创建和管理 trident -<backend-UUUUID> igroup 。

在这两种情况下，仍可访问卷附件。未来的卷附件将使用更新后的 igroup 。此更新不会中断对后端卷的访问。

可以为 managementLIF 选项指定完全限定域名 （ FQDN ）。

对于所有 ONTAP 驱动程序，也可以将 managementLIF 设置为 IPv6 地址。请务必使用 ` -use-ipv6` 标志安装 Trident 。必须注意在方括号内定义 managementLIF IPv6 地址。

> 使用 IPv6 地址时，请确保在方括号内定义 `managementLIF` 和 `dataLIF` （如果包含在后端定义中），例如 [28e8： d9fb： a825： b7bf： 69a8： d02f： 9e7b： 3555] 。如果未提供 `dataLIF` ，则 Astra Trident 将从 SVM 提取 IPv6 数据 LIF 。

要使 ontap-san 驱动程序能够使用 CHAP ，请在后端定义中将 `useCHAP` 参数设置为 `true` 。然后， Astra Trident 将配置双向 CHAP 并将其用作后端给定 SVM 的默认身份验证。请参见 "此处" 了解其工作原理。

对于 `ontap-san-economi` 驱动程序， `limitVolumeSize` 选项还会限制它为 qtree 和 LUN 管理的卷的最大大小。

> Astra Trident 会在使用 `ontap-san` 驱动程序创建的所有卷的 "Comments" 字段中设置配置标签。对于创建的每个卷， FlexVol 上的 "Comments" 字段将使用其所在存储池上的所有标签填充。存储管理员可以为每个存储池定义标签，并对存储池中创建的所有卷进行分组。这样，您就可以根据后端配置中提供的一组可自定义标签来方便地区分卷了。

用于配置卷的后端配置选项

您可以在配置的特殊部分中使用这些选项来控制默认配置每个卷的方式。有关示例，请参见以下配置示例。

参数	Description	Default
`spaceAllocation`	LUN 的空间分配	true
`s`页面预留	空间预留模式； " 无 " （精简）或 " 卷 " （厚）	无
`sSnapshot` 策略	要使用的 Snapshot 策略	无
`qosPolicy`	要为创建的卷分配的 QoS 策略组。选择每个存储池 / 后端的 qosPolicy 或 adaptiveQosPolicy 之一	"
`adaptiveQosPolicy`	要为创建的卷分配的自适应 QoS 策略组。选择每个存储池 / 后端的 qosPolicy 或 adaptiveQosPolicy 之一	"
`sSnapshot` 预留	为快照预留的卷百分比为 "0"	如果 `snapshotPolicy` 为 " 无 " ，则为 " 无 " ，否则为 " "
`splitOnClone`	创建克隆时，从其父级拆分该克隆	false
`splitOnClone`	创建克隆时，从其父级拆分该克隆	false
加密	启用 NetApp 卷加密	false
`securityStyle`	新卷的安全模式	"unix"
分层策略	使用 " 无 " 的分层策略	适用于 ONTAP 9.5 SVM-DR 之前的配置的 " 仅快照 "

> 在 Astra Trident 中使用 QoS 策略组需要 ONTAP 9.8 或更高版本。建议使用非共享 QoS 策略组，并确保策略组分别应用于每个成分卷。共享 QoS 策略组将对所有工作负载的总吞吐量实施上限。

下面是定义了默认值的示例：

```
{
 "version": 1,
 "storageDriverName": "ontap-san",
 "managementLIF": "10.0.0.1",
 "dataLIF": "10.0.0.2",
 "svm": "trident_svm",
 "username": "admin",
 "password": "password",
 "labels": {"k8scluster": "dev2", "backend": "dev2-sanbackend"},
 "storagePrefix": "alternate-trident",
 "igroupName": "custom",
 "debugTraceFlags": {"api":false, "method":true},
 "defaults": {
     "spaceReserve": "volume",
     "qosPolicy": "standard",
     "spaceAllocation": "false",
     "snapshotPolicy": "default",
     "snapshotReserve": "10"
 }
}
```

> 对于使用 `ontap-san` 驱动程序创建的所有卷， Astra Trident 会向 FlexVol 额外添加 10% 的容量，以容纳 LUN 元数据。LUN 将使用用户在 PVC 中请求的确切大小进行配置。Astra Trident 将 FlexVol 增加 10% （在 ONTAP 中显示为可用大小）。用户现在将获得所请求的可用容量。此更改还可防止 LUN 变为只读状态，除非已充分利用可用空间。这不适用于 ontap-san-economy.

对于定义 `snapshotReserve` 的后端， Astra Trident 将按如下方式计算卷的大小：

```
Total volume size = [(PVC requested size) / (1 - (snapshotReserve
percentage) / 100)] * 1.1
```

1.1 是 Astra Trident 向 FlexVol 额外添加 10% 以容纳 LUN 元数据。对于 `snapshotReserve` = 5% ， PVC 请求 = 5GiB ，卷总大小为 5.79GiB ，可用大小为 5.5GiB 。`volume show` 命令应显示与以下示例类似的结果：

```
Vserver   Volume       Aggregate    State      Type       Size  Available Used%
--------- ------------ ------------ ---------- ---- ---------- ---------- -----
          _pvc_89f1c156_3801_4de4_9f9d_034d54c395f4
                                  online     RW         10GB     5.00GB    0%
          _pvc_e42ec6fe_3baa_4af6_996d_134adbbb8e6d
                                  online     RW       5.79GB     5.50GB    0%
          _pvc_e8372153_9ad9_474a_951a_08ae15e1c0ba
                                  online     RW          1GB    511.8MB    0%
3 entries were displayed.
```

目前，调整大小是对现有卷使用新计算的唯一方法。

最低配置示例

以下示例显示了将大多数参数保留为默认值的基本配置。这是定义后端的最简单方法。

> 如果您正在将 NetApp ONTAP 上的 Amazon FSx 与 Astra Trident 结合使用，建议为 LIF 指定 DNS 名称，而不是 IP 地址。

具有基于证书的身份验证的 **ontap-san** 驱动程序

这是一个最低后端配置示例。`clientCertificate` ， `clientPrivateKey` 和 `trustedCACertifate` （如果使用可信 CA ，则可选）分别填充在 `backend.json` 中，并采用客户端证书，私钥和可信 CA 证书的 base64 编码值。

```
{
    "version": 1,
    "storageDriverName": "ontap-san",
    "backendName": "DefaultSANBackend",
    "managementLIF": "10.0.0.1",
    "dataLIF": "10.0.0.3",
    "svm": "svm_iscsi",
    "useCHAP": true,
    "chapInitiatorSecret": "cl9qxIm36DKyawxy",
    "chapTargetInitiatorSecret": "rqxigXgkesIpwxyz",
    "chapTargetUsername": "iJF4heBRT0TCwxyz",
    "chapUsername": "uh2aNCLSd6cNwxyz",
    "igroupName": "trident",
    "clientCertificate": "ZXR0ZXJwYXB...ICMgJ3BhcGVyc2",
    "clientPrivateKey": "vciwKIyAgZG...0cnksIGRlc2NyaX",
    "trustedCACertificate": "zcyBbaG...b3Igb3duIGNsYXNz"
}
```

具有双向 **CHAP** 的 **ONTAP SAN** 驱动程序

这是一个最低后端配置示例。此基本配置将创建一个 `ontap-san` 后端，并将 `useCHAP` 设置为 `true` 。

```
{
    "version": 1,
    "storageDriverName": "ontap-san",
    "managementLIF": "10.0.0.1",
    "dataLIF": "10.0.0.3",
    "svm": "svm_iscsi",
    "labels": {"k8scluster": "test-cluster-1", "backend": "testcluster1-
sanbackend"},
    "useCHAP": true,
    "chapInitiatorSecret": "cl9qxIm36DKyawxy",
    "chapTargetInitiatorSecret": "rqxigXgkesIpwxyz",
    "chapTargetUsername": "iJF4heBRT0TCwxyz",
    "chapUsername": "uh2aNCLSd6cNwxyz",
    "igroupName": "trident",
    "username": "vsadmin",
    "password": "secret"
}
```

ontap-san-economy-driver

```
{
    "version": 1,
    "storageDriverName": "ontap-san-economy",
    "managementLIF": "10.0.0.1",
    "svm": "svm_iscsi_eco",
    "useCHAP": true,
    "chapInitiatorSecret": "cl9qxIm36DKyawxy",
    "chapTargetInitiatorSecret": "rqxigXgkesIpwxyz",
    "chapTargetUsername": "iJF4heBRT0TCwxyz",
    "chapUsername": "uh2aNCLSd6cNwxyz",
    "igroupName": "trident",
    "username": "vsadmin",
    "password": "secret"
}
```

虚拟存储池后端示例

在下面显示的示例后端定义文件中，会为所有存储池设置特定的默认值，例如 `spaceReserve` at none ， `spaceAllocation` at false 和 `encryption` at false 。虚拟存储池在存储部分中进行定义。

在此示例中，某些存储池会设置自己的 `spaceReserve` ， `spaceAllocation` 和 `encryption` 值，而某些池会覆盖上述设置的默认值。

```
{
```

```
    "version": 1,
    "storageDriverName": "ontap-san",
    "managementLIF": "10.0.0.1",
    "dataLIF": "10.0.0.3",
    "svm": "svm_iscsi",
    "useCHAP": true,
    "chapInitiatorSecret": "cl9qxIm36DKyawxy",
    "chapTargetInitiatorSecret": "rqxigXgkesIpwxyz",
    "chapTargetUsername": "iJF4heBRT0TCwxyz",
    "chapUsername": "uh2aNCLSd6cNwxyz",
    "igroupName": "trident",
    "username": "vsadmin",
    "password": "secret",

    "defaults": {
          "spaceAllocation": "false",
          "encryption": "false",
          "qosPolicy": "standard"
    },
    "labels":{"store": "san_store", "kubernetes-cluster": "prod-cluster-
1"},
    "region": "us_east_1",
    "storage": [
        {
            "labels":{"protection":"gold", "creditpoints":"40000"},
            "zone":"us_east_1a",
            "defaults": {
                "spaceAllocation": "true",
                "encryption": "true",
                "adaptiveQosPolicy": "adaptive-extreme"
            }
        },
        {
            "labels":{"protection":"silver", "creditpoints":"20000"},
            "zone":"us_east_1b",
            "defaults": {
                "spaceAllocation": "false",
                "encryption": "true",
                "qosPolicy": "premium"
            }
        },
        {
            "labels":{"protection":"bronze", "creditpoints":"5000"},
            "zone":"us_east_1c",
            "defaults": {
                "spaceAllocation": "true",
```

```
                "encryption": "false"
            }
        }
    ]
}
```

以下是 `ontap-san-economy`-经济 驱动程序的 iSCSI 示例：

```
{
    "version": 1,
    "storageDriverName": "ontap-san-economy",
    "managementLIF": "10.0.0.1",
    "svm": "svm_iscsi_eco",
    "useCHAP": true,
    "chapInitiatorSecret": "cl9qxIm36DKyawxy",
    "chapTargetInitiatorSecret": "rqxigXgkesIpwxyz",
    "chapTargetUsername": "iJF4heBRT0TCwxyz",
    "chapUsername": "uh2aNCLSd6cNwxyz",
    "igroupName": "trident",
    "username": "vsadmin",
    "password": "secret",

    "defaults": {
          "spaceAllocation": "false",
          "encryption": "false"
    },
    "labels":{"store":"san_economy_store"},
    "region": "us_east_1",
    "storage": [
        {
            "labels":{"app":"oracledb", "cost":"30"},
            "zone":"us_east_1a",
            "defaults": {
                "spaceAllocation": "true",
                "encryption": "true"
            }
        },
        {
            "labels":{"app":"postgresdb", "cost":"20"},
            "zone":"us_east_1b",
            "defaults": {
                "spaceAllocation": "false",
                "encryption": "true"
            }
        },
```

```
        {
            "labels":{"app":"mysqldb", "cost":"10"},
            "zone":"us_east_1c",
            "defaults": {
                "spaceAllocation": "true",
                "encryption": "false"
            }
        }
    ]
}
```

将后端映射到 **StorageClasses**

以下 StorageClass 定义引用了上述虚拟存储池。使用 `parameters.selector` 字段，每个 StorageClass 都会调用可用于托管卷的虚拟池。卷将在选定虚拟池中定义各个方面。

- 第一个 StorageClass （`protection-gold`）将映射到 `ontap-nas-flexgroup` 后端的第一个，第二个虚拟存储池以及 `ontap-san` 后端的第一个虚拟存储池。这是唯一一个提供黄金级保护的池。
- 第二个 StorageClass （`protection-not-gold`）将映射到 `ontap-nas-flexgroup` 后端的第三个，第四个虚拟存储池以及 `ontap-san` 后端的第二个，第三个虚拟存储池。这些池是唯一提供黄金级以外保护级别的池。
- 第三个 StorageClass （`app-mysqldb`）将映射到 `ontap-NAS` 后端的第四个虚拟存储池和 `ontap-san-economy-backend` 的第三个虚拟存储池。这些池是唯一为 mysqldb 类型的应用程序提供存储池配置的池。
- 第四个存储类（`protection-silver-creditpoins-20k`）将映射到 `ontap-nas-flexgroup` 后端的第三个虚拟存储池和 `ontap-san` 后端的第二个虚拟存储池。这些池是唯一以 20000 个信用点提供黄金级保护的池。
- 第五个存储类（`credits-5k`）将映射到 `ontap-nas-economy-backend` 中的第二个虚拟存储池和 `ontap-san` 后端的第三个虚拟存储池。这些是唯一一款具有 5000 个信用点的池产品。

Astra Trident 将决定选择哪个虚拟存储池，并确保满足存储要求。

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: protection-gold
provisioner: netapp.io/trident
parameters:
  selector: "protection=gold"
  fsType: "ext4"
---
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: protection-not-gold
provisioner: netapp.io/trident
parameters:
  selector: "protection!=gold"
  fsType: "ext4"
---
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: app-mysqldb
provisioner: netapp.io/trident
parameters:
  selector: "app=mysqldb"
  fsType: "ext4"
---
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: protection-silver-creditpoints-20k
provisioner: netapp.io/trident
parameters:
  selector: "protection=silver; creditpoints=20000"
  fsType: "ext4"
---
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: creditpoints-5k
provisioner: netapp.io/trident
parameters:
  selector: "creditpoints=5000"
  fsType: "ext4"
```

使用 ONTAP NAS 驱动程序配置后端

了解如何使用 ONTAP 和 Cloud Volumes ONTAP NAS 驱动程序配置 ONTAP 后端。

- "准备"
- "配置和示例"

用户权限

Astra Trident 应以 ONTAP 或 SVM 管理员身份运行，通常使用 `admin` cluster 用户或 `vsadmin` SVM 用户，或者使用具有相同角色的其他名称的用户。对于适用于 NetApp ONTAP 的 Amazon FSX 部署， Astra Trident 应使用集群 `fsxadmin` user 或 `vsadmin` SVM 用户或具有相同角色的其他名称的用户作为 ONTAP 或 SVM 管理员运行。`fsxadmin` 用户是集群管理员用户的有限替代用户。

> 如果使用 `limitAggregateUsage` 参数，则需要集群管理员权限。将适用于 NetApp ONTAP 的 Amazon FSx 与 Astra Trident 结合使用时， `limitAggregateUsage` 参数不适用于 `vsadmin` 和 `fsxadmin` 用户帐户。如果指定此参数，配置操作将失败。

虽然可以在 ONTAP 中创建 Trident 驱动程序可以使用的限制性更强的角色，但我们不建议这样做。大多数新版本的 Trident 都会调用需要考虑的其他 API ，从而使升级变得困难且容易出错。

准备

了解如何准备使用 ONTAP NAS 驱动程序配置 ONTAP 后端。对于所有 ONTAP 后端， Astra Trident 需要至少为 SVM 分配一个聚合。

对于所有 ONTAP 后端， Astra Trident 需要至少为 SVM 分配一个聚合。

请记住，您还可以运行多个驱动程序，并创建指向其中一个驱动程序的存储类。例如，您可以配置一个使用 `ontap-nas` 驱动程序` 的金牌类和一个使用 `ontap-nas-economy-one 的铜牌类。

所有 Kubernetes 工作节点都必须安装适当的 NFS 工具。请参见 "此处" 有关详细信息：

身份验证

Astra Trident 提供了两种对 ONTAP 后端进行身份验证的模式。

- Credential Based ：具有所需权限的 ONTAP 用户的用户名和密码。建议使用 `admin` 或 `vsadmin` 等预定义的安全登录角色，以确保与 ONTAP 版本的最大兼容性。
- 基于证书： Astra Trident 还可以使用后端安装的证书与 ONTAP 集群进行通信。此处，后端定义必须包含客户端证书，密钥和可信 CA 证书的 Base64 编码值（如果使用）（建议）。

用户还可以选择更新现有后端，选择从基于凭据迁移到基于证书，反之亦然。如果 * 同时提供了凭据和证书 * ，则 Astra Trident 将在发出警告以从后端定义中删除凭据时默认使用证书。

启用基于凭据的身份验证

Astra Trident 需要 SVM 范围 / 集群范围的管理员的凭据才能与 ONTAP 后端进行通信。建议使用标准的预定义角色，例如 `admin` 或 `vsadmin` 。这样可以确保与未来的 ONTAP 版本向前兼容，这些版本可能会使功能 API 公开供未来的 Astra Trident 版本使用。可以创建自定义安全登录角色并将其用于 Astra Trident ，但不建议使用。

后端定义示例如下所示：

```
{
  "version": 1,
  "backendName": "ExampleBackend",
  "storageDriverName": "ontap-nas",
  "managementLIF": "10.0.0.1",
  "dataLIF": "10.0.0.2",
  "svm": "svm_nfs",
  "username": "vsadmin",
  "password": "secret"
}
```

请注意，后端定义是凭据以纯文本格式存储的唯一位置。创建后端后，用户名 / 密码将使用 Base64 进行编码并存储为 Kubernetes 密钥。创建 / 更新后端是唯一需要了解凭据的步骤。因此，这是一项仅由管理员执行的操作，由 Kubernetes 或存储管理员执行。

启用基于证书的身份验证

新的和现有的后端可以使用证书并与 ONTAP 后端进行通信。后端定义需要三个参数。

- clientCertificate ： 客户端证书的 Base64 编码值。
- clientPrivateKey ： 关联私钥的 Base64 编码值。
- trustedCACertifate ： 受信任 CA 证书的 Base64 编码值。如果使用可信 CA ，则必须提供此参数。如果不使用可信 CA ，则可以忽略此设置。

典型的工作流包括以下步骤。

步骤

1. 生成客户端证书和密钥。生成时，将公用名（ Common Name ， CN ）设置为要作为身份验证的 ONTAP 用户。

   ```
   openssl req -x509 -nodes -days 1095 -newkey rsa:2048 -keyout k8senv.key
   -out k8senv.pem -subj "/C=US/ST=NC/L=RTP/O=NetApp/CN=vsadmin"
   ```

2. 将可信 CA 证书添加到 ONTAP 集群。此问题可能已由存储管理员处理。如果未使用可信 CA ，则忽略。

   ```
   security certificate install -type server -cert-name <trusted-ca-cert-
   name> -vserver <vserver-name>
   ssl modify -vserver <vserver-name> -server-enabled true -client-enabled
   true -common-name <common-name> -serial <SN-from-trusted-CA-cert> -ca
   <cert-authority>
   ```

3. 在 ONTAP 集群上安装客户端证书和密钥（从步骤 1 开始）。

```
security certificate install -type client-ca -cert-name <certificate-
name> -vserver <vserver-name>
security ssl modify -vserver <vserver-name> -client-enabled true
```

4. 确认 ONTAP 安全登录角色支持 `cert` 身份验证方法。

```
security login create -user-or-group-name vsadmin -application ontapi
-authentication-method cert -vserver <vserver-name>
security login create -user-or-group-name vsadmin -application http
-authentication-method cert -vserver <vserver-name>
```

5. 使用生成的证书测试身份验证。将 <SVM 管理 LIF> 和 <SVM 名称 > 替换为管理 LIF IP 和 ONTAP 名称。您必须确保 LIF 的服务策略设置为 `default-data-management` 。

```
curl -X POST -Lk https://<ONTAP-Management-
LIF>/servlets/netapp.servlets.admin.XMLrequest_filer --key k8senv.key
--cert ~/k8senv.pem -d '<?xml version="1.0" encoding="UTF-8"?><netapp
xmlns="http://www.netapp.com/filer/admin" version="1.21"
vfiler="<vserver-name>"><vserver-get></vserver-get></netapp>'
```

6. 使用 Base64 对证书，密钥和可信 CA 证书进行编码。

```
base64 -w 0 k8senv.pem >> cert_base64
base64 -w 0 k8senv.key >> key_base64
base64 -w 0 trustedca.pem >> trustedca_base64
```

7. 使用从上一步获得的值创建后端。

```
$ cat cert-backend-updated.json
{
"version": 1,
"storageDriverName": "ontap-nas",
"backendName": "NasBackend",
"managementLIF": "1.2.3.4",
"dataLIF": "1.2.3.8",
"svm": "vserver_test",
"clientCertificate": "Faaaakkkkeeee...Vaaallllluuuueeee",
"clientPrivateKey": "LS0tFaKE...0VaLuES0tLS0K",
"storagePrefix": "myPrefix_"
}

#Update backend with tridentctl
$ tridentctl update backend NasBackend -f cert-backend-updated.json -n
trident
+------------+----------------+--------------------------------------
+--------+---------+
|    NAME    | STORAGE DRIVER |                 UUID                 |
STATE  | VOLUMES |
+------------+----------------+--------------------------------------
+--------+---------+
| NasBackend | ontap-nas      | 98e19b74-aec7-4a3d-8dcf-128e5033b214 |
online |       9 |
+------------+----------------+--------------------------------------
+--------+---------+
```

更新身份验证方法或轮换凭据

您可以更新现有后端以使用其他身份验证方法或轮换其凭据。这两种方式都适用：使用用户名 / 密码的后端可以更新为使用证书；使用证书的后端可以更新为基于用户名 / 密码的后端。为此，请使用更新后的 `backend.json` 文件，该文件包含执行 `tridentctl` 后端更新 所需的参数。

```
$ cat cert-backend-updated.json
{
"version": 1,
"storageDriverName": "ontap-nas",
"backendName": "NasBackend",
"managementLIF": "1.2.3.4",
"dataLIF": "1.2.3.8",
"svm": "vserver_test",
"username": "vsadmin",
"password": "secret",
"storagePrefix": "myPrefix_"
}

#Update backend with tridentctl
$ tridentctl update backend NasBackend -f cert-backend-updated.json -n
trident
+------------+----------------+--------------------------------------
+--------+---------+
|    NAME    | STORAGE DRIVER |                  UUID                |
STATE  | VOLUMES |
+------------+----------------+--------------------------------------
+--------+---------+
| NasBackend | ontap-nas      | 98e19b74-aec7-4a3d-8dcf-128e5033b214 |
online |       9 |
+------------+----------------+--------------------------------------
+--------+---------+
```

> 轮换密码时，存储管理员必须先在 ONTAP 上更新用户的密码。然后进行后端更新。轮换证书时，可以向用户添加多个证书。之后，后端将更新以使用新证书，然后可以从 ONTAP 集群中删除旧证书。

更新后端不会中断对已创建卷的访问，也不会影响在之后建立的卷连接。成功的后端更新表明， Astra Trident 可以与 ONTAP 后端进行通信并处理未来的卷操作。

管理 NFS 导出策略

Astra Trident 使用 NFS 导出策略来控制对其配置的卷的访问。

使用导出策略时， Astra Trident 提供了两个选项：

- Astra Trident 可以动态管理导出策略本身；在此操作模式下，存储管理员会指定一个表示可接受 IP 地址的 CIDR 块列表。Astra Trident 会自动将属于这些范围的节点 IP 添加到导出策略中。或者，如果未指定任何 CIDR ，则在节点上找到的任何全局范围的单播 IP 都将添加到导出策略中。
- 存储管理员可以手动创建导出策略和添加规则。除非在配置中指定了不同的导出策略名称，否则 Astra Trident 将使用默认导出策略。

动态管理导出策略

CSI Trident 20.04 版可以动态管理 ONTAP 后端的导出策略。这样，存储管理员就可以为工作节点 IP 指定允许的地址空间，而不是手动定义显式规则。它大大简化了导出策略管理；修改导出策略不再需要手动干预存储集群。此外，这有助于将对存储集群的访问限制为仅允许 IP 位于指定范围内的工作节点访问，从而支持精细的自动化管理。

> 只有 CSI Trident 才支持动态管理导出策略。请务必确保工作节点未被 NAT 处理。

示例

必须使用两个配置选项。下面是一个后端定义示例：

```
{
    "version": 1,
    "storageDriverName": "ontap-nas",
    "backendName": "ontap_nas_auto_export,
    "managementLIF": "192.168.0.135",
    "svm": "svm1",
    "username": "vsadmin",
    "password": "FaKePaSsWoRd",
    "autoExportCIDRs": ["192.168.0.0/24"],
    "autoExportPolicy": true
}
```

> 使用此功能时，您必须确保 SVM 中的根接合具有预先创建的导出策略，并具有允许节点 CIDR 块的导出规则（例如默认导出策略）。请始终遵循 NetApp 建议的最佳实践，为 Astra Trident 专用 SVM 。

以下是使用上述示例对此功能的工作原理进行的说明：

- `autosExportPolicy` 设置为 `true` 。这表示 Astra Trident 将为 `svm1` SVM 创建导出策略，并使用 `autosExportCIDRS` 地址块处理规则的添加和删除。例如， UUID 为 403b5326-8482-40db-96d0-d83fb3f4daec 且 `autodExportPolicy` 设置为 `true` 的后端会在 SVM 上创建一个名为 `trident-403b5326-8482-40db-96d0-d83fb3f4daec` 的导出策略。
- `autosExportCIDR` 包含地址块列表。此字段为可选字段，默认为 "0.0.0.0/0 ， "：： /0" 。如果未定义，则 Astra Trident 会添加在工作节点上找到的所有全局范围的单播地址。

在此示例中，提供了 `192.168.0.0/24` 地址空间。这表示此地址范围内的 Kubernetes 节点 IP 将添加到 Astra Trident 创建的导出策略中。当 Astra Trident 注册其运行的节点时，它会检索该节点的 IP 地址，并根据 `autosExportCIDRS` 中提供的地址块对其进行检查。筛选 IP 后， Astra Trident 会为其发现的客户端 IP 创建导出策略规则，并为其标识的每个节点创建一个规则。

创建后，您可以为后端更新 `autosExportPolicy` 和 `autosExportCIDR` 。您可以为自动管理的后端附加新的 CIDR ，也可以删除现有的 CIDR 。删除 CIDR 时请务必小心，以确保现有连接不会断开。您也可以选择对后端禁用 `autosExportPolicy` ，并回退到手动创建的导出策略。这需要在后端配置中设置 `exportPolicy` 参数。

在 Astra Trident 创建或更新后端后，您可以使用 `tridentctl` 或相应的 `tridentbackend` CRD 检查后端：

```
$ ./tridentctl get backends ontap_nas_auto_export -n trident -o yaml
items:
- backendUUID: 403b5326-8482-40db-96d0-d83fb3f4daec
  config:
    aggregate: ""
    autoExportCIDRs:
    - 192.168.0.0/24
    autoExportPolicy: true
    backendName: ontap_nas_auto_export
    chapInitiatorSecret: ""
    chapTargetInitiatorSecret: ""
    chapTargetUsername: ""
    chapUsername: ""
    dataLIF: 192.168.0.135
    debug: false
    debugTraceFlags: null
    defaults:
      encryption: "false"
      exportPolicy: <automatic>
      fileSystemType: ext4
```

当节点添加到 Kubernetes 集群并向 Astra Trident 控制器注册后，现有后端的导出策略将会更新（前提是它们位于后端的 `autosExportCIDR` 中指定的地址范围内）。

删除节点后， Astra Trident 会检查所有联机后端，以删除该节点的访问规则。通过从受管后端的导出策略中删除此节点 IP ， Astra Trident 可防止恶意挂载，除非此 IP 可由集群中的新节点重复使用。

对于以前存在的后端，使用 `tridentctl update backend` 更新后端可确保 Astra Trident 自动管理导出策略。这将创建一个以后端 UUID 命名的新导出策略，后端上存在的卷将在重新挂载时使用新创建的导出策略。

> 删除具有自动管理导出策略的后端将删除动态创建的导出策略。如果重新创建后端，则会将其视为新的后端，并会创建新的导出策略。

如果更新了活动节点的 IP 地址，则必须在此节点上重新启动 Astra Trident Pod 。然后， Astra Trident 将更新其管理的后端的导出策略，以反映此 IP 更改。

配置选项和示例

了解如何在您的 Astra Trident 安装中创建和使用 ONTAP NAS 驱动程序。本节提供了后端配置示例以及有关如何将后端映射到 StorageClasses 的详细信息。

后端配置选项

有关后端配置选项，请参见下表：

参数	Description	Default
ve版本		始终为 1
storageDriverName	存储驱动程序的名称	"ontap-nas" ， "ontap-nas-economy-" ， "ontap-nas-flexgroup" ， "ontap-san " ， "ontap-san-economy-"
backendName	自定义名称或存储后端	驱动程序名称 + "_" + dataLIF
m年 月 日	集群或 SVM 管理 LIF 的 IP 地址	"10.0.0.1 " ， "2001 ： 1234 ： abcd ： ： ： fefe] "
dataLIF	协议 LIF 的 IP 地址。对于 IPv6 ，请使用方括号。设置后无法更新	由 SVM 派生，除非另有说明
autosExportPolicy	启用自动创建和更新导出策略 [布尔值]	false
autosExportCIDR	启用 AutoExportPolicy 时用于筛选 Kubernetes 节点 IP 的 CIDR 列表	["0.0.0.0/0 " ， " ： ： /0 "]`
标签	要应用于卷的一组任意 JSON 格式的标签	"
客户端证书	客户端证书的 Base64 编码值。用于基于证书的身份验证	"
clientPrivateKey	客户端专用密钥的 Base64 编码值。用于基于证书的身份验证	"
trustedCACertifate	受信任 CA 证书的 Base64 编码值。可选。用于基于证书的身份验证	"
用户名	用于连接到集群 /SVM 的用户名。用于基于凭据的身份验证	
密码	连接到集群 /SVM 的密码。用于基于凭据的身份验证	
sVM	要使用的 Storage Virtual Machine	如果指定了 SVM managementLIF ，则派生
igroupName	要使用的 SAN 卷的 igroup 的名称	"trident — < 后端 UUID >"
s存储前缀	在 SVM 中配置新卷时使用的前缀。设置后无法更新	Trident
limitAggregateUsage	如果使用量超过此百分比，则配置失败。* 不适用于适用于 ONTAP 的 Amazon FSx *	" （默认情况下不强制实施）
limitVolumeSize	如果经济型驱动程序请求的卷大小超过此值，则配置失败。	" （默认情况下不强制实施）
lunsPerFlexvol	每个 FlexVol 的最大 LUN 数，必须在 50 ， 200 范围内	100

参数	Description	Default
debugTraceFlags	故障排除时要使用的调试标志。示例 ｛"api"： false ， "method "： true｝	空
nfsMountOptions	NFS 挂载选项的逗号分隔列表	"
qtreesPerFlexvol	每个 FlexVol 的最大 qtree 数，必须在 50 ， 300 范围内	200
useREST	用于使用 ONTAP REST API 的布尔参数。* 技术预览 *	false

> useREST 提供了一个 * 技术预览 * ，建议用于测试环境，而不是生产工作负载。如果设置为 true ，则 Astra Trident 将使用 ONTAP REST API 与后端进行通信。此功能需要使用 ONTAP 9.9 及更高版本。此外，使用的 ONTAP 登录角色必须能够访问 ONTAP 应用程序。这一点可通过预定义的 vsadmin 和 cluster-admin 角色来满足。

要与 ONTAP 集群通信，您应提供身份验证参数。这可以是安全登录的用户名 / 密码，也可以是已安装的证书。

> 如果您使用适用于 NetApp ONTAP 后端的 Amazon FSX ，请勿指定 limitAggregateUsage 参数。Amazon FSX for NetApp ONTAP 提供的 fsxadmin 和 vsadmin 角色不包含检索聚合使用情况并通过 Astra Trident 对其进行限制所需的访问权限。

> 请勿使用 debugTraceFlags ，除非您正在进行故障排除并需要详细的日志转储。

> 创建后端时，请记住，创建后无法修改 dataLIF 和 storagePrefix 。要更新这些参数，您需要创建一个新的后端。

可以为 managementLIF 选项指定完全限定域名 （ FQDN ）。也可以为 dataLIF 选项指定 FQDN ，在这种情况下， FQDN 将用于 NFS 挂载操作。这样，您就可以创建循环 DNS ，以便在多个数据 LIF 之间实现负载平衡。

对于所有 ONTAP 驱动程序，也可以将 managementLIF 设置为 IPv6 地址。请务必使用 ` -use-ipv6` 标志安装 Astra Trident 。必须 m方括号内的 anagementLIF IPv6 地址。

> 使用 IPv6 地址时，请确保在方括号内定义 managementLIF 和 dataLIF （如果包含在后端定义中），例如 [28e8： d9fb： a825： b7bf： 69a8： d02f： 9e7b： 3555] 。如果未提供 dataLIF ，则 Astra Trident 将从 SVM 提取 IPv6 数据 LIF 。

使用 autosExportPolicy 和 autosExportCIDR 选项， CSI Trident 可以自动管理导出策略。所有 ontap-nas-* 驱动程序均支持此功能。

对于 ontap-nas-economy 驱动程序， limitVolumeSize 选项还会限制它为 qtree 和 FlexVol 管理的卷的最大大小，而 qtreesPerFlexvol 选项允许自定义每个 的最大 qtree 数。

可以使用 nfsMountOptions 参数指定挂载选项。Kubernetes 永久性卷的挂载选项通常在存储类中指定，但如果在存储类中未指定挂载选项，则 Astra Trident 将回退为使用存储后端配置文件中指定的挂载选项。如果在存储类或配置文件中未指定挂载选项，则 Astra Trident 不会在关联的永久性卷上设置任何挂载选项。

> Astra Trident 会在使用 `ontap-NAS` 和 `ontap-nas-flexgroup` 创建的所有卷的 "Comments" 字段中设置配置标签。根据所使用的驱动程序，注释将在 FlexVol （`ontap-NAS` ）或 FlexGroup （`ontap-nas-flexgroup` ）上进行设置。Astra Trident 会在配置存储池时将存储池上的所有标签复制到该存储卷。存储管理员可以为每个存储池定义标签，并对存储池中创建的所有卷进行分组。这样，您就可以根据后端配置中提供的一组可自定义标签来方便地区分卷了。

用于配置卷的后端配置选项

您可以在配置的特殊部分中使用这些选项来控制默认配置每个卷的方式。有关示例，请参见以下配置示例。

参数	**Description**	**Default**
`spaceAllocation`	LUN 的空间分配	true
`s`页面预留	空间预留模式； " 无 " （精简）或 " 卷 " （厚）	无
`sSnapshot` 策略	要使用的 Snapshot 策略	无
`qosPolicy`	要为创建的卷分配的 QoS 策略组。选择每个存储池 / 后端的 qosPolicy 或 adaptiveQosPolicy 之一	"
`adaptiveQosPolicy`	要为创建的卷分配的自适应 QoS 策略组。选择每个存储池 / 后端的 qosPolicy 或 adaptiveQosPolicy 之一。不受 ontap-nas-economy.	"
`sSnapshot` 预留	为快照预留的卷百分比为 "0"	如果 `snapshotPolicy` 为 " 无 " ，则为 " 无 " ，否则为 " "
`splitOnClone`	创建克隆时，从其父级拆分该克隆	false
加密	启用 NetApp 卷加密	false
`securityStyle`	新卷的安全模式	"unix"
分层策略	使用 " 无 " 的分层策略	适用于 ONTAP 9.5 SVM-DR 之前的配置的 " 仅快照 "
unixPermissions	新卷的模式	777.
snapshotDir	控制 ` .snapshot` 目录的可见性	false
导出策略	要使用的导出策略	default
securityStyle	新卷的安全模式	"unix"

> 在 Astra Trident 中使用 QoS 策略组需要 ONTAP 9.8 或更高版本。建议使用非共享 QoS 策略组，并确保策略组分别应用于每个成分卷。共享 QoS 策略组将对所有工作负载的总吞吐量实施上限。

下面是定义了默认值的示例：

```
{
  "version": 1,
  "storageDriverName": "ontap-nas",
  "backendName": "customBackendName",
  "managementLIF": "10.0.0.1",
  "dataLIF": "10.0.0.2",
  "labels": {"k8scluster": "dev1", "backend": "dev1-nasbackend"},
  "svm": "trident_svm",
  "username": "cluster-admin",
  "password": "password",
  "limitAggregateUsage": "80%",
  "limitVolumeSize": "50Gi",
  "nfsMountOptions": "nfsvers=4",
  "debugTraceFlags": {"api":false, "method":true},
  "defaults": {
    "spaceReserve": "volume",
    "qosPolicy": "premium",
    "exportPolicy": "myk8scluster",
    "snapshotPolicy": "default",
    "snapshotReserve": "10"
  }
}
```

对于 `ontap-nas` 和 `ontap-nas-flexgroups` ， Astra Trident 现在使用新的计算方法来确保 FlexVol 的大小正确，并使用 snapshotReserve 百分比和 PVC 。当用户请求 PVC 时， Astra Trident 会使用新计算创建具有更多空间的原始 FlexVol 。此计算可确保用户在 PVC 中收到所请求的可写空间，而不是小于所请求的空间。在 v21.07 之前，如果用户请求 PVC （例如， 5GiB ），并且 snapshotReserve 为 50% ，则只会获得 2.5 GiB 的可写空间。这是因为用户请求的是整个卷，而 `snapshotReserve` 是其中的一个百分比。在 Trident 21.07 中，用户请求的是可写空间， Astra Trident 将 `snapshotReserve` number 定义为整个卷的百分比。这不适用于 `ontap-nas-economy"` 。请参见以下示例以了解其工作原理：

计算方法如下：

```
Total volume size = (PVC requested size) / (1 - (snapshotReserve
percentage) / 100)
```

对于 snapshotReserve = 50% ， PVC 请求 = 5GiB ，卷总大小为 2/.5 = 10GiB ，可用大小为 5GiB ，这是用户在 PVC 请求中请求的大小。`volume show` 命令应显示与以下示例类似的结果：

```
Vserver   Volume       Aggregate    State      Type       Size  Available Used%
--------- ------------ ------------ ---------- ---- ---------- ---------- -----
                  _pvc_89f1c156_3801_4de4_9f9d_034d54c395f4
                                   online     RW         10GB     5.00GB    0%
                  _pvc_e8372153_9ad9_474a_951a_08ae15e1c0ba
                                   online     RW          1GB    511.8MB    0%
2 entries were displayed.
```

在升级 Astra Trident 时，先前安装的现有后端将按照上述说明配置卷。对于在升级之前创建的卷，您应调整其卷的大小，以便观察到所做的更改。例如，一个 2 GiB PVC ，其 `snapshotReserve=50` earlier 会导致一个卷提供 1 GiB 的可写空间。例如，将卷大小调整为 3GiB 可为应用程序在一个 6 GiB 卷上提供 3GiB 的可写空间。

最低配置示例

以下示例显示了将大多数参数保留为默认值的基本配置。这是定义后端的最简单方法。

> 如果在采用 Trident 的 NetApp ONTAP 上使用 Amazon FSx ，建议为 LIF 指定 DNS 名称，而不是 IP 地址。

具有基于证书的身份验证的 **ONTAP NAS** 驱动程序

这是一个最低后端配置示例。`clientCertificate` ， `clientPrivateKey` 和 `trustedCACertifate` （如果使用可信 CA ，则可选）分别填充在 `backend.json` 中，并采用客户端证书，私钥和可信 CA 证书的 base64 编码值。

```
{
  "version": 1,
  "backendName": "DefaultNASBackend",
  "storageDriverName": "ontap-nas",
  "managementLIF": "10.0.0.1",
  "dataLIF": "10.0.0.15",
  "svm": "nfs_svm",
  "clientCertificate": "ZXR0ZXJwYXB...ICMgJ3BhcGVyc2",
  "clientPrivateKey": "vciwKIyAgZG...0cnksIGRlc2NyaX",
  "trustedCACertificate": "zcyBbaG...b3Igb3duIGNsYXNz",
  "storagePrefix": "myPrefix_"
}
```

具有自动导出策略的 **ontap-NAS** 驱动程序

此示例显示了如何指示 Astra Trident 使用动态导出策略自动创建和管理导出策略。这对于 `ontap-nas-economy.` 和 `ontap-nas-flexgroup` 驱动程序也是如此。

```
{
    "version": 1,
    "storageDriverName": "ontap-nas",
    "managementLIF": "10.0.0.1",
    "dataLIF": "10.0.0.2",
    "svm": "svm_nfs",
    "labels": {"k8scluster": "test-cluster-east-1a", "backend": "test1-
nasbackend"},
    "autoExportPolicy": true,
    "autoExportCIDRs": ["10.0.0.0/24"],
    "username": "admin",
    "password": "secret",
    "nfsMountOptions": "nfsvers=4",
}
```

ontap-nas-flexgroup 驱动程序

```
{
    "version": 1,
    "storageDriverName": "ontap-nas-flexgroup",
    "managementLIF": "10.0.0.1",
    "dataLIF": "10.0.0.2",
    "labels": {"k8scluster": "test-cluster-east-1b", "backend": "test1-
ontap-cluster"},
    "svm": "svm_nfs",
    "username": "vsadmin",
    "password": "secret",
}
```

使用 **IPv6** 的 **ONTAP NAS** 驱动程序

```
{
 "version": 1,
 "storageDriverName": "ontap-nas",
 "backendName": "nas_ipv6_backend",
 "managementLIF": "[5c5d:5edf:8f:7657:bef8:109b:1b41:d491]",
 "labels": {"k8scluster": "test-cluster-east-1a", "backend": "test1-ontap-
ipv6"},
 "svm": "nas_ipv6_svm",
 "username": "vsadmin",
 "password": "netapp123"
}
```

ontap-nas-economy-driver

```
{
    "version": 1,
    "storageDriverName": "ontap-nas-economy",
    "managementLIF": "10.0.0.1",
    "dataLIF": "10.0.0.2",
    "svm": "svm_nfs",
    "username": "vsadmin",
    "password": "secret"
}
```

虚拟存储池后端示例

在下面显示的示例后端定义文件中，会为所有存储池设置特定的默认值，例如 `spaceReserve` at none ， `spaceAllocation` at false 和 `encryption` at false 。虚拟存储池在存储部分中进行定义。

在此示例中，某些存储池会设置自己的 `spaceReserve` ， `spaceAllocation` 和 `encryption` 值，而某些池会覆盖上述设置的默认值。

ontap-NAS 驱动程序

```
{
    {
    "version": 1,
    "storageDriverName": "ontap-nas",
    "managementLIF": "10.0.0.1",
    "dataLIF": "10.0.0.2",
    "svm": "svm_nfs",
    "username": "admin",
    "password": "secret",
    "nfsMountOptions": "nfsvers=4",

    "defaults": {
          "spaceReserve": "none",
          "encryption": "false",
          "qosPolicy": "standard"
    },
    "labels":{"store":"nas_store", "k8scluster": "prod-cluster-1"},
    "region": "us_east_1",
    "storage": [
        {
            "labels":{"app":"msoffice", "cost":"100"},
            "zone":"us_east_1a",
            "defaults": {
                "spaceReserve": "volume",
```

```
                "encryption": "true",
                "unixPermissions": "0755",
                "adaptiveQosPolicy": "adaptive-premium"
            }
        },
        {
            "labels":{"app":"slack", "cost":"75"},
            "zone":"us_east_1b",
            "defaults": {
                "spaceReserve": "none",
                "encryption": "true",
                "unixPermissions": "0755"
            }
        },
        {
            "labels":{"app":"wordpress", "cost":"50"},
            "zone":"us_east_1c",
            "defaults": {
                "spaceReserve": "none",
                "encryption": "true",
                "unixPermissions": "0775"
            }
        },
        {
            "labels":{"app":"mysqldb", "cost":"25"},
            "zone":"us_east_1d",
            "defaults": {
                "spaceReserve": "volume",
                "encryption": "false",
                "unixPermissions": "0775"
            }
        }
    ]
}
```

ontap-nas-flexgroup 驱动程序

```
{
    "version": 1,
    "storageDriverName": "ontap-nas-flexgroup",
    "managementLIF": "10.0.0.1",
    "dataLIF": "10.0.0.2",
    "svm": "svm_nfs",
    "username": "vsadmin",
    "password": "secret",
```

```
    "defaults": {
          "spaceReserve": "none",
          "encryption": "false"
    },
    "labels":{"store":"flexgroup_store", "k8scluster": "prod-cluster-1"},
    "region": "us_east_1",
    "storage": [
        {
            "labels":{"protection":"gold", "creditpoints":"50000"},
            "zone":"us_east_1a",
            "defaults": {
                "spaceReserve": "volume",
                "encryption": "true",
                "unixPermissions": "0755"
            }
        },
        {
            "labels":{"protection":"gold", "creditpoints":"30000"},
            "zone":"us_east_1b",
            "defaults": {
                "spaceReserve": "none",
                "encryption": "true",
                "unixPermissions": "0755"
            }
        },
        {
            "labels":{"protection":"silver", "creditpoints":"20000"},
            "zone":"us_east_1c",
            "defaults": {
                "spaceReserve": "none",
                "encryption": "true",
                "unixPermissions": "0775"
            }
        },
        {
            "labels":{"protection":"bronze", "creditpoints":"10000"},
            "zone":"us_east_1d",
            "defaults": {
                "spaceReserve": "volume",
                "encryption": "false",
                "unixPermissions": "0775"
            }
        }
    ]
}
```

ontap-nas-economy-driver

```
{
    "version": 1,
    "storageDriverName": "ontap-nas-economy",
    "managementLIF": "10.0.0.1",
    "dataLIF": "10.0.0.2",
    "svm": "svm_nfs",
    "username": "vsadmin",
    "password": "secret",

    "defaults": {
          "spaceReserve": "none",
          "encryption": "false"
    },
    "labels":{"store":"nas_economy_store"},
    "region": "us_east_1",
    "storage": [
        {
            "labels":{"department":"finance", "creditpoints":"6000"},
            "zone":"us_east_1a",
            "defaults": {
                "spaceReserve": "volume",
                "encryption": "true",
                "unixPermissions": "0755"
            }
        },
        {
            "labels":{"department":"legal", "creditpoints":"5000"},
            "zone":"us_east_1b",
            "defaults": {
                "spaceReserve": "none",
                "encryption": "true",
                "unixPermissions": "0755"
            }
        },
        {
            "labels":{"department":"engineering", "creditpoints":"3000"},
            "zone":"us_east_1c",
            "defaults": {
                "spaceReserve": "none",
                "encryption": "true",
                "unixPermissions": "0775"
            }
        },
        {
```

```
            "labels":{"department":"humanresource",
"creditpoints":"2000"},
            "zone":"us_east_1d",
            "defaults": {
                "spaceReserve": "volume",
                "encryption": "false",
                "unixPermissions": "0775"
            }
        }
    ]
}
```

将后端映射到 **StorageClasses**

以下 StorageClass 定义引用了上述虚拟存储池。使用 `parameters.selector` 字段，每个 StorageClass 都会调用可用于托管卷的虚拟池。卷将在选定虚拟池中定义各个方面。

- 第一个 StorageClass （`protection-gold`）将映射到 `ontap-nas-flexgroup` 后端的第一个，第二个虚拟存储池以及 `ontap-san` 后端的第一个虚拟存储池。这是唯一一个提供黄金级保护的池。
- 第二个 StorageClass （`protection-not-gold`）将映射到 `ontap-nas-flexgroup` 后端的第三个，第四个虚拟存储池以及 `ontap-san` 后端的第二个，第三个虚拟存储池。这些池是唯一提供黄金级以外保护级别的池。
- 第三个 StorageClass （`app-mysqldb`）将映射到 `ontap-NAS` 后端的第四个虚拟存储池和 `ontap-san-economy-backend` 的第三个虚拟存储池。这些池是唯一为 mysqldb 类型的应用程序提供存储池配置的池。
- 第四个存储类（`protection-silver-creditpoins-20k`）将映射到 `ontap-nas-flexgroup` 后端的第三个虚拟存储池和 `ontap-san` 后端的第二个虚拟存储池。这些池是唯一以 20000 个信用点提供黄金级保护的池。
- 第五个存储类（`credits-5k`）将映射到 `ontap-nas-economy-backend` 中的第二个虚拟存储池和 `ontap-san` 后端的第三个虚拟存储池。这些是唯一一款具有 5000 个信用点的池产品。

Astra Trident 将决定选择哪个虚拟存储池，并确保满足存储要求。

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: protection-gold
provisioner: netapp.io/trident
parameters:
  selector: "protection=gold"
  fsType: "ext4"
---
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: protection-not-gold
provisioner: netapp.io/trident
parameters:
  selector: "protection!=gold"
  fsType: "ext4"
---
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: app-mysqldb
provisioner: netapp.io/trident
parameters:
  selector: "app=mysqldb"
  fsType: "ext4"
---
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: protection-silver-creditpoints-20k
provisioner: netapp.io/trident
parameters:
  selector: "protection=silver; creditpoints=20000"
  fsType: "ext4"
---
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: creditpoints-5k
provisioner: netapp.io/trident
parameters:
  selector: "creditpoints=5000"
  fsType: "ext4"
```

将 Astra Trident 与适用于 NetApp ONTAP 的 Amazon FSX 结合使用

"适用于 NetApp ONTAP 的 Amazon FSX"是一种完全受管的 AWS 服务，支持客户启动和运行由 NetApp 的 ONTAP 存储操作系统提供支持的文件系统。Amazon FSX for NetApp ONTAP 支持您利用您熟悉的 NetApp 功能，性能和管理功能，同时利用在 AWS 上存储数据的简便性，灵活性，安全性和可扩展性。FSX 支持 ONTAP 的许多文件系统功能和管理 API 。

文件系统是 Amazon FSX 中的主要资源，类似于内部部署的 ONTAP 集群。在每个 SVM 中，您可以创建一个或多个卷，这些卷是将文件和文件夹存储在文件系统中的数据容器。借助适用于 NetApp ONTAP 的 Amazon FSX ， Data ONTAP 将作为云中的托管文件系统提供。新的文件系统类型称为 * NetApp ONTAP * 。

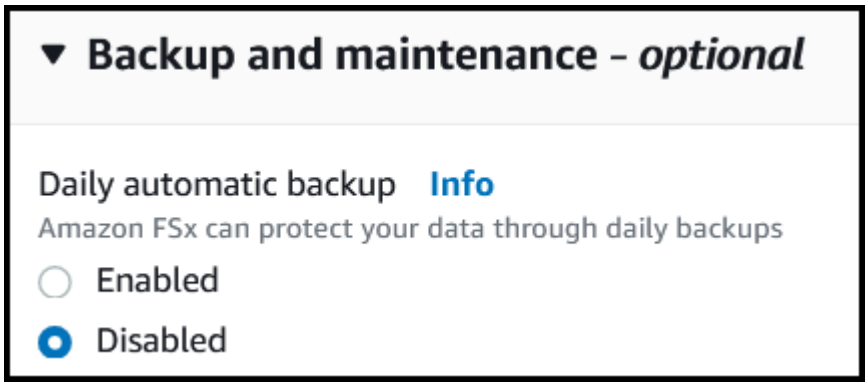
通过将 Astra Trident 与适用于 NetApp ONTAP 的 Amazon FSx 结合使用，您可以确保在 Amazon Elastic Kubernetes Service （ EKS ）中运行的 Kubernetes 集群可以配置由 ONTAP 备份的块和文件永久性卷。

创建适用于 **ONTAP** 的 **Amazon FSX** 文件系统

Trident 无法删除在启用了自动备份的 Amazon FSx 文件系统上创建的卷。要删除 PVC ，您需要手动删除 PV 和 ONTAP 的 FSX 卷。

要防止此问题描述 ，请执行以下操作：

- 请勿使用 "* 快速创建 "* 来创建适用于 ONTAP 的 FSX 文件系统。快速创建工作流可启用自动备份，但不提供选择退出选项。
- 使用 "** 标准创建 " 时，禁用自动备份。禁用自动备份可以使 Trident 成功删除卷，而无需进一步手动干预。



了解 **Astra Trident**

如果您是 Astra Trident 的新用户，请使用下面提供的链接进行熟悉：

- "常见问题解答"
- "使用 Astra Trident 的要求"
- "部署 Astra Trident"
- "配置适用于 NetApp ONTAP 的 ONTAP， Cloud Volumes ONTAP 和 Amazon FSX 的最佳实践"
- "集成 Astra Trident"
- "ONTAP SAN 后端配置"
- "ONTAP NAS 后端配置"

详细了解驱动程序功能 "此处"。

适用于 NetApp ONTAP 的 Amazon FSX 使用 "FabricPool" 以管理存储层。通过它，您可以根据数据是否经常访问来将数据存储在层中。

Astra Trident应以`vsadmin` SVM用户或具有相同角色的其他名称的用户的身份运行。适用于NetApp ONTAP的Amazon FSX具有一个`fsxadmin`用户、该用户是ONTAP `admin` cluster用户的有限替代。建议不要在Trident中使用`fsxadmin`用户、因为`vsadmin` SVM用户可以访问更多Astra Trident功能。

驱动程序

您可以使用以下驱动程序将 Astra Trident 与适用于 NetApp ONTAP 的 Amazon FSx 集成：

- `ontap-san`：配置的每个 PV 都是其自己的 Amazon FSX for NetApp ONTAP 卷中的一个 LUN 。
- `ontap-san-economi`：为 NetApp ONTAP 卷配置的每个 PV 都是一个 LUN ，每个 Amazon FSX 具有可配置的 LUN 数量。
- `ONONTAP NAS`：配置的每个 PV 都是适用于 NetApp ONTAP 的完整 Amazon FSX 卷。
- `ontap-nas-economy.`：配置的每个 PV 都是一个 qtree ，对于 NetApp ONTAP 卷，每个 Amazon FSx 的 qtree 数量是可配置的。
- `ontap-nas-flexgroup`：配置的每个 PV 都是适用于 NetApp ONTAP FlexGroup 卷的完整 Amazon FSX 。

身份验证

Astra Trident 提供两种身份验证模式：

- 基于证书： Astra Trident 将使用 SVM 上安装的证书与 FSX 文件系统上的 SVM 进行通信。
- 基于凭据：您可以使用文件系统的 `fsxadmin` 用户或为 SVM 配置的 `vsadmin` 用户。

> 我们强烈建议使用`vsadmin` user而非`fsxadmin`来配置后端。Astra Trident 将使用此用户名和密码与 FSX 文件系统进行通信。

要了解有关身份验证的详细信息，请参见以下链接：

- "ONTAP NAS"
- "ONTAP SAN"

使用适用于 **NetApp ONTAP** 的 **Amazon FSX** 在 **EKS** 上部署和配置 **Astra Trident**

您需要什么？ **#8217**；将需要什么

- 已安装 `kubectl` 的现有 Amazon EKS 集群或自管理 Kubernetes 集群。
- 可从集群的工作节点访问的适用于 NetApp ONTAP 文件系统和 Storage Virtual Machine （ SVM ）的现有 Amazon FSX 。
- 为准备工作的工作节点 "NFS 和 / 或 iSCSI"。

> 确保按照 Amazon Linux 和 Ubuntu 所需的节点准备步骤进行操作 "Amazon Machine 映像"（AMIS），具体取决于您的 EKS AMI 类型。

有关其他 Astra Trident 要求，请参见 "此处"。

步骤

1. 使用 ./trident 部署 Astra Trident 。 Get-started/Kubernetes 部署 .html[部署方法^] 之一部署 Astra Trident 。
2. 按照以下步骤配置 Astra Trident：
   a. 收集 SVM 的管理 LIF DNS 名称。例如，通过使用 AWS 命令行界面，在运行以下命令后，在 `Endpoints` → `Management` 下找到 `DNSName` 条目：

   ```
   aws fsx describe-storage-virtual-machines --region <file system
   region>
   ```

3. 创建并安装用于身份验证的证书。如果您使用的是 `ontap-san` 后端，请参见 "此处"。如果您使用的是 `ontap-nas` 后端，请参见 "此处"。

   > 您可以从可以访问文件系统的任何位置使用 SSH 登录到文件系统（例如，安装证书）。使用 `fsxadmin` 用户，创建文件系统时配置的密码以及 `AWS FSx describe` **文件系统** 中的管理 DNS 名称。

4. 使用您的证书和管理 LIF 的 DNS 名称创建后端文件，如以下示例所示：

   ```
   {
     "version": 1,
     "storageDriverName": "ontap-san",
     "backendName": "customBackendName",
     "managementLIF": "svm-XXXXXXXXXXXXXXXXX.fs-XXXXXXXXXXXXXXXXX.fsx.us-
   east-2.aws.internal",
     "svm": "svm01",
     "clientCertificate": "ZXR0ZXJwYXB...ICMgJ3BhcGVyc2",
     "clientPrivateKey": "vciwKIyAgZG...0cnksIGRlc2NyaX",
     "trustedCACertificate": "zcyBbaG...b3Igb3duIGNsYXNz",
    }
   ```

有关创建后端的信息，请参见以下链接：

- "使用 ONTAP NAS 驱动程序配置后端"
- "使用 ONTAP SAN 驱动程序配置后端"

> 请勿为 `ontap-san` 和 `ontap-san-economy-` 驱动程序指定 `dataLIF` ，以允许 Astra Trident 使用多路径。

> `limitAggregateUsage` 参数不适用于 `vsadmin` 和 `fsxadmin` 用户帐户。如果指定此参数，配置操作将失败。

部署完成后，执行以下步骤以创建 "存储类，配置卷以及将卷挂载到 Pod 中"。

了解更多信息

- "Amazon FSX for NetApp ONTAP 文档"
- "有关适用于 NetApp ONTAP 的 Amazon FSX 的博客文章"

使用 kubectl 创建后端

后端定义了 Astra Trident 与存储系统之间的关系。它告诉 Astra Trident 如何与该存储系统进行通信，以及 Astra Trident 如何从该存储系统配置卷。安装 Astra Trident 后，下一步是创建后端。使用 `TridentBackendConfig` Custom Resource Definition （ CRD ），您可以直接通过 Kubernetes 界面创建和管理 Trident 后端。为此，您可以对 Kubernetes 分发版使用 `kubectl` 或等效的命令行界面工具。

TridentBackendConfig

`TridentBackendConfig` （`tbc`， `tbconfig`， `tbackendconfig`）是一个前端命名的 CRD ，可用于使用 `kubectl` 管理 Astra Trident 后端。现在， Kubernetes 和存储管理员可以直接通过 Kubernetes 命令行界面创建和管理后端，而无需专用命令行实用程序（`tridentctl`）。

创建 `TridentBackendConfig` 对象时，将发生以下情况：

- Astra Trident 会根据您提供的配置自动创建后端。此值在内部表示为 `TridentBackend` （`tbe`， `tridentbackend`） CR 。
- `TridentBackendConfig` 唯一绑定到由 Astra Trident 创建的 `TridentBackend` 。

每个 `TridentBackendConfig` 都与 `TridentBackend` 保持一对一映射。前者是为用户提供的用于设计和配置后端的接口；后者是 Trident 表示实际后端对象的方式。

> `TridentBackend` CRS 由 Astra Trident 自动创建。您 * 不应 * 修改它们。如果要更新后端，请通过修改 `TridentBackendConfig` 对象来执行此操作。

有关 `TridentBackendConfig` CR 的格式，请参见以下示例：

```
apiVersion: trident.netapp.io/v1
kind: TridentBackendConfig
metadata:
  name: backend-tbc-ontap-san
spec:
  version: 1
  backendName: ontap-san-backend
  storageDriverName: ontap-san
  managementLIF: 10.0.0.1
  dataLIF: 10.0.0.2
  svm: trident_svm
  credentials:
    name: backend-tbc-ontap-san-secret
```

您还可以查看中的示例 "Trident 安装程序" 所需存储平台 / 服务的示例配置目录。

`sPec` 采用后端特定的配置参数。在此示例中，后端使用 `ontap-san` 存储驱动程序并使用此处所示的配置参数。有关所需存储驱动程序的配置选项列表，请参见 "存储驱动程序的后端配置信息"。

`sPec` 部分还包括 `credentials` 和 `deletionPolicy` 字段，这些字段在 `TridentBackendConfig` CR 中新增：

- `credentials` ： 此参数为必填字段，包含用于向存储系统 / 服务进行身份验证的凭据。此密码设置为用户创建的 Kubernetes Secret 。凭据不能以纯文本形式传递，因此会导致错误。
- `deeltionPolicy` ： 此字段定义删除 `TridentBackendConfig` 时应发生的情况。它可以采用以下两种可能值之一：
  - `delete` ： 这将同时删除 `TridentBackendConfig` CR 和关联后端。这是默认值。
  - `retain` ： 删除 `TrdentBackendConfig` CR 后，后端定义仍存在，可使用 `tridentctl` 进行管理。将删除策略设置为 `retain` 允许用户降级到早期版本（ 21.04 之前）并保留创建的后端。创建 `TridentBackendConfig` 后，可以更新此字段的值。

> 后端名称使用 `sPec.backendName` 设置。如果未指定，则后端的名称将设置为 `TridentBackendConfig` 对象的名称（ metadata.name ）。建议使用 `sPec.backendName` 显式设置后端名称。

> 使用 `tridentctl` 创建的后端没有关联的 `TridentBackendConfig` 对象。您可以通过创建 `TridentBackendConfig` CR ，选择使用 `kubectl` 来管理此类后端。必须注意指定相同的配置参数（例如 `sPec.backendName` ， `sPec.storagePrefix` ， `sPec.storageDriverName` 等）。Astra Trident 会自动将新创建的 `TridentBackendConfig` 与已有的后端绑定。

步骤概述

要使用 `kubectl` 创建新后端，应执行以下操作：

1. 创建 "Kubernetes 机密"。此密钥包含 Astra Trident 与存储集群 / 服务通信所需的凭据。

2. 创建 `TridentBackendConfig` 对象。其中包含有关存储集群 / 服务的详细信息，并引用了上一步中创建的密钥。

创建后端后，您可以使用 `kubectl get tbc <tbc-name> -n <trident` 命名空间 `>` 来观察其状态，并收集其他详细信息。

第 1 步：创建 Kubernetes 机密

创建一个机密，其中包含后端的访问凭据。这是每个存储服务 / 平台所特有的。以下是一个示例：

```
$ kubectl -n trident create -f backend-tbc-ontap-san-secret.yaml
apiVersion: v1
kind: Secret
metadata:
  name: backend-tbc-ontap-san-secret
type: Opaque
stringData:
  username: cluster-admin
  password: t@Ax@7q(>
```

下表汇总了每个存储平台的机密中必须包含的字段：

存储平台机密字段问题描述	机密	字段问题描述
Azure NetApp Files	clientId	应用程序注册中的客户端 ID
适用于 GCP 的 Cloud Volumes Service	private_key_id	专用密钥的 ID 。具有 CVS 管理员角色的 GCP 服务帐户的 API 密钥的一部分
适用于 GCP 的 Cloud Volumes Service	private_key	专用密钥。具有 CVS 管理员角色的 GCP 服务帐户的 API 密钥的一部分
Element （ NetApp HCI/SolidFire ）	端点	使用租户凭据的 SolidFire 集群的 MVIP
ONTAP	username	用于连接到集群 /SVM 的用户名。用于基于凭据的身份验证
ONTAP	password	连接到集群 /SVM 的密码。用于基于凭据的身份验证
ONTAP	客户端权限密钥	客户端专用密钥的 Base64 编码值。用于基于证书的身份验证

存储平台机密字段问题描述	机密	字段问题描述
ONTAP	用户名	入站用户名。如果 useCHAP=true ，则为必需项。适用于 `ontap-san` 和 `ontap-san-economy`.
ONTAP	chapInitiatorSecret	CHAP 启动程序密钥。如果 useCHAP=true ，则为必需项。适用于 `ontap-san` 和 `ontap-san-economy`.
ONTAP	chapTargetUsername	目标用户名。如果 useCHAP=true ，则为必需项。适用于 `ontap-san` 和 `ontap-san-economy`.
ONTAP	chapTargetInitiatorSecret	CHAP 目标启动程序密钥。如果 useCHAP=true ，则为必需项。适用于 `ontap-san` 和 `ontap-san-economy`.

在下一步中创建的 `TrdentBackendConfig` 对象的 `sPec.credentials` 字段将引用此步骤中创建的机密。

第 2 步：创建 TridentBackendConfig CR

现在，您可以创建 `TridentBackendConfig` CR 了。在此示例中，使用 `TriventBackendConfig` 对象创建使用` ontap-san `驱动程序的后端，如下所示：

```
$ kubectl -n trident create -f backend-tbc-ontap-san.yaml
```

```
apiVersion: trident.netapp.io/v1
kind: TridentBackendConfig
metadata:
  name: backend-tbc-ontap-san
spec:
  version: 1
  backendName: ontap-san-backend
  storageDriverName: ontap-san
  managementLIF: 10.0.0.1
  dataLIF: 10.0.0.2
  svm: trident_svm
  credentials:
    name: backend-tbc-ontap-san-secret
```

第 3 步：验证 TridentBackendConfig CR 的状态

现在，您已创建 `TridentBackendConfig` CR ，可以验证状态。请参见以下示例：

```
$ kubectl -n trident get tbc backend-tbc-ontap-san
NAME                      BACKEND NAME          BACKEND UUID
PHASE   STATUS
backend-tbc-ontap-san   ontap-san-backend     8d24fce7-6f60-4d4a-8ef6-
bab2699e6ab8   Bound   Success
```

已成功创建后端并将其绑定到 `TridentBackendConfig` CR 。

阶段可以采用以下值之一：

- `bound`： `TridentBackendConfig` CR 与后端关联，后端包含 `configRef` 设置为 `TridentBackendConfig` CR 的 UID 。
- `Unbound`：使用 `""` 表示。`TridentBackendConfig` 对象未绑定到后端。默认情况下，所有新创建的 `TridentBackendConfig` CRS 均处于此阶段。此阶段发生更改后，它将无法再次还原为 "Unbound （已取消绑定） " 。
- `deleting`： The `TridentBackendConfig` CR 's `deletionPolicy` was set to delete.删除 `TridentBackendConfig` CR 后，它将过渡到 Deleting 状态。
  - 如果后端不存在永久性卷请求（ PVC ），则删除 `TridentBackendConfig` 将导致 Astra Trident 删除后端以及 `TridentBackendConfig` CR 。
  - 如果后端存在一个或多个 PVC ，则会进入删除状态。`TridentBackendConfig` CR 随后也会进入删除阶段。只有在删除所有 PVC 后，才会删除后端和 `TridentBackendConfig` 。
- `Lost`：与 `TridentBackendConfig` CR 关联的后端被意外或故意删除， `TridentBackendConfig` CR 仍引用已删除的后端。无论 `detionPolicy` 值如何，仍可删除 `TridentBackendConfig` CR 。
- 未知： Astra Trident 无法确定与 `TridentBackendConfig` CR 关联的后端的状态或存在。例如，如果 API 服务器未响应或缺少 `tridentbackends.trident.netapp.io` CRD 。这可能需要用户干预。

在此阶段，已成功创建后端！此外，还可以处理多个操作，例如 "后端更新和后端删除"。

（可选）第 4 步：获取更多详细信息

您可以运行以下命令来获取有关后端的详细信息：

```
kubectl -n trident get tbc backend-tbc-ontap-san -o wide
```

```
NAME                      BACKEND NAME        BACKEND UUID
PHASE   STATUS    STORAGE DRIVER   DELETION POLICY
backend-tbc-ontap-san   ontap-san-backend   8d24fce7-6f60-4d4a-8ef6-
bab2699e6ab8   Bound   Success   ontap-san        delete
```

此外，您还可以获取 YAML/JSON 转储 `TridentBackendConfig` 。

```
$ kubectl -n trident get tbc backend-tbc-ontap-san -o yaml
```

```
apiVersion: trident.netapp.io/v1
kind: TridentBackendConfig
metadata:
  creationTimestamp: "2021-04-21T20:45:11Z"
  finalizers:
  - trident.netapp.io
  generation: 1
  name: backend-tbc-ontap-san
  namespace: trident
  resourceVersion: "947143"
  uid: 35b9d777-109f-43d5-8077-c74a4559d09c
spec:
  backendName: ontap-san-backend
  credentials:
    name: backend-tbc-ontap-san-secret
  managementLIF: 10.0.0.1
  dataLIF: 10.0.0.2
  storageDriverName: ontap-san
  svm: trident_svm
  version: 1
status:
  backendInfo:
    backendName: ontap-san-backend
    backendUUID: 8d24fce7-6f60-4d4a-8ef6-bab2699e6ab8
  deletionPolicy: delete
  lastOperationStatus: Success
  message: Backend 'ontap-san-backend' created
  phase: Bound
```

`backendInfo` 包含为响应 `TridentBackendConfig` CR 而创建的后端的 `backendName` 和 `backendUUID` 。`lastOperationStatus` 字段表示 `TridentBackendConfig` CR 的上次操作状态，该操作可以是用户触发的（例如，用户在 `sPec` 中更改了某个内容），也可以是由 Astra Trident 触发的（例如，在 Astra Trident 重新启动期间）。可以是成功，也可以是失败。`phase` 表示 `TridentBackendConfig` CR 与后端之间关系的状态。在上面的示例中， `phase` 的值为 bound ，这意味着 `TridentBackendConfig` CR 与后端关联。

您可以运行 `kubectl -n trident describe tbc <tbc-cr-name>` 命令来获取事件日志的详细信息。

> 您不能使用 `tridentctl` 更新或删除包含关联的 `TridentBackendConfig` 对象的后端。要了解在 `tridentctl` 和 `TridentBackendConfig` 之间切换所涉及的步骤， "请参见此处"。

使用 kubectl 执行后端管理

了解如何使用 `kubectl` 执行后端管理操作。

删除后端

通过删除 `TridentBackendConfig` ，您可以指示 Astra Trident 删除 / 保留后端（基于 `detionPolicy` ）。要删除后端，请确保将 `deletionPolicy` 设置为 delete 。要仅删除 `TridentBackendConfig` ，请确保将 `deletionPolicy` 设置为 Retain 。这样可以确保后端仍然存在，并可使用 `tridentctl` 进行管理。

运行以下命令：

```
$ kubectl delete tbc <tbc-name> -n trident
```

Astra Trident 不会删除 `TridentBackendConfig` 使用的 Kubernetes 机密。Kubernetes 用户负责清理密钥。删除机密时必须小心。只有在后端未使用机密时，才应将其删除。

查看现有后端

运行以下命令：

```
$ kubectl get tbc -n trident
```

您也可以运行 `tridentctl get backend -n trident` 或 `tridentctl get backend -o YAML -n trident` 来获取存在的所有后端的列表。此列表还将包括使用 `tridentctl` 创建的后端。

更新后端

更新后端可能有多种原因：

- 存储系统的凭据已更改。要更新凭据，必须更新 `TridentBackendConfig` 对象中使用的 Kubernetes Secret 。Astra Trident 将使用提供的最新凭据自动更新后端。运行以下命令以更新 Kubernetes Secret：

```
$ kubectl apply -f <updated-secret-file.yaml> -n trident
```

- 需要更新参数（例如所使用的 ONTAP SVM 的名称）。在这种情况下，可以直接通过 Kubernetes 更新 `TridentBackendConfig` 对象。

```
$ kubectl apply -f <updated-backend-file.yaml>
```

或者，也可以运行以下命令来更改现有的 `TridentBackendConfig` CR：

```
$ kubectl edit tbc <tbc-name> -n trident
```

如果后端更新失败，则后端仍会保持在其上次已知配置中。您可以通过运行 `kubectl get tbc <tbc-name> -o yaml -n trident` 或 `kubectl describe tbc <tbc-name> -n trident` 来查看日志以确定发生原因。

确定并更正配置文件中的问题后，您可以重新运行 update 命令。

使用 tridentctl 执行后端管理

了解如何使用 `tridentctl` 执行后端管理操作。

创建后端

创建后 "后端配置文件"下，运行以下命令：

```
$ tridentctl create backend -f <backend-file> -n trident
```

如果后端创建失败，则后端配置出现问题。您可以运行以下命令来查看日志以确定发生原因：

```
$ tridentctl logs -n trident
```

确定并更正配置文件中的问题后，您只需再次运行 `create` 命令即可。

删除后端

要从 Astra Trident 中删除后端，请执行以下操作：

1. 检索后端名称：

   ```
   $ tridentctl get backend -n trident
   ```

2. 删除后端：

   ```
   $ tridentctl delete backend <backend-name> -n trident
   ```

> 如果 Astra Trident 从此后端配置了仍存在的卷和快照，则删除后端将阻止其配置新卷。后端将继续处于 " 删除 " 状态，而 Trident 将继续管理这些卷和快照，直到将其删除为止。

查看现有后端

要查看 Trident 了解的后端，请执行以下操作：

- 要获取摘要，请运行以下命令：

```
$ tridentctl get backend -n trident
```

- 要获取所有详细信息，请运行以下命令：

```
$ tridentctl get backend -o json -n trident
```

更新后端

创建新的后端配置文件后，运行以下命令：

```
$ tridentctl update backend <backend-name> -f <backend-file> -n trident
```

如果后端更新失败，则后端配置出现问题或您尝试的更新无效。您可以运行以下命令来查看日志以确定发生原因：

```
$ tridentctl logs -n trident
```

确定并更正配置文件中的问题后，您只需再次运行 `update` 命令即可。

确定使用后端的存储类

以下是您可以使用问题解答与 JSON 回答的问题的示例，这些问题会 `tridentctl` 后端对象的输出。此操作将使用 `JQ` 实用程序，您需要安装该实用程序。

```
$ tridentctl get backend -o json | jq '[.items[] | {backend: .name,
storageClasses: [.storage[].storageClasses]|unique}]'
```

这也适用于使用 `TridentBackendConfig` 创建的后端。

在后端管理选项之间移动

了解如何在 Astra Trident 中管理后端。随着 `TridentBackendConfig` 的推出，管理员现在可以通过两种独特的方式管理后端。这会提出以下问题：

- 使用 `tridentctl` 创建的后端是否可以使用 `TridentBackendConfig` 进行管理？

- 使用 `TridentBackendConfig` 创建的后端是否可以使用 `tridentctl` 进行管理?

使用 TridentBackendConfig 管理 tridentctl 后端

本节介绍通过创建 `TridentBackendConfig` 对象直接通过 Kubernetes 界面管理使用 `tridentctl` 创建的后端所需的步骤。

这适用于以下情形：

- 预先存在的后端，没有 `TridentBackendConfig` ，因为它们是使用 `tridentctl` 创建的。
- 使用 `tridentctl` 创建的新后端，而存在其他 `TridentBackendConfig` 对象。

在这两种情况下，后端仍会存在，其中 Astra Trident 会计划卷并对其进行操作。管理员可以选择以下两种方式之一：

- 继续使用 `tridentctl` 管理使用它创建的后端。
- 使用 `tridentctl` 创建的后端绑定到新的 `TridentBackendConfig` 对象。这样做意味着将使用 `kubectl` 而不是 `tridentctl` 来管理后端。

要使用 `kubectl` 管理已有后端，您需要创建一个绑定到现有后端的 `TridentBackendConfig` 。下面简要介绍了它的工作原理：

1. 创建 Kubernetes 机密。此密钥包含 Astra Trident 与存储集群 / 服务通信所需的凭据。
2. 创建 `TridentBackendConfig` 对象。其中包含有关存储集群 / 服务的详细信息，并引用了上一步中创建的密钥。必须注意指定相同的配置参数（例如 `sPec.backendName` ， `sPec.storagePrefix` ， `sPec.storageDriverName` 等）。`sPec.backendName` 必须设置为现有后端的名称。

第 **0** 步：确定后端

要创建绑定到现有后端的 `TridentBackendConfig` ，您需要获取后端的配置。在此示例中，假设已使用以下 JSON 定义创建了后端：

```
$ tridentctl get backend ontap-nas-backend -n trident
+---------------------+----------------
+--------------------------------------+--------+---------+
|          NAME       | STORAGE DRIVER |                  UUID
| STATE  | VOLUMES |
+---------------------+----------------
+--------------------------------------+--------+---------+
| ontap-nas-backend   | ontap-nas      | 52f2eb10-e4c6-4160-99fc-
96b3be5ab5d7 | online |      25 |
+---------------------+----------------
+--------------------------------------+--------+---------+

$ cat ontap-nas-backend.json

{
```

```
    "version": 1,
    "storageDriverName": "ontap-nas",
    "managementLIF": "10.10.10.1",
    "dataLIF": "10.10.10.2",
    "backendName": "ontap-nas-backend",
    "svm": "trident_svm",
    "username": "cluster-admin",
    "password": "admin-password",

    "defaults": {
        "spaceReserve": "none",
        "encryption": "false"
    },
    "labels":{"store":"nas_store"},
    "region": "us_east_1",
    "storage": [
        {
            "labels":{"app":"msoffice", "cost":"100"},
            "zone":"us_east_1a",
            "defaults": {
                "spaceReserve": "volume",
                "encryption": "true",
                "unixPermissions": "0755"
            }
        },
        {
            "labels":{"app":"mysqldb", "cost":"25"},
            "zone":"us_east_1d",
            "defaults": {
                "spaceReserve": "volume",
                "encryption": "false",
                "unixPermissions": "0775"
            }
        }
    ]
}
```

第 **1** 步：创建 **Kubernetes** 机密

创建一个包含后端凭据的机密，如以下示例所示：

```
$ cat tbc-ontap-nas-backend-secret.yaml

apiVersion: v1
kind: Secret
metadata:
  name: ontap-nas-backend-secret
type: Opaque
stringData:
  username: cluster-admin
  passWord: admin-password

$ kubectl create -f tbc-ontap-nas-backend-secret.yaml -n trident
secret/backend-tbc-ontap-san-secret created
```

第 **2** 步：创建 **TridentBackendConfig CR**

下一步是创建一个 `TridentBackendConfig` CR ，该 CR 将自动绑定到已有的 `ontap-nas-backend` （如本示例所示）。确保满足以下要求：

- 在 `sPec.backendName` 中定义了相同的后端名称。
- 配置参数与原始后端相同。
- 虚拟存储池（如果存在）必须与原始后端保持相同的顺序。
- 凭据通过 Kubernetes Secret 提供，而不是以纯文本形式提供。

在这种情况下， `TridentBackendConfig` 将如下所示：

```
$ cat backend-tbc-ontap-nas.yaml
apiVersion: trident.netapp.io/v1
kind: TridentBackendConfig
metadata:
  name: tbc-ontap-nas-backend
spec:
  version: 1
  storageDriverName: ontap-nas
  managementLIF: 10.10.10.1
  dataLIF: 10.10.10.2
  backendName: ontap-nas-backend
  svm: trident_svm
  credentials:
    name: mysecret
  defaults:
    spaceReserve: none
    encryption: 'false'
  labels:
    store: nas_store
  region: us_east_1
  storage:
  - labels:
      app: msoffice
      cost: '100'
    zone: us_east_1a
    defaults:
      spaceReserve: volume
      encryption: 'true'
      unixPermissions: '0755'
  - labels:
      app: mysqldb
      cost: '25'
    zone: us_east_1d
    defaults:
      spaceReserve: volume
      encryption: 'false'
      unixPermissions: '0775'

$ kubectl create -f backend-tbc-ontap-nas.yaml -n trident
tridentbackendconfig.trident.netapp.io/tbc-ontap-nas-backend created
```

第 **3** 步：验证 **TridentBackendConfig CR** 的状态

创建 `TridentBackendConfig` 后，其阶段必须为 `bound` 。它还应反映与现有后端相同的后端名称和 UUID 。

```
$ kubectl -n trident get tbc tbc-ontap-nas-backend -n trident
NAME                    BACKEND NAME          BACKEND UUID
PHASE   STATUS
tbc-ontap-nas-backend  ontap-nas-backend     52f2eb10-e4c6-4160-99fc-
96b3be5ab5d7   Bound   Success

#confirm that no new backends were created (i.e., TridentBackendConfig did
not end up creating a new backend)
$ tridentctl get backend -n trident
+---------------------+----------------
+--------------------------------------+--------+---------+
|          NAME       | STORAGE DRIVER |                  UUID
| STATE  | VOLUMES |
+---------------------+----------------
+--------------------------------------+--------+---------+
| ontap-nas-backend   | ontap-nas      | 52f2eb10-e4c6-4160-99fc-
96b3be5ab5d7 | online |      25 |
+---------------------+----------------
+--------------------------------------+--------+---------+
```

现在，可以使用 `tbc-ontap-nas-backend` `TridentBackendConfig` 对象对后端进行全面管理。

使用 tridentctl 管理 TridentBackendConfig 后端

`tridentctl` 可用于列出使用 `TridentBackendConfig` 创建的后端。此外，管理员还可以选择通过 `tridentctl` 来完全管理此类后端，方法是删除 `TridentBackendConfig` 并确保将 `spec.deletionPolicy` 设置为 `retain` 。

第 0 步：确定后端

例如，假设使用 `TridentBackendConfig` 创建了以下后端：

```
$ kubectl get tbc backend-tbc-ontap-san -n trident -o wide
NAME                     BACKEND NAME        BACKEND UUID
PHASE   STATUS    STORAGE DRIVER   DELETION POLICY
backend-tbc-ontap-san   ontap-san-backend   81abcb27-ea63-49bb-b606-
0a5315ac5f82   Bound   Success   ontap-san        delete

$ tridentctl get backend ontap-san-backend -n trident
+-------------------+----------------
+--------------------------------------+--------+---------+
|       NAME        | STORAGE DRIVER |                 UUID
| STATE  | VOLUMES |
+-------------------+----------------
+--------------------------------------+--------+---------+
| ontap-san-backend | ontap-san      | 81abcb27-ea63-49bb-b606-
0a5315ac5f82 | online |      33 |
+-------------------+----------------
+--------------------------------------+--------+---------+
```

从输出中可以看到 `TridentBackendConfig` 已成功创建并绑定到后端【观察后端的 UUUID] 。

第 1 步：确认 deletionPolicy 设置为 Retain

让我们来看看 `deletionPolicy` 的价值。需要将此值设置为 `retain` 。这样可以确保删除 `TridentBackendConfig` CR 时，后端定义仍存在，并可使用 `tridentctl` 进行管理。

```
$ kubectl get tbc backend-tbc-ontap-san -n trident -o wide
NAME                     BACKEND NAME        BACKEND UUID
PHASE   STATUS    STORAGE DRIVER   DELETION POLICY
backend-tbc-ontap-san   ontap-san-backend   81abcb27-ea63-49bb-b606-
0a5315ac5f82   Bound   Success   ontap-san        delete

# Patch value of deletionPolicy to retain
$ kubectl patch tbc backend-tbc-ontap-san --type=merge -p
'{"spec":{"deletionPolicy":"retain"}}' -n trident
tridentbackendconfig.trident.netapp.io/backend-tbc-ontap-san patched

#Confirm the value of deletionPolicy
$ kubectl get tbc backend-tbc-ontap-san -n trident -o wide
NAME                     BACKEND NAME        BACKEND UUID
PHASE   STATUS    STORAGE DRIVER   DELETION POLICY
backend-tbc-ontap-san   ontap-san-backend   81abcb27-ea63-49bb-b606-
0a5315ac5f82   Bound   Success   ontap-san        retain
```

请勿继续执行下一步，除非将 `deletionPolicy` 设置为 `retain` 。

第 2 步：删除 TridentBackendConfig CR

最后一步是删除 `TridentBackendConfig` CR 。确认 `deeltionPolicy` 设置为 `retain` 后，您可以继续执行删除：

```
$ kubectl delete tbc backend-tbc-ontap-san -n trident
tridentbackendconfig.trident.netapp.io "backend-tbc-ontap-san" deleted

$ tridentctl get backend ontap-san-backend -n trident
+--------------------+----------------
+--------------------------------------+--------+---------+
|       NAME         | STORAGE DRIVER |                  UUID
| STATE   | VOLUMES |
+--------------------+----------------
+--------------------------------------+--------+---------+
| ontap-san-backend | ontap-san      | 81abcb27-ea63-49bb-b606-
0a5315ac5f82 | online |      33 |
+--------------------+----------------
+--------------------------------------+--------+---------+
```

删除 `TridentBackendConfig` 对象后， Astra Trident 只需删除该对象，而无需实际删除后端本身。

管理存储类

查找有关创建存储类，删除存储类以及查看现有存储类的信息。

设计存储类

请参见 "此处" 有关什么是存储类以及如何配置这些类的详细信息，请参见。

创建存储类。

创建存储类文件后，运行以下命令：

```
kubectl create -f <storage-class-file>
```

` <storage-class-file>` 应替换为存储类文件名。

删除存储类

要从 Kubernetes 中删除存储类，请运行以下命令：

```
kubectl delete storageclass <storage-class>
```

`< 存储类 >` 应替换为您的存储类。

通过此存储类创建的任何永久性卷将保持不变， Astra Trident 将继续对其进行管理。

> Astra Trident 会为其创建的卷强制使用空的 `FSType` 。对于 iSCSI 后端，建议在 StorageClass 中强制实施 `parameters.FSType` 。您应删除现有 StorageClasses 并使用指定的 `parameters.FSType` 重新创建它们。

查看现有存储类

- 要查看现有 Kubernetes 存储类，请运行以下命令：

```
kubectl get storageclass
```

- 要查看 Kubernetes 存储类详细信息，请运行以下命令：

```
kubectl get storageclass <storage-class> -o json
```

- 要查看 Astra Trident 的同步存储类，请运行以下命令：

```
tridentctl get storageclass
```

- 要查看 Astra Trident 的同步存储类详细信息，请运行以下命令：

```
tridentctl get storageclass <storage-class> -o json
```

设置默认存储类

Kubernetes 1.6 增加了设置默认存储类的功能。如果用户未在永久性卷声明（ PVC ）中指定永久性卷，则此存储类将用于配置永久性卷。

- 通过在存储类定义中将标注 `storageclass.Kubernetes 。 io/is-default-class` 设置为 true 来定义默认存储类。根据规范，任何其他值或标注不存在均视为 false 。
- 您可以使用以下命令将现有存储类配置为默认存储类：

```
kubectl patch storageclass <storage-class-name> -p '{"metadata":
{"annotations":{"storageclass.kubernetes.io/is-default-class":"true"}}}'
```

- 同样，您也可以使用以下命令删除默认存储类标注：

```
kubectl patch storageclass <storage-class-name> -p '{"metadata":
{"annotations":{"storageclass.kubernetes.io/is-default-class":"false"}}}'
```

Trident 安装程序包中也有包含此标注的示例。

> 在任何给定时间，集群中只能有一个默认存储类。Kubernetes 在技术上不会阻止您拥有多个存储类，但其行为就像根本没有默认存储类一样。

确定存储类的后端

以下是您可以使用问题解答与 JSON 一起为 Astra Trident 后端对象输出的 `tridentctl` 类问题的示例。此操作将使用 `JQ` 实用程序，您可能需要先安装该实用程序。

```
tridentctl get storageclass -o json | jq  '[.items[] | {storageClass:
.Config.name, backends: [.storage]|unique}]'
```

执行卷操作

了解 Astra Trident 为管理卷提供的功能。

- "使用 CSI 拓扑"
- "使用快照"
- "展开卷"
- "导入卷"

使用 **CSI** 拓扑

Astra Trident 可以通过使用有选择地创建卷并将其附加到 Kubernetes 集群中的节点 "CSI 拓扑功能"。使用 CSI 拓扑功能，可以根据区域和可用性区域将对卷的访问限制为一小部分节点。如今，借助云提供商， Kubernetes 管理员可以生成基于分区的节点。节点可以位于一个区域内的不同可用性区域中，也可以位于不同区域之间。为了便于在多区域架构中为工作负载配置卷， Astra Trident 使用了 CSI 拓扑。

> 了解有关 CSI 拓扑功能的更多信息 "此处"。

Kubernetes 提供了两种唯一的卷绑定模式：

- 如果将 `VolumeBindingMode` 设置为 `immediate` ，则 Astra Trident 将创建卷，而不会感知任何拓扑。创建 PVC 时会处理卷绑定和动态配置。这是默认值 `VolumeBindingMode` ，适用于不强制实施拓扑约束的集群。创建永久性卷时，不会依赖于请求的 Pod 的计划要求。
- 如果将 `VolumeBindingMode` 设置为 `WaitForFirstConsuming` ，则为 PVC 创建和绑定永久性卷的操作将延迟，直到计划并创建使用 PVC 的 Pod 为止。这样，卷就会根据拓扑要求强制实施的计划限制来创建。

> `WaitForFirstConsumer"` 绑定模式不需要拓扑标签。此功能可独立于 CSI 拓扑功能使用。

要使用 CSI 拓扑，您需要满足以下条件：

- 运行 1.17 或更高版本的 Kubernetes 集群。

```
$ kubectl version
Client Version: version.Info{Major:"1", Minor:"19",
GitVersion:"v1.19.3",
GitCommit:"1e11e4a2108024935ecfcb2912226cedeafd99df",
GitTreeState:"clean", BuildDate:"2020-10-14T12:50:19Z",
GoVersion:"go1.15.2", Compiler:"gc", Platform:"linux/amd64"}
Server Version: version.Info{Major:"1", Minor:"19",
GitVersion:"v1.19.3",
GitCommit:"1e11e4a2108024935ecfcb2912226cedeafd99df",
GitTreeState:"clean", BuildDate:"2020-10-14T12:41:49Z",
GoVersion:"go1.15.2", Compiler:"gc", Platform:"linux/amd64"}
```

- 集群中的节点应具有可引入拓扑感知的标签（`topology.Kubernetes IO/` 区域 和 `topology.Kubernetes IO/` 区域 ）。在安装 Astra Trident 之前，集群中的节点上应存在这些标签 *，以使 Astra Trident 能够识别拓扑。

```
$ kubectl get nodes -o=jsonpath='{range .items[*]}[{.metadata.name},
{.metadata.labels}]{"\n"}{end}' | grep --color "topology.kubernetes.io"
[node1,
{"beta.kubernetes.io/arch":"amd64","beta.kubernetes.io/os":"linux","kube
rnetes.io/arch":"amd64","kubernetes.io/hostname":"node1","kubernetes.io/
os":"linux","node-
role.kubernetes.io/master":"","topology.kubernetes.io/region":"us-
east1","topology.kubernetes.io/zone":"us-east1-a"}]
[node2,
{"beta.kubernetes.io/arch":"amd64","beta.kubernetes.io/os":"linux","kube
rnetes.io/arch":"amd64","kubernetes.io/hostname":"node2","kubernetes.io/
os":"linux","node-
role.kubernetes.io/worker":"","topology.kubernetes.io/region":"us-
east1","topology.kubernetes.io/zone":"us-east1-b"}]
[node3,
{"beta.kubernetes.io/arch":"amd64","beta.kubernetes.io/os":"linux","kube
rnetes.io/arch":"amd64","kubernetes.io/hostname":"node3","kubernetes.io/
os":"linux","node-
role.kubernetes.io/worker":"","topology.kubernetes.io/region":"us-
east1","topology.kubernetes.io/zone":"us-east1-c"}]
```

第 **1** 步：创建可感知拓扑的后端

可以设计 Astra Trident 存储后端，以便根据可用性区域有选择地配置卷。每个后端都可以包含一个可选的

`supportedTopatis` 块，该块表示必须支持的分区和区域的列表。对于使用此后端的 StorageClasses ，只有在受支持区域 / 区域中计划的应用程序请求时，才会创建卷。

下面是后端定义示例：

```
{
 "version": 1,
 "storageDriverName": "ontap-san",
 "backendName": "san-backend-us-east1",
 "managementLIF": "192.168.27.5",
 "svm": "iscsi_svm",
 "username": "admin",
 "password": "xxxxxxxxxxxx",
 "supportedTopologies": [
{"topology.kubernetes.io/region": "us-east1",
"topology.kubernetes.io/zone": "us-east1-a"},
{"topology.kubernetes.io/region": "us-east1",
"topology.kubernetes.io/zone": "us-east1-b"}
]
}
```

> `supportedTopologies` 用于提供每个后端的区域和分区列表。这些区域和分区表示可在 StorageClass 中提供的允许值列表。对于包含后端提供的部分区域和分区的 StorageClasses ，Astra Trident 将在后端创建卷。

您也可以为每个存储池定义 `supportedTopologies` 。请参见以下示例：

```
{"version": 1,
"storageDriverName": "ontap-nas",
"backendName": "nas-backend-us-central1",
"managementLIF": "172.16.238.5",
"svm": "nfs_svm",
"username": "admin",
"password": "Netapp123",
"supportedTopologies": [
      {"topology.kubernetes.io/region": "us-central1",
"topology.kubernetes.io/zone": "us-central1-a"},
      {"topology.kubernetes.io/region": "us-central1",
"topology.kubernetes.io/zone": "us-central1-b"}
    ]
"storage": [
   {
       "labels": {"workload":"production"},
        "region": "Iowa-DC",
        "zone": "Iowa-DC-A",
        "supportedTopologies": [
            {"topology.kubernetes.io/region": "us-central1",
"topology.kubernetes.io/zone": "us-central1-a"}
        ]
    },
    {
        "labels": {"workload":"dev"},
         "region": "Iowa-DC",
         "zone": "Iowa-DC-B",
         "supportedTopologies": [
             {"topology.kubernetes.io/region": "us-central1",
"topology.kubernetes.io/zone": "us-central1-b"}
         ]
     }
]
}
```

在此示例中， `reGion` 和 `zone` 标签表示存储池的位置。`topology.Kubernees.io/zone` 和 `topology.Kubernees.io/zone` 指定存储池的使用位置。

第 **2** 步：定义可识别拓扑的 **StorageClasses**

根据为集群中的节点提供的拓扑标签，可以将 StorageClasses 定义为包含拓扑信息。这将确定用作 PVC 请求候选对象的存储池，以及可使用 Trident 配置的卷的节点子集。

请参见以下示例：

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
name: netapp-san-us-east1
provisioner: csi.trident.netapp.io
volumeBindingMode: WaitForFirstConsumer
allowedTopologies:
- matchLabelExpressions:
- key: topology.kubernetes.io/zone
  values:
  - us-east1-a
  - us-east1-b
- key: topology.kubernetes.io/region
  values:
  - us-east1
parameters:
  fsType: "ext4"
```

在上述 StorageClass 定义中， `volumeBindingMode` 设置为 `WaitForFirstConsumer"` 。在此存储类中请求的 PVC 在 Pod 中引用之前不会执行操作。此外， `allowedTopologies` 还提供了要使用的分区和区域。`NetApp-san-us-East1` StorageClass 将在上述 `san-backend-us-East1` 后端创建 PVC 。

第 **3** 步：创建和使用 **PVC**

创建 StorageClass 并将其映射到后端后，您现在可以创建 PVC 。

请参见以下示例 `sPec`：

```
---
kind: PersistentVolumeClaim
apiVersion: v1
metadata:
name: pvc-san
spec:
accessModes:
  - ReadWriteOnce
resources:
  requests:
    storage: 300Mi
storageClassName: netapp-san-us-east1
```

使用此清单创建 PVC 将导致以下结果：

```
$ kubectl create -f pvc.yaml
persistentvolumeclaim/pvc-san created
$ kubectl get pvc
NAME      STATUS    VOLUME   CAPACITY   ACCESS MODES   STORAGECLASS
AGE
pvc-san   Pending                                      netapp-san-us-east1
2s
$ kubectl describe pvc
Name:          pvc-san
Namespace:     default
StorageClass:  netapp-san-us-east1
Status:        Pending
Volume:
Labels:        <none>
Annotations:   <none>
Finalizers:    [kubernetes.io/pvc-protection]
Capacity:
Access Modes:
VolumeMode:    Filesystem
Mounted By:    <none>
Events:
  Type    Reason                Age   From                         Message
  ----    ------                ----  ----                         -------
  Normal  WaitForFirstConsumer  6s    persistentvolume-controller  waiting
for first consumer to be created before binding
```

要使 Trident 创建卷并将其绑定到 PVC ，请在 Pod 中使用 PVC 。请参见以下示例：

```
apiVersion: v1
kind: Pod
metadata:
  name: app-pod-1
spec:
  affinity:
    nodeAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:
        nodeSelectorTerms:
        - matchExpressions:
          - key: topology.kubernetes.io/region
            operator: In
            values:
            - us-east1
      preferredDuringSchedulingIgnoredDuringExecution:
      - weight: 1
        preference:
          matchExpressions:
          - key: topology.kubernetes.io/zone
            operator: In
            values:
            - us-east1-a
            - us-east1-b
  securityContext:
    runAsUser: 1000
    runAsGroup: 3000
    fsGroup: 2000
  volumes:
  - name: vol1
    persistentVolumeClaim:
      claimName: pvc-san
  containers:
  - name: sec-ctx-demo
    image: busybox
    command: [ "sh", "-c", "sleep 1h" ]
    volumeMounts:
    - name: vol1
      mountPath: /data/demo
    securityContext:
      allowPrivilegeEscalation: false
```

此 podSpec 指示 Kubernetes 在 `us-East1` 区域中的节点上计划 Pod ，并从 `us-East1-a` 或 `us-East1-b` 区域中的任何节点中进行选择。

请参见以下输出：

```
$ kubectl get pods -o wide
NAME        READY   STATUS    RESTARTS   AGE   IP                NODE
NOMINATED NODE   READINESS GATES
app-pod-1   1/1     Running   0          19s   192.168.25.131   node2
<none>           <none>
$ kubectl get pvc -o wide
NAME      STATUS   VOLUME                                     CAPACITY
ACCESS MODES   STORAGECLASS          AGE   VOLUMEMODE
pvc-san   Bound    pvc-ecb1e1a0-840c-463b-8b65-b3d033e2e62b   300Mi
RWO            netapp-san-us-east1   48s   Filesystem
```

更新后端以包括支持的拓扑

可以使用 `tridentctl backend update` 更新原有后端，以包含 `supportedTopologies` 列表。这不会影响已配置的卷，并且仅用于后续的 PVC 。

了解更多信息

- "管理容器的资源"
- "节点选择器"
- "关联性和反关联性"
- "损害和公差"

使用快照

从 Astra Trident 的 20.01 版开始，您可以在 Kubernetes 层创建 PV 快照。您可以使用这些快照维护由 Astra Trident 创建的卷的时间点副本，并计划创建其他卷（克隆）。卷快照受 `ontap-nas` ， `ontap-san` ， `ontap-san-economics` ， `solidfire-san` ， `GCP-CVS` 支持。 和 `azure-netapp-files` 驱动程序。

> 此功能可从 Kubernetes 1.17 （测试版）获得，从 1.20 开始正式上市。要了解从测试版迁移到 GA 所涉及的变化，请参见 "发布博客"。升级到 GA 后，将推出 `v1` API 版本，并向后兼容 `v1bea1` 快照。

您需要什么？ **#8217**；将需要什么

- 创建卷快照需要创建外部快照控制器和自定义资源定义(Custom Resource Definitions、CRD)。这是所使用的Kubernetes流程编排程序(例如：Kubeadm、GKE、OpenShift)的责任。

如果您的Kubernetes分发版不包含快照控制器和CRD、则可以按如下所示进行部署。

1. 创建卷快照CRD。

   对于Kubernetes 1.20及更高版本、请使用v5.0或更高版本的Snapshot组件的v1 Snapshot CRD。对于Kubernetes 1.17-1.19版、请将v1bea1与v3.0.3 Snapshot组件结合使用。

5.0版组件

```
$ cat snapshot-setup.sh
#!/bin/bash
# Create volume snapshot CRDs
kubectl apply -f https://raw.githubusercontent.com/kubernetes-
csi/external-snapshotter/release-
5.0/client/config/crd/snapshot.storage.k8s.io_volumesnapshotclasses.
yaml
kubectl apply -f https://raw.githubusercontent.com/kubernetes-
csi/external-snapshotter/release-
5.0/client/config/crd/snapshot.storage.k8s.io_volumesnapshotcontents
.yaml
kubectl apply -f https://raw.githubusercontent.com/kubernetes-
csi/external-snapshotter/release-
5.0/client/config/crd/snapshot.storage.k8s.io_volumesnapshots.yaml
```

v3.0.3组件

```
$ cat snapshot-setup.sh
#!/bin/bash
# Create volume snapshot CRDs
kubectl apply -f https://raw.githubusercontent.com/kubernetes-
csi/external-
snapshotter/v3.0.3/client/config/crd/snapshot.storage.k8s.io_volumes
napshotclasses.yaml
kubectl apply -f https://raw.githubusercontent.com/kubernetes-
csi/external-
snapshotter/v3.0.3/client/config/crd/snapshot.storage.k8s.io_volumes
napshotcontents.yaml
kubectl apply -f https://raw.githubusercontent.com/kubernetes-
csi/external-
snapshotter/v3.0.3/client/config/crd/snapshot.storage.k8s.io_volumes
napshots.yaml
```

2. 在所需命名空间中创建Snapshot控制器。编辑以下 YAML 清单以修改命名空间。

   对于Kubernetes 1.20及更高版本、请使用5.0或更高版本。对于Kubernetes 1.17-1.19、请使用v3.0.3

   > 如果在GKE-环境中设置按需卷快照、请勿创建快照控制器。GKE- 使用内置的隐藏快照控制器。

5.0版控制器

```
kubectl apply -f https://raw.githubusercontent.com/kubernetes-csi/external-snapshotter/release-5.0/deploy/kubernetes/snapshot-controller/rbac-snapshot-controller.yaml
kubectl apply -f https://raw.githubusercontent.com/kubernetes-csi/external-snapshotter/release-5.0/deploy/kubernetes/snapshot-controller/setup-snapshot-controller.yaml
```

v3.0.3控制器

```
kubectl apply -f https://raw.githubusercontent.com/kubernetes-csi/external-snapshotter/v3.0.3/deploy/kubernetes/snapshot-controller/rbac-snapshot-controller.yaml
kubectl apply -f https://raw.githubusercontent.com/kubernetes-csi/external-snapshotter/v3.0.3/deploy/kubernetes/snapshot-controller/setup-snapshot-controller.yaml
```

CSI Snapshotter 提供了 "正在验证 webhook" 帮助用户验证现有 v1bea1 快照并确认它们是有效的资源对象。正在验证的 webhook 会自动标记无效的快照对象，并防止将来创建无效对象。验证 webhook 由 Kubernetes 流程编排程序部署。请参见有关手动部署验证 webhook 的说明 "此处"。查找无效快照清单的示例 "此处"。

下面详细介绍的示例说明了使用快照所需的构造，并说明了如何创建和使用快照。

第 **1** 步：设置 **VolumeSnapshotClass**

在创建卷快照之前，请设置 “VolumeSnapshotClass”。

```
$ cat snap-sc.yaml
#Use apiVersion v1 for Kubernetes 1.20 and above. For Kubernetes 1.17 - 1.19, use apiVersion v1beta1.
apiVersion: snapshot.storage.k8s.io/v1
kind: VolumeSnapshotClass
metadata:
  name: csi-snapclass
driver: csi.trident.netapp.io
deletionPolicy: Delete
```

d河 指向 Astra Trident 的 CSI 驱动程序。`deeltionPolicy` 可以是 `Delete` 或 `Retain` 。如果设置为 `Retain` ，则即使删除了 `VolumeSnapshot` 对象，存储集群上的底层物理快照也会保留。

第 **2** 步：创建现有 **PVC** 的快照

```
$ cat snap.yaml
#Use apiVersion v1 for Kubernetes 1.20 and above. For Kubernetes 1.17 -
1.19, use apiVersion v1beta1.
apiVersion: snapshot.storage.k8s.io/v1
kind: VolumeSnapshot
metadata:
  name: pvc1-snap
spec:
  volumeSnapshotClassName: csi-snapclass
  source:
    persistentVolumeClaimName: pvc1
```

正在为名为 `PVC1` 的 PVC 创建快照，并且快照名称设置为 `PVC1-Snap` 。

```
$ kubectl create -f snap.yaml
volumesnapshot.snapshot.storage.k8s.io/pvc1-snap created

$ kubectl get volumesnapshots
NAME                   AGE
pvc1-snap              50s
```

这样就创建了一个 `VolumeSnapshot` 对象。VolumeSnapshot 类似于 PVC ，并与代表实际快照的 `VolumeSnapshotContent` 对象关联。

可以通过描述来标识 `PVC1-Snap` VolumeSnapshot 的 `VolumeSnapshotContent` 对象。

```
$ kubectl describe volumesnapshots pvc1-snap
Name:         pvc1-snap
Namespace:    default
.
.
.
Spec:
  Snapshot Class Name:    pvc1-snap
  Snapshot Content Name:  snapcontent-e8d8a0ca-9826-11e9-9807-525400f3f660
  Source:
    API Group:
    Kind:       PersistentVolumeClaim
    Name:       pvc1
Status:
  Creation Time:  2019-06-26T15:27:29Z
  Ready To Use:   true
  Restore Size:   3Gi
.
.
```

`Snapshot Content Name` 用于标识提供此快照的 VolumeSnapshotContent 对象。`Ready to Use` 参数表示可使用 Snapshot 创建新的 PVC 。

第 3 步：从 VolumeSnapshots 创建 PVC

有关使用快照创建 PVC 的示例，请参见以下示例：

```
$ cat pvc-from-snap.yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: pvc-from-snap
spec:
  accessModes:
    - ReadWriteOnce
  storageClassName: golden
  resources:
    requests:
      storage: 3Gi
  dataSource:
    name: pvc1-snap
    kind: VolumeSnapshot
    apiGroup: snapshot.storage.k8s.io
```

`dataSource` 显示必须使用名为 `PVC1-Snap` 的 VolumeSnapshot 作为数据源来创建 PVC 。此操作将指示 Astra Trident 从快照创建 PVC 。创建 PVC 后，可以将其附加到 Pod 上，并像使用任何其他 PVC 一样使用。

> 删除具有关联快照的永久性卷时，相应的 Trident 卷将更新为 " 正在删除 " 状态。要删除 Astra Trident 卷，应删除该卷的快照。

了解更多信息

- "卷快照"
- “VolumeSnapshotClass”

展开卷

通过 Astra Trident ， Kubernetes 用户可以在创建卷后对其进行扩展。查找有关扩展 iSCSI 和 NFS 卷所需配置的信息。

展开 **iSCSI** 卷

您可以使用 CSI 配置程序扩展 iSCSI 永久性卷（ PV ）。

> iSCSI 卷扩展受 `ontap-san` ， `ontap-san-economy` ， `solidfire-san` 驱动程序支持，需要 Kubernetes 1.16 及更高版本。

扩展 iSCSI PV 包括以下步骤：

- 编辑 StorageClass 定义以将 `allowVolumeExpansion` 字段设置为 `true` 。
- 编辑 PVC 定义并更新 `sPec.resources.requests.storage` 以反映新需要的大小，该大小必须大于原始大小。
- 要调整 PV 大小，必须将 PV 连接到 Pod 。调整 iSCSI PV 大小时，有两种情况：
  - 如果 PV 连接到 Pod ，则 Astra Trident 会扩展存储后端的卷，重新扫描设备并调整文件系统大小。
  - 尝试调整未连接 PV 的大小时， Astra Trident 会扩展存储后端的卷。将 PVC 绑定到 Pod 后， Trident 会重新扫描设备并调整文件系统大小。然后， Kubernetes 会在扩展操作成功完成后更新 PVC 大小。

以下示例显示了扩展 iSCSI PV 的工作原理。

第 **1** 步：配置 **StorageClass** 以支持卷扩展

```
$ cat storageclass-ontapsan.yaml
---
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: ontap-san
provisioner: csi.trident.netapp.io
parameters:
  backendType: "ontap-san"
allowVolumeExpansion: True
```

对于已存在的 StorageClass ，请对其进行编辑，使其包含 `allowVolumeExpansion` 参数。

第 2 步：使用您创建的 StorageClass 创建 PVC

```
$ cat pvc-ontapsan.yaml
kind: PersistentVolumeClaim
apiVersion: v1
metadata:
  name: san-pvc
spec:
  accessModes:
  - ReadWriteOnce
  resources:
    requests:
      storage: 1Gi
  storageClassName: ontap-san
```

Astra Trident 会创建一个永久性卷（ PV ）并将其与此永久性卷声明（ PVC ）关联。

```
$ kubectl get pvc
NAME      STATUS   VOLUME                                     CAPACITY
ACCESS MODES   STORAGECLASS   AGE
san-pvc   Bound    pvc-8a814d62-bd58-4253-b0d1-82f2885db671   1Gi
RWO            ontap-san      8s

$ kubectl get pv
NAME                                       CAPACITY   ACCESS MODES
RECLAIM POLICY   STATUS   CLAIM             STORAGECLASS   REASON   AGE
pvc-8a814d62-bd58-4253-b0d1-82f2885db671   1Gi        RWO
Delete           Bound    default/san-pvc   ontap-san               10s
```

第 3 步：定义连接 PVC 的 POD

在此示例中，创建了一个使用 `san-PVC` 的 Pod 。

```
$  kubectl get pod
NAME         READY   STATUS    RESTARTS   AGE
centos-pod   1/1     Running   0          65s

$  kubectl describe pvc san-pvc
Name:          san-pvc
Namespace:     default
StorageClass:  ontap-san
Status:        Bound
Volume:        pvc-8a814d62-bd58-4253-b0d1-82f2885db671
Labels:        <none>
Annotations:   pv.kubernetes.io/bind-completed: yes
               pv.kubernetes.io/bound-by-controller: yes
               volume.beta.kubernetes.io/storage-provisioner:
csi.trident.netapp.io
Finalizers:    [kubernetes.io/pvc-protection]
Capacity:      1Gi
Access Modes:  RWO
VolumeMode:    Filesystem
Mounted By:    centos-pod
```

第 4 步：展开 PV

要将已创建的 PV 从 1Gi 调整为 2Gi ，请编辑 PVC 定义并将 `sPec.resources.requests.storage` 更新为 2Gi 。

```
$ kubectl edit pvc san-pvc
# Please edit the object below. Lines beginning with a '#' will be
ignored,
# and an empty file will abort the edit. If an error occurs while saving
this file will be
# reopened with the relevant failures.
#
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  annotations:
    pv.kubernetes.io/bind-completed: "yes"
    pv.kubernetes.io/bound-by-controller: "yes"
    volume.beta.kubernetes.io/storage-provisioner: csi.trident.netapp.io
  creationTimestamp: "2019-10-10T17:32:29Z"
  finalizers:
  - kubernetes.io/pvc-protection
  name: san-pvc
  namespace: default
  resourceVersion: "16609"
  selfLink: /api/v1/namespaces/default/persistentvolumeclaims/san-pvc
  uid: 8a814d62-bd58-4253-b0d1-82f2885db671
spec:
  accessModes:
  - ReadWriteOnce
  resources:
    requests:
      storage: 2Gi
 ...
```

第 **5** 步：验证扩展

您可以通过检查 PVC， PV 和 Astra Trident 卷的大小来验证扩展是否正常运行：

```
$ kubectl get pvc san-pvc
NAME      STATUS   VOLUME                                     CAPACITY
ACCESS MODES   STORAGECLASS   AGE
san-pvc   Bound    pvc-8a814d62-bd58-4253-b0d1-82f2885db671   2Gi
RWO            ontap-san      11m
$ kubectl get pv
NAME                                       CAPACITY   ACCESS MODES
RECLAIM POLICY   STATUS   CLAIM             STORAGECLASS   REASON   AGE
pvc-8a814d62-bd58-4253-b0d1-82f2885db671   2Gi        RWO
Delete           Bound    default/san-pvc   ontap-san               12m
$ tridentctl get volumes -n trident
+------------------------------------------+---------+---------------
+----------+--------------------------------------+--------+---------+
|                   NAME                   |  SIZE   | STORAGE CLASS |
PROTOCOL |             BACKEND UUID             | STATE  | MANAGED |
+------------------------------------------+---------+---------------
+----------+--------------------------------------+--------+---------+
| pvc-8a814d62-bd58-4253-b0d1-82f2885db671 | 2.0 GiB | ontap-san     |
block    | a9b7bfff-0505-4e31-b6c5-59f492e02d33 | online | true    |
+------------------------------------------+---------+---------------
+----------+--------------------------------------+--------+---------+
```

展开 **NFS** 卷

Astra Trident 支持对在 `ontap-nas` ， `ontap-nas-economy` ， `ontap-nas-flexgroup` ， `GCP-CVS` 和 `azure-netapp-files` 后端配置的 NFS PV 进行卷扩展。

第 **1** 步：配置 **StorageClass** 以支持卷扩展

要调整 NFS PV 的大小，管理员首先需要将 `allowVolumeExpansion` 字段设置为 `true` 来配置存储类以允许卷扩展：

```
$ cat storageclass-ontapnas.yaml
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: ontapnas
provisioner: csi.trident.netapp.io
parameters:
  backendType: ontap-nas
allowVolumeExpansion: true
```

如果您已创建没有此选项的存储类，则只需使用 `kubectl edit storageclass` 编辑现有存储类即可进行卷扩展。

第 **2** 步：使用您创建的 **StorageClass** 创建 **PVC**

```
$ cat pvc-ontapnas.yaml
kind: PersistentVolumeClaim
apiVersion: v1
metadata:
  name: ontapnas20mb
spec:
  accessModes:
  - ReadWriteOnce
  resources:
    requests:
      storage: 20Mi
  storageClassName: ontapnas
```

Astra Trident 应为此 PVC 创建一个 20 MiB NFS PV：

```
$ kubectl get pvc
NAME           STATUS   VOLUME
CAPACITY     ACCESS MODES   STORAGECLASS    AGE
ontapnas20mb   Bound    pvc-08f3d561-b199-11e9-8d9f-5254004dfdb7   20Mi
RWO            ontapnas        9s

$ kubectl get pv pvc-08f3d561-b199-11e9-8d9f-5254004dfdb7
NAME                                       CAPACITY   ACCESS MODES
RECLAIM POLICY   STATUS   CLAIM                  STORAGECLASS    REASON
AGE
pvc-08f3d561-b199-11e9-8d9f-5254004dfdb7   20Mi       RWO
Delete           Bound    default/ontapnas20mb   ontapnas
2m42s
```

第 **3** 步：展开 **PV**

要将新创建的 20MiB PV 调整为 1GiB ，请编辑 PVC 并将 `sPec.resources.requests.storage` 设置为 1GB：

```
$ kubectl edit pvc ontapnas20mb
# Please edit the object below. Lines beginning with a '#' will be
ignored,
# and an empty file will abort the edit. If an error occurs while saving
this file will be
# reopened with the relevant failures.
#
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  annotations:
    pv.kubernetes.io/bind-completed: "yes"
    pv.kubernetes.io/bound-by-controller: "yes"
    volume.beta.kubernetes.io/storage-provisioner: csi.trident.netapp.io
  creationTimestamp: 2018-08-21T18:26:44Z
  finalizers:
  - kubernetes.io/pvc-protection
  name: ontapnas20mb
  namespace: default
  resourceVersion: "1958015"
  selfLink: /api/v1/namespaces/default/persistentvolumeclaims/ontapnas20mb
  uid: c1bd7fa5-a56f-11e8-b8d7-fa163e59eaab
spec:
  accessModes:
  - ReadWriteOnce
  resources:
    requests:
      storage: 1Gi
...
```

第 **4** 步：验证扩展

您可以通过检查 PVC ， PV 和 Astra Trident 卷的大小来验证调整大小是否正常工作：

```
$ kubectl get pvc ontapnas20mb
NAME           STATUS   VOLUME
CAPACITY   ACCESS MODES   STORAGECLASS    AGE
ontapnas20mb   Bound    pvc-08f3d561-b199-11e9-8d9f-5254004dfdb7   1Gi
RWO            ontapnas        4m44s

$ kubectl get pv pvc-08f3d561-b199-11e9-8d9f-5254004dfdb7
NAME                                       CAPACITY   ACCESS MODES
RECLAIM POLICY   STATUS   CLAIM                  STORAGECLASS    REASON
AGE
pvc-08f3d561-b199-11e9-8d9f-5254004dfdb7   1Gi        RWO
Delete           Bound    default/ontapnas20mb   ontapnas
5m35s

$ tridentctl get volume pvc-08f3d561-b199-11e9-8d9f-5254004dfdb7 -n
trident
+------------------------------------------+---------+---------------
+----------+--------------------------------------+--------+---------+
|                   NAME                   |  SIZE   | STORAGE CLASS |
PROTOCOL |             BACKEND UUID             | STATE  | MANAGED |
+------------------------------------------+---------+---------------
+----------+--------------------------------------+--------+---------+
| pvc-08f3d561-b199-11e9-8d9f-5254004dfdb7 | 1.0 GiB | ontapnas      |
file     | c5a6f6a4-b052-423b-80d4-8fb491a14a22 | online | true    |
+------------------------------------------+---------+---------------
+----------+--------------------------------------+--------+---------+
```

导入卷

您可以使用 `tridentctl import` 将现有存储卷作为 Kubernetes PV 导入。

支持卷导入的驱动程序

下表介绍了支持导入卷的驱动程序及其引入的版本。

驱动程序	版本。
`ontap-NAS`	19.04
`ontap-nas-flexgroup`	19.04
`solidfire-san`	19.04
`azure-netapp-files`	19.04

驱动程序	版本。
GCP-CVS	19.04
ontap-san	19.04

为什么应导入卷?

将卷导入到 Trident 的使用情形有多种:

- 对应用程序进行容器化并重复使用其现有数据集
- 为临时应用程序使用数据集的克隆
- 重建发生故障的 Kubernetes 集群
- 在灾难恢复期间迁移应用程序数据

导入的工作原理是什么?

卷导入过程使用永久性卷声明（ PVC ）文件创建 PVC 。PVC 文件应至少包含 name ， namespace ， accessModes 和 storageClassName 字段，如以下示例所示。

```
kind: PersistentVolumeClaim
apiVersion: v1
metadata:
  name: my_claim
  namespace: my_namespace
spec:
  accessModes:
    - ReadWriteOnce
  storageClassName: my_storage_class
```

使用 `tridentctl` 客户端导入现有存储卷。Trident 通过保留卷元数据并创建 PVC 和 PV 来导入卷。

```
$ tridentctl import volume <backendName> <volumeName> -f <path-to-pvc-
file>
```

要导入存储卷，请指定包含该卷的 Astra Trident 后端的名称以及用于唯一标识存储上的卷的名称（例如: ONTAP FlexVol ， Element Volume ， CVS 卷路径）。存储卷必须允许读 / 写访问，并可由指定的 Astra Trident 后端访问。需要 ` -f` 字符串参数，并指定 YAML 或 JSON PVC 文件的路径。

当 Astra Trident 收到导入卷请求时，系统会在 PVC 中确定并设置现有卷大小。存储驱动程序导入卷后，系统将创建 PV ，并为其创建一个 Claims Ref 。在 PV 中，回收策略最初设置为 `retain` 。Kubernetes 成功绑定 PVC 和 PV 后，将更新回收策略以匹配存储类的回收策略。如果存储类的回收策略为 `delete` ，则在删除 PV 时，存储卷将被删除。

使用 ` -no-manage` 参数导入卷时， Trident 不会在对象的生命周期内对 PVC 或 PV 执行任何其他操作。由于

Trident 会忽略 `-no-manage` 对象的 PV 和 PVC 事件，因此删除 PV 时不会删除存储卷。卷克隆和卷大小调整等其他操作也会被忽略。如果要对容器化工作负载使用 Kubernetes ，但希望在 Kubernetes 外部管理存储卷的生命周期，则此选项非常有用。

PVC 和 PV 中会添加一个标注，用于指示卷已导入以及 PVC 和 PV 是否已管理。不应修改或删除此标注。

Trident 19.07 及更高版本可处理 PV 的连接，并在导入卷时挂载该卷。对于使用早期版本的 Astra Trident 进行的导入，数据路径中不会执行任何操作，卷导入将不会验证是否可以挂载卷。如果卷导入出错（例如 StorageClass 不正确），您可以通过将 PV 上的回收策略更改为 `retain` ，删除 PVC 和 PV 并重试 volume import 命令来恢复。

ontap-nas 和 ontap-nas-flexgroup 导入

使用 `ontap-nas` 驱动程序创建的每个卷都是 ONTAP 集群上的一个 FlexVol 。使用 `ontap-NAS` 驱动程序导入 FlexVol 的工作原理相同。ONTAP 集群上已存在的 FlexVol 可以导入为 `ONONTAP -NAS` PVC 。同样，可以将 FlexGroup vols 导入为 `ontap-nas-flexgroup` PVC 。

> 要由 Trident 导入 ONTAP 卷，必须为 rw 类型。如果卷的类型为 DP ，则为 SnapMirror 目标卷；应先中断镜像关系，然后再将卷导入到 Trident 中。

> `ontap-NAS` 驱动程序无法导入和管理 qtree 。`ontap-nas` 和 `ontap-nas-flexgroup` 驱动程序不允许使用重复的卷名称。

例如，要在名为 `ontap_NAS` 的后端导入名为 `manage_volume` 的卷，请使用以下命令：

```
$ tridentctl import volume ontap_nas managed_volume -f <path-to-pvc-file>

+------------------------------------------+---------+---------------
+----------+--------------------------------------+--------+---------+
|                   NAME                   |  SIZE   | STORAGE CLASS |
PROTOCOL |             BACKEND UUID             | STATE  | MANAGED |
+------------------------------------------+---------+---------------
+----------+--------------------------------------+--------+---------+
| pvc-bf5ad463-afbb-11e9-8d9f-5254004dfdb7 | 1.0 GiB | standard      |
file     | c5a6f6a4-b052-423b-80d4-8fb491a14a22 | online | true    |
+------------------------------------------+---------+---------------
+----------+--------------------------------------+--------+---------+
```

要导入名为 `unmanageed_volume` 的卷（位于 `ontap_NAS` 后端 上），而 Trident 不会管理该卷，请使用以下命令：

```
$ tridentctl import volume nas_blog unmanaged_volume -f <path-to-pvc-file>
--no-manage

+------------------------------------------+---------+---------------
+----------+--------------------------------------+--------+---------+
|                   NAME                   |  SIZE   | STORAGE CLASS |
PROTOCOL |             BACKEND UUID             | STATE  | MANAGED |
+------------------------------------------+---------+---------------
+----------+--------------------------------------+--------+---------+
| pvc-df07d542-afbc-11e9-8d9f-5254004dfdb7 | 1.0 GiB | standard      |
file     | c5a6f6a4-b052-423b-80d4-8fb491a14a22 | online | false   |
+------------------------------------------+---------+---------------
+----------+--------------------------------------+--------+---------+
```

使用 ` -no-manage` 参数时， Trident 不会重命名卷或验证卷是否已挂载。如果卷未手动挂载，则卷导入操作将失败。

先前存在的使用自定义 UnixPermissions 导入卷的错误已得到修复。您可以在 PVC 定义或后端配置中指定 unixPermissions ，并指示 Astra Trident 相应地导入卷。

ontap-san 导入

Astra Trident 还可以导入包含单个 LUN 的 ONTAP SAN FlexVol 。这与 `ontap-san` 驱动程序一致，该驱动程序会为 FlexVol 中的每个 PVC 和 LUN 创建一个 FlexVol 。您可以像在其他情况下一样使用 `tridentctl import` 命令：

- 请输入 `ontap-san` 后端的名称。
- 提供需要导入的 FlexVol 的名称。请记住，此 FlexVol 仅包含一个必须导入的 LUN 。
- 提供必须与 ` -f` 标志结合使用的 PVC 定义路径。
- 可以选择对 PVC 进行管理，也可以选择不对其进行管理。默认情况下， Trident 将管理 PVC 并重命名后端的 FlexVol 和 LUN 。要作为非受管卷导入，请传递 ` -no-manage` 标志。

导入非受管 `ontap-san` 卷时，应确保 FlexVol 中的 LUN 名为 `lun0` ，并已映射到具有所需启动程序的 igroup 。Astra Trident 会自动为受管导入处理此问题。

然后， Astra Trident 将导入 FlexVol 并将其与 PVC 定义关联。Astra Trident 还会将 FlexVol 重命名为 `vc-<uid>` 格式，并将 FlexVol 中的 LUN 重命名为 `lun0` 。

建议导入没有活动连接的卷。如果要导入当前使用的卷，请先克隆该卷，然后再执行导入。

示例

要导入 `ontap_san_default` 后端上存在的 `ontap-san-managed` FlexVol ，请运行 `tridentctl import` 命令：

```
$ tridentctl import volume ontapsan_san_default ontap-san-managed -f pvc-
basic-import.yaml -n trident -d

+------------------------------------------+--------+---------------
+----------+--------------------------------------+--------+---------+
|                   NAME                   |  SIZE  | STORAGE CLASS |
PROTOCOL |             BACKEND UUID             | STATE  | MANAGED |
+------------------------------------------+--------+---------------
+----------+--------------------------------------+--------+---------+
| pvc-d6ee4f54-4e40-4454-92fd-d00fc228d74a | 20 MiB | basic         |
block    | cd394786-ddd5-4470-adc3-10c5ce4ca757 | online | true    |
+------------------------------------------+--------+---------------
+----------+--------------------------------------+--------+---------+
```

ONTAP 卷的类型必须为 RW ，才能由 Astra Trident 导入。如果卷的类型为 DP ，则为 SnapMirror 目标卷；在将卷导入到 Astra Trident 之前，应中断镜像关系。

元素导入

您可以使用 Trident 将 NetApp Element 软件 /NetApp HCI 卷导入到 Kubernetes 集群中。您需要提供 Astra Trident 后端的名称以及卷和 PVC 文件的唯一名称作为 `tridentctl import` 命令的参数。

```
$ tridentctl import volume element_default element-managed -f pvc-basic-
import.yaml -n trident -d

+------------------------------------------+--------+---------------
+----------+--------------------------------------+--------+---------+
|                   NAME                   |  SIZE  | STORAGE CLASS |
PROTOCOL |             BACKEND UUID             | STATE  | MANAGED |
+------------------------------------------+--------+---------------
+----------+--------------------------------------+--------+---------+
| pvc-970ce1ca-2096-4ecd-8545-ac7edc24a8fe | 10 GiB | basic-element |
block    | d3ba047a-ea0b-43f9-9c42-e38e58301c49 | online | true    |
+------------------------------------------+--------+---------------
+----------+--------------------------------------+--------+---------+
```

Element 驱动程序支持重复的卷名称。如果卷名称重复，则 Trident 的卷导入过程将返回错误。作为临时解决策，克隆卷并提供唯一的卷名称。然后导入克隆的卷。

GCP-CVS 导入

要在 GCP 中导入由 NetApp Cloud Volumes Service 支持的卷，请按卷路径而非名称来标识该卷。

要在后端导入名为 gcpcvs_yEppr 的 GCP-CVS 卷，并将卷路径设置为 adrot-joly-swift ，请使用以下命令：

```
$ tridentctl import volume gcpcvs_YEppr adroit-jolly-swift -f <path-to-
pvc-file> -n trident

+------------------------------------------+--------+---------------
+----------+--------------------------------------+--------+---------+
|                   NAME                   |  SIZE  | STORAGE CLASS |
PROTOCOL |             BACKEND UUID             | STATE  | MANAGED |
+------------------------------------------+--------+---------------
+----------+--------------------------------------+--------+---------+
| pvc-a46ccab7-44aa-4433-94b1-e47fc8c0fa55 | 93 GiB | gcp-storage   | file
| e1a6e65b-299e-4568-ad05-4f0a105c888f | online | true    |
+------------------------------------------+--------+---------------
+----------+--------------------------------------+--------+---------+
```

> 卷路径是卷导出路径中： / 之后的部分。例如，如果导出路径为 10.0.0.1 ： /adrot-joly-swift ，则卷路径为 adrot-joly-swift 。

Azure-netapp-files 导入

要在后端导入 azure-netapp-files 卷，该卷名为 azurenetappfiles_40517 ，卷路径为 importvol1 ，请运行以下命令：

```
$ tridentctl import volume azurenetappfiles_40517 importvol1 -f <path-to-
pvc-file> -n trident

+------------------------------------------+---------+---------------
+----------+--------------------------------------+--------+---------+
|                   NAME                   |  SIZE   | STORAGE CLASS |
PROTOCOL |             BACKEND UUID             | STATE  | MANAGED |
+------------------------------------------+---------+---------------
+----------+--------------------------------------+--------+---------+
| pvc-0ee95d60-fd5c-448d-b505-b72901b3a4ab | 100 GiB | anf-storage   |
file     | 1c01274f-d94b-44a3-98a3-04c953c9a51e | online | true    |
+------------------------------------------+---------+---------------
+----------+--------------------------------------+--------+---------+
```

> ANF 卷的卷路径位于： / 之后的挂载路径中。例如，如果挂载路径为 10.0.0.2 ： /importvol1 ，则卷路径为 importvol1 。

准备工作节点

Kubernetes 集群中的所有工作节点都需要能够挂载为 Pod 配置的卷。如果您在其中一个后端使用 `ontap-nas` ， `ontap-nas-economy` 或 `ontap-nas-flexgroup` 驱动程序，则您的工作节点需要 NFS 工具。否则，它们需要使用 iSCSI 工具。

默认情况下，最新版本的 RedHat CoreOS 同时安装了 NFS 和 iSCSI 。

安装 NFS 或 iSCSI 工具后，您应始终重新启动工作节点，否则将卷连接到容器可能会失败。

NFS volumes

协议	操作系统	命令
NFS	RHEL/CentOS	`sudo yum install -y nfs-utils`
NFS	Ubuntu 或 Debian	`sudo apt-get install -y nfs-common`

您应确保 NFS 服务在启动期间启动。

iSCSI 卷

使用 iSCSI 卷时，请考虑以下事项：

- Kubernetes 集群中的每个节点都必须具有唯一的 IQN 。* 这是必要的前提条件 * 。
- 如果将 RHCOS 4.5 或更高版本，或者将 RHEL 或 CentOS 8.2 或更高版本与 `solidfire-san` 驱动程序结合使用，请确保在 ` /etc/iscsi/iscsid.conf` 中将 CHAP 身份验证算法设置为 MD5 。

```
sudo sed -i 's/^\(node.session.auth.chap_algs\).*/\1 = MD5/'
/etc/iscsi/iscsid.conf
```

- 使用运行 RHEL/RedHat CoreOS 和 iSCSI PV 的工作节点时，请确保在 StorageClass 中指定 `discard` mountOption 以执行实时空间回收。请参见 "RedHat 的文档"。

协议	操作系统	命令

iSCSI	RHEL/CentOS	1. 安装以下系统软件包：
		
		`sudo yum install -y lsscsi iscsi-initiator-utils sg3_utils device-mapper-multipath`
		
		2. 检查 iscsi-initiator-utils 版本是否为 6.2.0.877-2.el7 或更高版本：
		
		`rpm -q iscsi-initiator-utils`
		
		3. 将扫描设置为手动：
		
		`sUdo sed -i 's/^ \ ( node.session.scan\).*/\1 = manual/' /etc/iscsi/iscsid.conf`
		
		4. 启用多路径：
		
		`sudo mpathconf -enable -for_multipathd y -find_multipaths n`
		
		ⓘ 确保 `detc/multipath.conf` contains `find_multipaths no` under` efaults `.
		
		5. 确保 `iscsid` 和 `multipathd` 正在运行：
		
		`sudo systemctl enable -now iscsid multipathd`
		
		6. 启用并启动 `iSCSI`：
		
		`sudo systemctl enable —` 现在为 `iSCSI`

协议	操作系统	命令

iSCSI	Ubuntu 或 Debian	1. 安装以下系统软件包：
		
		`sUdo apt-get install -y open-iscsi lsscsi sg3-utils multipath-tools scsitool`
		
		2. 检查 open-iscsi 版本是否为 2.0.877-5ubuntu2.10 或更高版本（对于双子系统）或 2.0.877-7.1ubuntu6.1 或更高版本（对于 Focal ）：
		
		`dpkg -l open-iscsi`
		
		3. 将扫描设置为手动：
		
		`sUdo sed -i 's/^ \ (node.session.scan\).*/\1 = manual/' /etc/iscsi/iscsid.conf`
		
		4. 启用多路径：
		
		`sUdo tee /etc/multipath.conf <←'EOF' defaults {user_friendly_names yes find_multipaths no } eOF sudo systemctl enable -now multipath-tools.service sudo service multipath-tools restart`
		
		ⓘ 确保 `detc/multipath.conf` contains `find_multipaths no` under` efaults `.
		
		5. 确保已启用并运行 `open-iscsi` 和 `multipath-tools` ：
		
		`sudo systemctl status multipath-tools sudo systemctl enable -now open-iscsi.service sudo systemctl status open-iscsi`

> 对于 Ubuntu 18.04 ，您必须先使用 `iscsiadm` 发现目标端口，然后再启动 `open-iscsi` ，iSCSI 守护进程才能启动。您也可以将 `iscsi` 服务修改为自动启动 `iscsid` 。

> 如果您希望了解有关自动员工节点准备的更多信息，这是一项测试功能，请参见 "此处"。

自动工作节点准备

Astra Trident 可以在 Kubernetes 集群中的节点上自动安装所需的 `NFS` 和 `iSCSI` 工具。这是一项 * 测试版功能 * ， * 不适用于 * 生产集群。目前，该功能可用于运行 * CentOS ， RHEL 和 Ubuntu * 的节点。

对于此功能， Astra Trident 提供了一个新的安装标志： ` -enable-node-prep` for installations deployed with `tridentctl` 。对于使用 Trident 运算符的部署，请使用布尔选项 `enableNodePrep` 。

> ` -enable-node-prep` 安装选项指示 Astra Trident 安装并确保在工作节点上挂载卷时 NFS 和 iSCSI 软件包和 / 或服务正在运行。这是一项 * 测试版功能 * ，用于 * 不符合生产条件 * 的开发 / 测试环境。

如果使用 `tridentctl` 部署的 Astra Trident 安装包含 ` -enable-node-prep` 标志，则会发生以下情况：

1. 在安装过程中， Astra Trident 会注册其运行所在的节点。
2. 发出永久性卷请求（ PVC ）请求后， Astra Trident 会从其管理的后端创建 PV 。
3. 在 Pod 中使用 PVC 需要使用 Astra Trident 在 Pod 运行的节点上挂载卷。Astra Trident 会尝试安装所需的 NFS/iSCSI 客户端实用程序，并确保所需服务处于活动状态。此操作会在挂载卷之前完成。

在首次尝试挂载卷时，只会对工作节点进行一次准备。只要 Astra Trident 之外的任何更改均未触及 `nfs` 和 `iscsi` 实用程序，所有后续卷挂载都应成功。

通过这种方式， Astra Trident 可以确保 Kubernetes 集群中的所有节点都具有挂载和连接卷所需的实用程序。对于 NFS 卷，导出策略还应允许挂载该卷。Trident 可以自动管理每个后端的导出策略；用户也可以管理带外导出策略。

监控 Astra Trident

Astra Trident 提供了一组 Prometheus 指标端点，您可以使用这些端点监控 Astra Trident 的性能。

通过 Astra Trident 提供的指标，您可以执行以下操作：

- 保留有关 Astra Trident 运行状况和配置的选项卡。您可以检查操作的成功程度以及它是否能够按预期与后端进行通信。
- 检查后端使用情况信息，并了解在后端配置的卷数量以及占用的空间量等。
- 维护可用后端配置的卷数量的映射关系。
- 跟踪性能。您可以了解 Astra Trident 与后端通信并执行操作所需的时间。

> 默认情况下， Trident 的指标会显示在 `/metrics` 端点的目标端口` 8001 ` 上。安装 Trident 时，这些指标默认为 * 已启用 * 。

您需要什么？ **#8217**；将需要什么

- 安装了 Astra Trident 的 Kubernetes 集群。
- 一个 Prometheus 实例。可以是 "容器化 Prometheus 部署" 或者，您也可以选择将 Prometheus 作为运行 "原生应用程序"。

第 1 步：定义 Prometheus 目标

您应定义一个 Prometheus 目标以收集指标并获取有关后端 Astra Trident 管理的信息，它创建的卷等。这 "博客" 介绍如何将 Prometheus 和 Grafana 与 Astra Trident 结合使用来检索指标。博客介绍了如何在 Kubernetes 集群中以操作员身份运行 Prometheus ，以及如何创建 ServiceMonitor 来获取 Astra Trident 的指标。

第 2 步：创建 Prometheus ServiceMonitor

要使用 Trident 指标，您应创建一个 Prometheus ServiceMonitor ，该监控器可监控 `trident CSI` 服务并侦听 `mtrigics` 端口。示例 ServiceMonitor 如下所示：

```
apiVersion: monitoring.coreos.com/v1
kind: ServiceMonitor
metadata:
  name: trident-sm
  namespace: monitoring
  labels:
      release: prom-operator
  spec:
    jobLabel: trident
    selector:
      matchLabels:
        app: controller.csi.trident.netapp.io
    namespaceSelector:
      matchNames:
      - trident
    endpoints:
    - port: metrics
      interval: 15s
```

此 ServiceMonitor 定义会检索 `trident CSI` 服务返回的指标，并专门查找服务的 `mtrics` 端点。因此，Prometheus 现在已配置为了解 Astra Trident 的指标。

除了直接从 Astra Trident 获得的指标之外， kubelet 还通过自己的指标端点公开了许多 `kubelet_volume_*` 指标。Kubelet 可以提供有关已连接的卷， Pod 及其处理的其他内部操作的信息。请参见 "此处"。

第 3 步：使用 PromQL 查询 Trident 指标

PromQL 非常适合创建返回时间序列或表格数据的表达式。

您可以使用以下 PromQL 查询：

获取 **Trident** 运行状况信息

- 来自 Astra Trident 的 HTTP 2XX 响应的百分比

```
(sum (trident_rest_ops_seconds_total_count{status_code=~"2.."} OR on()
vector(0)) / sum (trident_rest_ops_seconds_total_count)) * 100
```

- 通过状态代码来自 Astra Trident 的 REST 响应的百分比

```
(sum (trident_rest_ops_seconds_total_count) by (status_code)  / scalar
(sum (trident_rest_ops_seconds_total_count))) * 100
```

- * 由 Astra Trident 执行的操作的平均持续时间（毫秒） *

```
sum by (operation)
(trident_operation_duration_milliseconds_sum{success="true"}) / sum by
(operation)
(trident_operation_duration_milliseconds_count{success="true"})
```

获取 **Astra Trident** 使用信息

- 卷大小 * 平均值 *

```
trident_volume_allocated_bytes/trident_volume_count
```

- * 每个后端配置的卷总空间 *

```
sum (trident_volume_allocated_bytes) by (backend_uuid)
```

获取单个卷的使用情况

> 只有在同时收集 kubelet 指标时，才会启用此功能。

- * 每个卷的已用空间百分比 *

```
kubelet_volume_stats_used_bytes / kubelet_volume_stats_capacity_bytes *
100
```

了解有关 **Astra Trident AutoSupport** 遥测的信息

默认情况下， Astra Trident 会按每日节奏向 NetApp 发送 Prometheus 指标和基本后端信息。

- 要阻止 Astra Trident 向 NetApp 发送 Prometheus 指标和基本后端信息，请在 Astra Trident 安装期间传递 `-silning-autosupport` 标志。
- Astra Trident 还可以通过 `tridentctl send AutoSupport` 按需将容器日志发送到 NetApp 支持部门。您需要触发 Astra Trident 以上传其日志。在提交日志之前，您应接受 NetApp 的https://www.netapp.com/company/legal/privacy-policy/["隐私政策"^]。
- 除非另有说明，否则 Astra Trident 会从过去 24 小时提取日志。
- 您可以使用 `-since` 标志指定日志保留时间范围。例如： `tridentctl send AutoSupport -sase=1h` 。此信息通过与 Astra Trident 一起安装的 `trident — autosupport` 容器进行收集和发送。您可以从获取容器映像 "Trident AutoSupport"。
- Trident AutoSupport 不会收集或传输个人身份信息（ PiII ）或个人信息。它附带了 "EULA" 这不适用于 Trident 容器映像本身。您可以详细了解 NetApp 对数据安全和信任的承诺 "此处"。

Astra Trident 发送的有效负载示例如下：

```
{
  "items": [
    {
      "backendUUID": "ff3852e1-18a5-4df4-b2d3-f59f829627ed",
      "protocol": "file",
      "config": {
        "version": 1,
        "storageDriverName": "ontap-nas",
        "debug": false,
        "debugTraceFlags": null,
        "disableDelete": false,
        "serialNumbers": [
          "nwkvzfanek_SN"
        ],
        "limitVolumeSize": ""
      },
      "state": "online",
      "online": true
    }
  ]
}
```

- AutoSupport 消息将发送到 NetApp 的 AutoSupport 端点。如果使用私有注册表存储容器映像，则可以使用 `-image-regRegistry` 标志。
- 您也可以通过生成安装 YAML 文件来配置代理 URL 。为此，可以使用 `tridentctl install -generate-custom-yaml` 创建 YAML 文件，并在 `trident dedeployment.yaml` 中为 `trident autosupport` 容器添加 `-proxy-url` 参数。

禁用 **Astra Trident** 指标

要 ` 报告指标，您应生成自定义 YAML （使用` -generate-custom-yaml ` 标志）并对其进行编辑，以删除为 `trident` 主 容器调用的` -metrics 标志。

版权信息

版权所有©2022 NetApp、Inc.。保留所有权利。Printed in the U.S.版权所涵盖的本文档的任何部分不得以任何形式或任何手段复制、包括影印、录制、 磁带或存储在电子检索系统中—未经版权所有者事先书面许可。

Software derived from copyrighted NetApp material is subject to the following license and disclaimer:

本软件由NetApp按"原样"提供、不含任何明示或默示担保、包括但不限于适销性和特定用途适用性的默示担保、特此声明不承担任何责任。IN NO EVENT SHALL NETAPP BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

NetApp reserves the right to change any products described herein at any time, and without notice.NetApp assumes no responsibility or liability arising from the use of products described herein, except as expressly agreed to in writing by NetApp.The use or purchase of this product does not convey a license under any patent rights, trademark rights, or any other intellectual property rights of NetApp.

The product described in this manual may be protected by one or more U.S.patents, foreign patents, or pending applications.

RESTRICTED RIGHTS LEGEND: Use, duplication, or disclosure by the government is subject to restrictions as set forth in subparagraph (c)(1)(ii) of the Rights in Technical Data and Computer Software clause at DFARS 252.277-7103 (October 1988) and FAR 52-227-19 (June 1987).

商标信息

NetApp、NetApp标识和中列出的标记 http://www.netapp.com/TM 是NetApp、Inc.的商标。其他公司和产品名称可能是其各自所有者的商标。